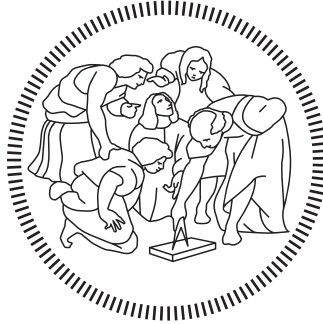


POLITECNICO DI MILANO
Scuola di Ingegneria Industriale e dell'Informazione
Corso di Laurea Magistrale in Ingegneria Informatica
Dipartimento di Elettronica, Informazione e Bioingegneria



Devil is in the Detail
A compiler based solution to microarchitecture-induced side
channel leakages

Advisor: Prof. Alessandro Barenghi
Co-Advisor: Prof. Gerardo Pelosi

Thesis by:
Lorenzo Casalino Matr. 877421

Academic Year 2019–2020

Fortuna Audentes Juvat.

Author's Thanks

A due thank goes to my advisor, Barengi Alessandro, and co-advisor, Pelosi Gerardo, who, a part for having guided me since the beginning of this long research project, tirelessly answered to all my doubts and questions. Luca, Fabrizio, Valerio and Marco, thanks for all the bad and good moments spent together, but, most importantly, thank for having supported me. Fabrizio (Cremonesi), Eleonora and Nipun thank you: you are ones of few that really believe in me and my abilities, despite of what other people think of me. Nicolò, thank you: we met just two month ago, but your help has been really precious. Silvia and Gianluca: even if you were, literally, on the other side of Earth, you have always been there for me. Your support is invaluable. A special thank is dedicated to my parents, Patrizia and Gianfranco, that supported me through all these years. Finally, Mattia, my brother: I did it, and it's all thanks to you.

Sommario

Il *masking* è una contromisura ampiamente studiata per la protezione di cifrari simmetrici contro attacchi side-channel passivi: la suddivisione dei valori sensibili in $n + 1$ *shares*, con n ordine del mascheramento, riduce il legame statistico tra i primi e i parametri fisici misurati (e.g., emissione elettromagnetica) di un'architettura ospitante l'implementazione mascherata; idealmente, solo la conoscenza di tutte le shares di un dato valore permette il recupero di quest'ultimo. Tuttavia, è noto come l'attività di commutazione dei registri (micro-)architetturali sia in grado di rilasciare un contenuto informativo dipendente dalla combinazione di più shares, riducendo la complessità di un attacco. Il presente documento propone una metodologia per sfruttare gli algoritmi di code-generation adottati nei compilatori, impedendo assegnamenti di registri o ri-organizzazioni del codice che potrebbero violare la sicurezza fornita dal masking. Differentemente dai lavori precedenti, in cui il compilatore può ritrovarsi nella situazione di non poter allocare nessun registro, o schedare nessuna istruzione, fallendo la compilazione, un eventuale stallo viene ovviato mediante *sbiancamento* degli elementi (micro-)architetturali, riciclando attentamente le shares già compute. L'approccio ideato permette di diminuire la complessità temporale dell'implementazione, e minimizzare la domanda di entropia, due parametri critici per piattaforme hardware con risorse limitate, tipiche nell'ambito Internet of Things. La metodologia è implementata come estensione di CryptoShield, un framework basato su LLVM per il mascheramento parziale automatizzato delle implementazioni software di cifrari simmetrici. Una prima analisi della metodologia viene fornita con la valutazione dell'overhead introdotto sulle performance di un'implementazione di AES-128 con T-Table, mascherato al primo e secondo ordine, ed eseguito sull'architettura super-scalare Cortex-A7. Il documento si conclude delineando possibili sviluppi futuri, sia nel metodo, notando come sia possibile estendere il metodo ad architetture più complesse, sia nell'implementazione, per esempio, introducendo la vettorizzazione delle operazioni.

Abstract

Masking is a countermeasure widely studied for the protection of symmetric ciphers against passive side-channel attacks: sensitive values are split in $n + 1$ *shares*, with n the masking order, making negligible the statistical relation between the first and the measured physical parameters (e.g., electromagnetic emission) of an architecture running the masked implementation; ideally, only the knowledge of all the shares of a given value unveils it. Nevertheless, it is well known that the switching activity of (micro-)architectural registers is able to release an information content dependent on more than one share, reducing the effort for a successful attack. This work introduces a methodology to exploit the code-generation algorithms encompassed in compilers, forbidding register assignments and instruction stream re-organizations that might undermine the security provided by masking. Differently from previous works, where the compiler may be unable to allocate a register, or schedule an instruction, failing the compilation, the compilation failure is overcome by means of *whitening* of the (micro-)architectural elements, carefully recycling the already computed shares. Such an approach enables a lower time complexity for the implementation, while minimizing the randomness demand, two critical parameters for resource-constrained devices, typical in the Internet of Things landscape. The methodology is implemented as an extension of CryptoShield, a LLVM based framework for automating the partial masking of software symmetric ciphers implementations. A performance analysis of the methodology is given with an evaluation of the implied overhead on the performances of a AES-128 with T-Table implementation, masked at first and second order, and executed on the Cortex-A7 super-scalar architecture. This work concludes by outlining possible future work directions, either in the method, highlighting how it may be readily extended to more complex architectures, and in the implementation, for instance introducing vectorization.

Contents

Sommario	i
Abstract	i
List of Figures	vii
List of Tables	vii
Listings	ix
Introduction	1
1 Background and Current State of the Art	6
1.1 Masking Schemes	6
1.1.1 Notation	7
1.1.2 Theoretical preliminaries	7
1.1.3 Open Challenges in Masking Scheme Design	10
1.2 Higher-Order Boolean Masking Schemes	11
1.2.1 Taxonomy for Higher-Order Boolean Schemes	11
1.2.2 Re-computation based approach	13
1.2.3 Decomposition based approach	14
1.3 Memory Transitions against Software Masking	19
1.3.1 Power Consumption in CMOS	19
1.3.2 Hamming-Distance Power Model	20
1.3.3 Assumptions on Leakage and Software Masking	21
1.3.4 Micro-architectural Memory-Transition Leakage	24
1.3.5 Related Works	26
1.4 Code Generation in LLVM	26
1.4.1 LLVM: an Overview	26
1.4.2 Target-Independent Code Generator	27
1.4.3 Instruction Selection	28
1.4.4 Basic Register Allocator	29
1.4.5 Machine Scheduler	30
1.5 CryptoShield	33

1.5.1	Overview	33
1.5.2	Pipeline	34
1.5.3	Features Recognition	34
1.5.4	Attack Points Selection	35
1.5.5	Propagation	35
1.5.6	Filtering and Aggregation	35
1.5.7	Target Masking	35
1.6	Boolean masking schemes in CRYPTOSHIELD	36
1.6.1	ISW AND - Rivain-Prouffs variant	36
1.6.2	Coron's Higher-Order Look-up Table Masking	38
2	A Compiler-based Solution	49
2.1	Methodology	49
2.2	Computing Architecture	51
2.3	Tagging	53
2.3.1	Tagging Masked Transformations	53
2.3.2	Tagging the RP scheme	54
2.3.3	Tagging the CORON scheme	57
2.4	BASIC Register Allocator Hardening	60
2.5	MACHINE SCHEDULER Hardening	61
3	Performance Analysis and Results	72
3.0.1	Target Architecture: Cortex-A7	73
3.0.2	OlinuXino A20	74
3.1	Implementation Aspects	74
3.1.1	Intrinsics and Tags Assignment	75
3.1.2	TABLEGEN: Micro-Architectural Buffers Modelization	76
3.1.3	Whitening Strategy for CORTEX-A7 - Post-RA Scheduling	77
3.2	Performance Evaluation	80
3.2.1	Experimental Parameters	80
3.2.2	Results	81
	Conclusions and Future Works	83
	Bibliography	89

List of Figures

1.1	Masking Schemes - Taxonomy's Structure	11
1.2	CMOS inverter.	20
1.3	Secure Data-Flow Framework	34
2.1	Generic computing architecture	52
3.1	Cortex-A7 Pipeline Structure	73
3.2	Pipeline Issue History Legend	78
3.3	Leakage occuring on the first lane.	78
3.4	Leakage occurring on the first lane (dual-issue case).	80
3.5	Leakage occurring on the second lane.	80
3.6	Clock cycles - Order graphs	84
3.7	Size - Order graphs	84

List of Tables

1.1	Minimal notation for Masking Schemes description.	7
1.2	HO Masking Schemes - Classification	13
1.3	Recomputation methods - Asymptotical Complexities (Large Registers)	15
1.4	CMOS Inverter - Currents drawn and switching activity.	20
1.5	CMOS Memory Register - Transitions and Dynamic Power	21
3.1	Cortex-A7 Microarchitectural Buffers - Leakage Description.	74
3.2	Prefetcher/D\$/I\$ Performance Impact	83
3.3	Timings performances on CORTEX-A7 in clock cycles	83

3.4	Timings performances on CORTEX-A7 in seconds	85
3.5	AES T-TABLE implementations - Sizes of <code>.text</code> segment	85
3.6	Relative % overhead (mean execution time) of codegen protections .	86
3.7	Relative % overhead (size of <code>.text</code> segment) of codegen protections	86

List of Algorithms

1	ISW $\mathbb{F}_2$ AND	22
2	Leakage Aware Topological Sorting	25
3	Basic Register Allocator	42
4	checkInterference - Look for overlapping virtual registers	43
5	checkHazard - Check for Structural Hazards.	43
6	getRegions - Divide Basic Block in Sub-Blocks.	44
7	pickBest - Retrieve the best available instruction.	45
8	MachineScheduler - Main Logic	46
9	ISW AND - Rivain-Prouff variant	47
10	Coron's Higher-Order Look-Up Table Masking	47
11	Rivain and Prouff's Mask Refresh	48
11		55
12	Coron's Higher-Order Look-Up Table Masking	58
13	checkInterference (leakage-aware)	62
14	checkLeakage - Verifying potential leakage in μ arch buffers	65
15	updateBuffers - Update micro-architectural buffers	68
16	updateTags - Update tracked tags (pre-RA version)	68
17	pickBest - Get best ready instruction (Leakage-aware)	70
18	MachineScheduler - Main Logic (Leakage aware)	71
19	Post-RA whitening - ARM-specific Approach	79

Listings

1.1	Thumb2 ISW AND	23
-----	--------------------------	----

Introduction

The *Internet of Things* is a revolutionary computational paradigm, introducing new opportunities in several fields, e.g., industry and healthcare, but also security-related issues [48].

Characterized by a tremendous number of interconnected, heterogeneous resource-constrained (i.e., low computational power, storage, battery) devices, locatable in any arbitrary place, IoT applications process and exchange large volume of data every instant. The security of such data is of primary importance, in particular for those settings where data confidentiality, integrity, privacy, authentication, authorization, device trust, device identification, device update and access control are paramount [48, 34, 60].

As entities dealing with valuable asset, i.e., information, IoT devices are subject to different attacks, either of software and hardware nature [34, 23, 62, 24]. Among the several physical threats to which IoT devices are exposed (e.g., hardware trojans), the so-called *passive side-channel attacks* are one of the most subtle menaces: by measuring a physical characteristic of the target device (e.g., electro-magnetic emanation, instantaneous power consumption), the attacker is given enough information to recover secrets held by the target platform.

An extensively studied class of passive side-channel attacks is the *differential power analysis* attack [36, 41]. In a differential power analysis attack, in brief DPA, the attacker exploits the potential correlation binding the instantaneous power consumption of the platform and the intermediate values manipulated by the algorithm running on top of the device. Several runs of the algorithm under analysis are executed; for each of them, *leakage traces* – traces sampling the power consumed in a time interval by the platform – are acquired, and eventually analyzed via statistical methods, to discern dependences with secrets kept by the device, e.g., the cryptographic key. To improve statistical analysis, *higher-order* attacks are mountable, by employing more than one sample from the same leakage trace; such number determines the *order* of the attack.

Existing countermeasures endeavour to remove data-power dependency by randomizing the power measured on each trace, making it constant throughout the runs, or reducing the signal-to-noise ratio [41, 57].

In the last two decades, the randomization technique known as *masking* [13, 41], thanks to its simple idea and mathematical formalization of security,

sprung particular attention from the cryptology community, quickly enestablishing as an effective countermeasure against differential power analysis attacks.

Differently from other randomization techniques, masking acts at *algorithmic* level, proceeding in two steps: firstly, each sensitive data, e.g., the cryptographic key, is split into several pieces – called *shares* – such that any strict subset of these pieces is statistically independent on the original secret. Secondly, each computation of the algorithm is modified, in order to process the shared representation of the inputs, while keeping the semantic of the program. The number of shares determines the *order* of the masking: a masking of order n employs $n + 1$ shares. According to the mathematical operator employed for sharing the secrets, i.e., exclusive OR or an arithmetic operation, masking is said to be, respectively, *boolean* and *arithmetic*.

Higher-order masking has been shown to be a sound method for counter-acting higher-order DPA: in particular, n -th order masking is able to withstand DPA of order at most n [13].

Due to its nature, masking is, in principle, suitable either for the hardware and software setting; the scientific literature is overwhelmed by application of masking to several hardware and software cryptographic schemes, e.g., [30, 51].

Despite its simple nature, manual implementation and verification of masking schemes is a tedious and error-prone work. To overcome such burden, tools for automating such activities started to appear in the literature, e.g., [1, 44, 7, 5, 11]. Regarding automated synthesis, most of the works endeavoured to correctly mask implementation, but did not consider physical phenomena that may thwart the security they provide; in particular, this is true for tools employed in automated masking of software algorithms, where the software abstraction makes hard considering low-level leaking details [44].

In the hardware setting, *glitches*, delays in the propagation of signals in combinatorial sections of circuits, leak an amount of power through which an attacker may successfully mount a DPA attack [41, 45].

Concerning the software side, the power leaked by *memory transitions* have been proved to be a valid attack vector for differential power analysis [41, 20, 2]. Memory transitions are a really subtle problem for software implementations, since not only reuse of ISA registers may leak informations, but also memory elements of the micro-architecture, e.g., bus, pipeline buffers, functional unit buffers and the like [4, 56]. Only in the last three years, attempts to provide approaches for sound implementation of masking in software, addressing leakages stem from micro-architectural elements, started to appear [54, 56, 61].

Contribution

The hereby thesis aims to improve the state-of-the-art on automated masking approaches, envisioning a pro-active methodology for tackling hardening of boolean masked software implementations, warding off leakages stemming from the (micro-)architectural buffers and registers of a partial dual-issue, super-

scalar CPU: the ARM Cortex-A7 [38]. Based on the *Secure Data Flow Analysis* [1] and the primer of Seuschek, De Santis, and Guillen [54], register allocators and instruction schedulers algorithms are instructed to enforce specific *leakage-aware* policies. Differently from the work of Seuschek, De Santis, and Guillen, the new methodology is *non-blocking*: when the algorithm is not able to allocate or schedule an instruction due to leakage, instead of failing the compilation, a *whitening* operation is performed – register or buffer is overwritten with a random value – and the compilation resumed. Such step differs for register-allocation and scheduling, and requires a careful design to handle subtle cases that may block the compilation, introduce weaknesses or semantically break the program. The methodology is implemented as an extension of *CryptoShield* [52], leveraging the underlying LLVM [37] code-generation algorithms as case study for the leakage-aware policies. To the best of my knowledge, the resulting tool is the first of its kind to generate, from a C specification of an unprotected cryptosystem and the desired masking order, a (higher-order) boolean masked software implementation resilient to leakages derived from micro-architectural buffers of a multiple-issue, super-scalar architecture. The implementation is empirically validated performing side-channel measurements on the resulting masked implementation of the AES cryptosystem [46].

Summary of Related Work

In the last decade, several works faced the problem of automating application of masking to software implementation of cryptographic algorithms. The work of Moss et al. [44] is the first successful attempt to address automatic synthesis of masked software implementations. Characterized by a secrecy-aware type-system and a type-inference engine, the proposed compiler processes a (potentially) unprotected software specification of a cryptosystem, explicitly annotating secret variables, and automatically apply type-inference rules to identify faulty computations, i.e., computation insecurely processing secret-related inputs. Repairing rules are employed to enforce security when the type-inference engine identify a leaking operation. The output of the compiler is a first-order secure boolean masked implementation against DPA. Barthe et al. provides a proof-of-concept synthetic tool [7], able to process a program specification written in a subset of C programming language, and produce a higher-order masked implementation. The tool is parameterized on the masking order. In [61], Wang, Sung, and Wang make a step towards DPA hardening in case of register overlapping of related shares, by implementing a LLVM-based tool where a Datalog-based type-inference system allows detection of leakages not only from a high-level point of view, but even at low-level, constraining the register allocator to forbid same register allocation to shares masking the same intermediate value.

Eldib and Wang propose *SCMasker* [27], a refinement-based tool for synthesis of perfectly masked software implementation from unprotected C-

based specifications. From the input description, the design-space of possible masked implementation is incrementally explored, checking if the candidate solution satisfies perfect masking and functional equivalence. Luo et al. follow an approach similar to [27], envisioning *CATI* [39], a LLVM/SAT-based tool for automated application of *Threshold Implementation* scheme (schemes are discussed in the Background chapter) to a software specification.

With ROSITA [56], Shelton et al. provide a code-rewrite engine able to modify a masked software implementation, actively removing leakage arising from micro-architectural elements of the target platform. The code-rewrite engine is supported by a leakage emulator, in order to profile the leakage behavior of the target platform, making possible identification of leaking instructions. Seuschek, De Santis, and Guillen tackle the leakage derived from ISA registers and micro-architectural buffers [54]. Ad-hoc code-generation tools for register allocation and instruction scheduling are designed and instructed to avoid particular register assignment and re-organization of the input instruction stream, generating first-order secure implementations. Either works assume a masked implementation in input, which does not require explicit annotation of secrets.

Agosta et al. propose an data-flow-based approach to the hardening of software cryptographic implementations [1]. Such method, named *Secure Data-Flow Analysis*, employs the data-flow analysis of an annotated software cryptoscheme, to identify which intermediate variables are dependent on the secrets (e.g., round keys) and, for each of them, identify and apply masking solely to the most likely attack points. As in [7], the tool is parameterized on the security order. In 2016, Sanfilippo envisions *CryptoShield* [52], a LLVM-based middle-end library implementing the SDFA [1], able to apply higher-order boolean and arithmetic masking.

Chapter 1

Background and Current State of the Art

The present chapter provides the reader with the fundamental background to understand the goal of this research work, as well as the tools and strategies employed, and problematics faced: Section 1.1 provides an introduction on *masking*, a generic countermeasure against differential power/EM attacks; Section 1.2 reviews the most prominent *higher-order boolean* masking schemes existent in literature, providing an improved taxonomy of these schemes. The leakage induced by the switching activity of CMOS based components, such as registers, is briefly outlined in Section 1.3, along with an explanation of how such leakage may thwart the security of masked software implementations. Section 1.4 introduces the LLVM’s code-generation tools employed for the development of a compiler based solution against micro-architectural attacks. An overview of the CRYPTOSHIELD library, which represents the based for this project, is provided in Section 1.5, describing the overall functionality and the pipeline of phases composing it. Finally, to provide the reader with the required knowledge for understanding the security proofs exhibited in Chapter 2, in Section 1.6 the boolean masking schemes employed in CRYPTOSHIELD are reported, together with their formal security proofs.

1.1 Masking Schemes

Masking is a really wide, deep and fast-pace evolving methodology, counting several scientific publications each year. Among the several related sub-topics, *generic higher-order* schemes, thanks to their security guarantees against higher-order attacks and applicability to any arbitrary algorithm, are extensively studied. The following section briefly recap masking, introducing primarily concepts revolving around it – such as masking schemes, and security models – and main challenges for masking scheme designers.

Table 1.1: Minimal notation for Masking Schemes description.

SYMBOL	MEANING
$\mathbb{S}$	A set.
$s \in \mathbb{S}$	An element belonging to $\mathbb{S}$.
F	Function or algorithm.
$\bar{a}$	An encoding of $a = \{s \in \mathbb{S}, F\}$
$\bar{a}_i$	i-th component of $\bar{a}$.
$\bar{a}^i$	$\bar{a}$ lacking of i-th component.
$\odot$	Generic binary operator.
$\oplus$	Linear field addition.
$\llbracket \bar{a} \rrbracket = \odot_{0 \leq i \leq n} a_i$	Recombination of $\bar{a}$'s components.
$\mathcal{O}(\cdot)$	Big-O notation (complexity)
$\mathcal{U}_{2^k}$	Uniform random distribution over elements of the set $\{0, 1\}^k$.
$\leftarrow^{\$}$	Extraction from a uniform random distribution

1.1.1 Notation

Throughout this section, a minimal notation is used. Most of the time, the symbols are explained in place, or made clear by the context. However, as a reference, Table 1.1 reports the main notation employed.

1.1.2 Theoretical preliminaries

Masking The rationale behind masking is to *mask* the correlation existing between the data-processing operations of a crypto routine and the power consumed by the device hosting the implementation. This operation takes the form of a (n, n) -secret-sharing [55] of the cryptographic algorithm, where each intermediate value produced and processed is concealed by a tuple $\bar{x}$ of $n \geq 2$ random values $\bar{x}_i$, called *shares*, such that *correctness* and n^{th} -order *independence* are satisfied; generally, $n - 1$ shares out of n are effectively drawn from a random distribution, while $x_0 = \odot_{1 \leq i \leq n} x_i$.

Definition 1. Correctness

Given any finite set $\mathbb{X}$, a secret sharing of an arbitrary value $x \in \mathbb{X}$ is said to be correct if and only if $x = \llbracket \bar{x} \rrbracket = \odot_{0 \leq i \leq n} x_i$, where n is the number of $\bar{x}_i$ values randomly drawn from $\mathbb{X}$ and $\odot$ is some group law.

Definition 2. n^{th} -order Independence

Given any finite set $\mathbb{X}$ and an encoding $\bar{x}$ of any $x \in \mathbb{X}$, $\bar{x}$ is said to satisfy n^{th} -order independence if and only if any subset $\mathbb{S} := \{\bar{x}_i\} \wedge |\mathbb{S}| \leq n$ is statistically independent on the original value x .

The tuple $\bar{x}$ is called *probabilistic encoding* of x , or *encoding*; the number $n - 1$ of randomly drawn values composing the encoding is called *order* of the mask. When $n > 2$, the masking is said to be of *higher-order*.

According to the group law employed, masking is denoted as *boolean*, where the *exclusive-OR* is used to split the raw intermediate, or *arithmetic*, which uses modular addition or multiplication. The choice of using boolean or arithmetic masking highly depends on the operations employed by the cryptoscheme; it may happen that either boolean and arithmetic operations are present, requiring both forms of masking. In such case, it is fundamental to envision efficient, and secure, systems to convert boolean encoded intermediates into arithmetic versions, and viceversa, e.g., [15].

Masking Schemes When masking is employed as a countermeasure against power analysis, it is not sufficient to mask each intermediate value: the operations working on them must be converted, in order to accept, and produce, encoding of values and preserving the input-output semantic. Such conversion, or *randomization*, is described by algorithms known as *masking schemes*. Without loss of generality, in the following definitions operations are modeled as algorithms. Algorithms are simply modeled as finite sequences of atomic computations; algorithms composed by other algorithms may re-defined as *inlined* version (each sub-algorithm is replaced by the sequences of atomic computations composing it).

Definition 3. Masking Scheme

Given an arbitrary k -ary algorithm $A(x_1, x_2, \dots, x_k)$, a n^{th} -order masking scheme M is any first-class algorithm such that

$$\bar{A} = M(A) \wedge \forall \bar{A}_j \in \bar{A} : \exists C_i \in A : \bar{C}_j = M(C_i)$$

where:

- $C_i(y_1, y_2, \dots, y_m) \in A$ is an atomic computation
- $\bar{A}(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_k)$ is the $(k * n)$ -ary randomized version of A
- $\bar{C}_i(\bar{y}_1, \bar{y}_2, \dots, \bar{y}_m)$ is the $(k * n)$ -ary randomized versions of C_i .

The result of the randomization is called *gadget*.

Definition 4. Gadget

Given an arbitrary k -ary algorithm $A(x_1, x_2, \dots, x_k)$ and a n^{th} -order masking scheme M , the randomization $\bar{A}(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_k) = M(A)$ of A is called *gadget*.

The type of operation – linear or non-linear – highly influences the features, e.g., efficiency, of a masking scheme. While linear operations may be easily randomized by applying the operation on each share independently, non-linear operations are harder to mask; masking schemes designers must carefully envision their protection, in order to guarantee the n^{th} -order independence.

One of the first works to put forward a solid and concrete theoretical framework, for development and analysis of generic and secure scheme, has been provided by Ishai, Sahai, and Wagner [33], which describes a generic algorithm for the randomization of any boolean circuit. From this groundbreaking work, countless others have been proposed, each tackling new issues, or improving over previous constructions.

Security of Masking Schemes The effectivity of masking against power analysis was formally analyzed by Chari et al. [13], proving that the number m of observations an attacker is required to perform – defined as *order* of the attack – to mount a successful statistical attack, is lower-bounded exponentially on the number n of shares, in which each sensitive value is split; thus, the attack order defines a security parameter, called *security order* of the mask, for evaluating the resistance of a masking scheme. The proof was supported by a modelization of the leaking behaviour of the device and of the attacker’s capabilities. This formal approach laid the foundations for the sound verification of schemes’ security. Furthermore, a notion of security for masking scheme is introduced: a masking scheme is said to be secure *if and only if* an attacker, exploiting a side-channel, is able to retrieve the same information about the secrets that it would derive observing only the inputs and outputs, i.e., no knowledge on the device’s internal state is employed.

Willing to detail the security of a masking scheme, we need to analyze the *security model* under which it is proven secure. Such model is informally defined as the combination of three orthogonal notions [5]:

- Execution model. It describes the actual computations performed by the protected algorithm.
- Leakage model. It describes the amount of information leaked by a single computation.
- Adversarial model. It describes the amount of information a potential attacker may obtain from a complete execution of the protected implementation.

Several security models have been proposed through the years. Two of the best known are the *noisy* model [13] and the *t-probing* model [33]. In the noisy model, the attacker is able to retrieve a *noisy* view – affected by some independent gaussian noise – of the power consumed by the device during an algorithm invocation.

In the t -probing model, instead, an attacker the attacker is said to be *t-limited*: (s)he is able to perform a limited number of observations in each computational instant (e.g., each clock cycle), up to a threshold t . Each observation is not affected by any noise, and reveals the entire information content of the corrupted value.

1.1.3 Open Challenges in Masking Scheme Design

Overheads Masking is not a cheap solution, and comes with a non-negligible footprint. While randomization of linear operations incur in a time, space and randomness overheads linear in the masking order, non-linear transformations are the ones driving the costs, which are quadratic in the masking order. For most of the modern cryptographic schemes include some non-linear elements, i.e., substitution boxes – hence, any randomization faces a quadratic blow-up of the complexities – and since implementations are generally hosted on resource constrained devices, several works tried to conceive new masking approaches, e.g., lightweight masking [9], or techniques to improve the bounds on the costs. Regarding the latter, the most prominent generic higher-order methods, for boolean masking, are presented in Section 1.2.

Composition Most of the works in literature focus on randomize a specific non-linear transformation, e.g, field multiplication, certifying its security under a given security model. However, despite the sound proof, their security guarantees, most of the times, are not verified under the composition with other gadgets. As a consequence, schemes adopting such gadgets has suffered reductions on the claimed security level, e.g., [49, 22].

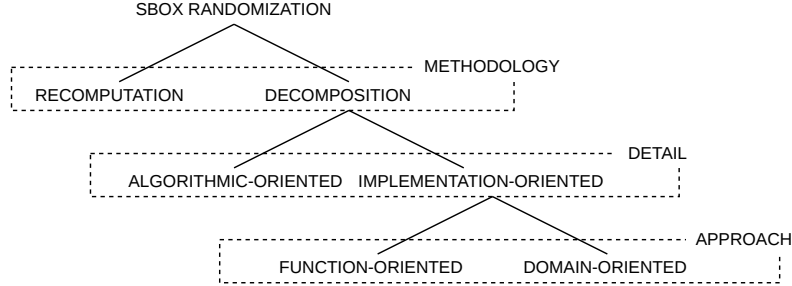
An important step towards certification of secure composition is given by Barthe et al., which propose a stronger refinement of security under the t -probing model, known as *t-strong non-interference*(t-SNI) [7].

Definition 5. *t-strong non-interference* A gadget G is said to be *t-strong non-interfering* if, given a set of observed computations $t_i + t_o \leq t$ (with t_i computations internal to the gadget and t_o computations related to G 's output), the number of inferrable G 's inputs shares is limited by the number of internal observations t_i .

Informally, a gadget satisfying such property breaks the relation between input and output encoding, i.e., output encoding is independent on inputs, making unfeasible attacks on higher-order implementations exploiting compositions.

Non-ideal Behavior of the Hardware Some of the most well-known masking schemes are proved to be secure under an ideal leakage model: the target platform leaks information only about the observed value. Sometimes, this

Figure 1.1: Masking Schemes - Taxonomy's Structure



statement is referred to as *independent leakage assumption* [2]. Another recurrent assumption is that *only computation leaks*; that is, the only source of leakage are the computations performed, while memory operations, such as storing a value in a buffer, does not provide any meaningful insight to the attacker [13, 26]. This two assumptions are easily violated by off-the-shelf hardware, which may show unexpected physical phenomena: *glitches*, a non-ideality in the propagation of signals, are source of additional power consumption of CMOS gates, allowing an attacker to observe a leakage depending on more than one value [42, 43]. Memory elements may leak an amount of power proportional to the previously stored element and new one. Consequently, such *transition* may be profitably exploited by attackers to halve security order of schemes [15, 2].

In order to enable sound security elaboration of masking schemes, established leakage models have been extended, encompassing such non-idealities; see [5] for instance.

1.2 Higher-Order Boolean Masking Schemes

In the following, a review of the current state-of-the-art on the most prominent high-order *boolean* masking schemes is detailed, along with an improvement on the classical *decomposition vs. re-computation* categorization.

1.2.1 Taxonomy for Higher-Order Boolean Schemes

Most of the modern cryptographic schemes embed, in their design, the s-BOX as unique non-linear element, driving the complexities – time, space and randomness – of randomized implementations [47]. It is possible to primarily classify schemes according to how the single s-BOX is randomized: via its *re-computation* or *decomposition* [58, 40].

Re-computation Schemes relying on re-computation consist in, for each query, compute a new random version of the same s-box, such that any statistical relation between input and output is made negligible.

Formally, given a s-box

$$S : \{0, 1\}^k \rightarrow \{0, 1\}^{k'}$$

with $k' \leq k$, and a query to the shared index $x = \bigoplus_{i=0}^n x_i$, a randomized table T is computed applying, $\forall u \in \{0, 1\}^k$, the following pair of equations:

$$T(u) = S(u \oplus x_0) \bigoplus_{j=1}^n s_j \quad (1.1)$$

$$T(u) = T(u \oplus x_i) \quad (1.2)$$

where the second equation is computed iteratively for each i from 1 to $n-1$, and $s_j \leftarrow^{\$} \mathcal{U}_{2^{k'}}$. Eventually, the masked output is retrieved by means of x_n :

$$T(x_n) = S(x_n \bigoplus_{i=0}^{n-1} x_i) \bigoplus_{j=1}^n s_j = S(x) \bigoplus_{j=1}^n s_j$$

The described higher-order scheme of a s-box was developed by Schramm and Paar [53], later proved to be flawed for orders greater than 2 [17]. Nonetheless, it is base for new, secure, masked designs based on re-computation.

Decomposition The decomposition approach expresses the s-box S as a polynomial P , such that its evaluation is performed by means of linear (sum, scalar multiplication, squaring) and non-linear operations (field-multiplication) on a finite field $\mathbb{F}_{2^k}$, $k > 0$ [12]:

$$S(x) \rightarrow P(x) = \sum_{\alpha \in \mathbb{F}_{2^k}} P(\alpha) \cdot l_{\alpha}(x) \quad (1.3)$$

where l_{α} are the *Lagrange basis polynomials*.

Being the field-multiplication the unique non-linear transformation, masking schemes focus on its secure randomization.

Improving Classification To the best of my knowledge, literature lacks of a classification for decomposition based schemes. Thus, to fill this gap, I introduce two further categorizations of these methods. In the first one, decomposition schemes may be grouped according to the *details* considered within the design process:

- *algorithmic-oriented*: the scheme is designed abstracting away all physical details, e.g., micro-architectural elements.
- *implementation-oriented*: actual physical defaults, e.g., glitches, are taken into account during the design process.

Table 1.2: HO Masking Schemes - Classification

DECOMPOSITION		RE-COMPUTATION
AO	[31, 59, 21]	[14, 18, 58]
IO	FO	-
	DO	
	[45, 50]	[30, 29, 28]

AO: Algorithm-Oriented. **IO**: Implementation-Oriented. **FO**: Function-Oriented. **DO**: Domain-Oriented.

The second categorization is focused on *how* the implementation-oriented schemes thwart leakages, stemming from physical effects.

Given a function $z = F(x, y)$, its randomization may be:

- *function-oriented*: F is decomposed in several *sub-transformations* F_i , such that each of them compute different intermediate values. Randomization is performed on each F_i .
- *domain-oriented*: the transformation F is masked such that *share domains* are separately processed. That is, for each $\bar{x}, \bar{y}$ in input, and for each $i \neq j$, shares $\bar{x}_i, \bar{y}_i$ are employed within the same function, while $\bar{x}_i, \bar{y}_j$ are kept separated.

Table 1.2 visualizes the classification of generic higher-order masking schemes.

Although domain-oriented schemes may be regarded as a sub-category of function-oriented ones, the actual focus is on the separation of the share domains; this fundamental aspect is reflected by the above classification.

1.2.2 Re-computation based approach

Schramm and Paar were the first to propose a generic higher-order re-computation scheme [53], later proved to be secure only up to the second order [17]: informally, the vulnerability stems by the fact that each entry of the new table shares the same randomness.

Basing on this observation, Coron [14] envisions an improvement on Schramm and Paar's, by re-computing each table's entry with a fresh set of random values. The effort's result is a n^{th} -order generic recomputation scheme, with $n \geq 2t + 1$, whose security is evaluated under the *full-adaptive* t -probing model – the attacker may change observed computations with each new run of the

algorithm. Interestingly, the authors argue how it is possible to reduce the time requirements by a factor $2^{\frac{k}{2}}$, where k is the input length of the lookup table, exploiting the *large registers* provided by some architectures (for instance, the 32bit general purpose registers characterizing the CORTEX-A7 processor): several entries are packed into a single register, and processed altogether, instead of processing each entry one at a time. Not less important, their design establishes the complexity bounds required for sound and secure re-computation approaches. As a consequence, subsequent works, among which the following, try to work out techniques to improve the hidden constant of such complexities.

Coron, Rondepierre, and Zeitoun [18] improve the asymptotic complexities of [14], by analyzing, first separately, then combined, the enhancements brought by (1) an incremental randomization of the table, (2) the usage of the *common-shares* technique [18], speeding up computations in presence of simultaneous lookups, and (3) halving the lower bound on the number n of shares – from $2t + 1$ to $t + 1$. All the three variants are proved to achieve t^{th} -order security, under the notion of *strong non-interference* [7]. Although reaching significant time improvements individually – decrement by a factor of (up to) 4 – their combination does not multiply such terms independently, entailing a smoother optimization.

Tang et al. [58] enhance the asymptotic results for space and randomness complexities of [14], by noting that is possible to query a randomized table half of the size of the original one, accessing it twice to recover the shares of the output. Although the minimal increased number of accesses, which keeps the time complexity substantially invariated, the space and required randomness are effectively halved, still preserving the security guarantee of Coron’s.

Table 1.3 summarizes the number of shares and complexities of each presented scheme, under the *large register* optimization. A part for the complexity figures, which represent a fundamental discerning factor for the selection of a randomization algorithm, the compositional security of gadgets should also be considered: the approach of [58] certifies the t -order security of the single gadget only. To the best of my knowledge, no analysis on the security of its composition exists, and, thus, its employment as component of a masking algorithm should be carefully evaluated. On the contrary, in [18] the *t-SNI* property enables the secure composition of all the proposed gadgets by default; further, the security of the gadget in [14] is improved, showing that it is secure under the same property.

1.2.3 Decomposition based approach

Masking based on decomposition act of the non-linear elements characterizing the polynomial representation of the s-boxes composing the target encryption scheme. As described in 1.2.1, current works on s-box decomposition may be

Table 1.3: Recomputation methods - Asymptotical Complexities (Large Registers). n is the order of the mask. k is the bit-width of the s-BOX's input. t is the number of observed computation. All the complexities refers to the number $n = 2t + 1$ of shares of [14].

	#SHARES	TIME	SPACE	RANDOMNESS	t-SNI COMPOSABILITY
[14]	$n = 2t + 1$	$\mathcal{O}(n^2 \cdot 2^{\frac{k}{2}})$	$\mathcal{O}(n \cdot 2^k)$	$\mathcal{O}(n \cdot 2^k)$	Yes
[18] ^a	$t + 1$	$\mathcal{O}(\frac{n^2}{4} \cdot 2^{\frac{k}{2}})$	$\mathcal{O}(\frac{n}{2} \cdot 2^k)$	$\mathcal{O}(\frac{n}{2} \cdot 2^k)$	Yes
[18] ^b	$2t + 1$	$\mathcal{O}(\frac{n^2}{2} \cdot 2^{\frac{k}{2}})$	$\mathcal{O}(n \cdot 2^k)$	$\mathcal{O}(n \cdot 2^k)$	Yes
[18] ^c	$t + 1$	$\mathcal{O}(10.7n^2 \cdot 2^{\frac{k}{2}})$	$\mathcal{O}(\frac{n}{2} \cdot 2^k)$	$\mathcal{O}(\frac{n}{2} \cdot 2^k)$	Yes
[58]	$2t + 1$	$\mathcal{O}(n^2 \cdot 2^{\frac{k}{2}})$	$\mathcal{O}(\frac{n-1}{2} \cdot 2^k)$	$\mathcal{O}(\frac{n-1}{2} \cdot 2^k)$	Unknown

^a share-reduction variant.

^b incremental randomization and common shares.

^c combination of all the three variants.

classified according to the low-level detail considered in the protection.

Algorithm-oriented

In [12], Carlet et al. argue that, in the polynomial evaluation of s-boxes, the most expensive operation is the non-linear multiplication, requiring $(n + 1)^2$ field multiplications and $k \cdot \frac{n(n+1)}{2}$ bit of randomness, where k is the bit-width of the s-box's input and n the order of the masking. Consequently, to evaluate the efficiency of a s-box decomposition based randomization, they introduce the notion of *s-box masking complexity*, defined as the minimum number of non-linear multiplication for the evaluation of a masked s-box. Coron et al. [21] tackles improvement on masking complexity of s-boxes by minimizing the number of field multiplications for each non-linear one: with the introduction of the *common shares* technique – shared inputs are allowed to share up to the first $\frac{n}{2}$ components of their encodings – it is possible to efficiently evaluate two multiplications in the form $\bar{c} \odot \bar{a}$ and $\bar{c} \odot \bar{b}$, by storing the first $\frac{n^2}{2}$ intermediate multiplications employed in the former calculation, and re-using them in the latter. Such technique, shown to be secure under the *t-SNI* property – hence, compositionally secure – virtually reduces the number of non-linear multiplications, enabling the efficient polynomial evaluation of s-boxes under the *t-probing* model, with $n \geq t + 1$ shares. The common share method is compared with state-of-the-art decomposition ones, exhibiting the best improvements on the s-box masking complexity.

Basing on the observation that non-linear multiplications in the form $x \odot g(x)$, with $g(\cdot)$ a linear operation, may be secured more efficiently than general ones,

Grosso, Prouff, and Standaert profitably employ such computations to improve over the construction in [12], by identifying the best sequence of linear field additions and non-linear multiplications, with the additional goal of replacing, as much as possible, the latter with $x \odot g(x)$. Efficient randomization under the *t-probing* model is empirically proved, by comparing with latencies of other state-of-the-art schemes based on Carlet et al.'s. It is worth noting, however, that their method is effective when the input's size does not allow tabulation of the multiplication, i.e., for $n \geq 5$.

Tang et al. [59] study the opportunities given by re-configurable hardware, exploiting the disjunctive normal form of the polynomial representation of a s-box, to derive a randomized *configuration sequence*. Such configuration sequence is generated with each query, and employed to generate a randomized circuit on top of a re-configurable hardware. The configuration sequence may be generated such that the final circuit satisfies some non-functional requirements, e.g., maximize performances and/or minimize resources consumption. The derived strategy is secure under the *t-probing* model, although $n \geq 2t + 1$ shares are mandatory.

An efficiency comparison between the three methods is, unfortunately, not possible: although [31] and [21] are compared with state-of-the-art decomposition designs, the implementations and empirical analysis are performed with different equipment and comparison metrics. Due to the possible flaws in the polynomial based decomposition methods, in [59] authors compare the performances with the re-computation method described in [14]. Concerning the compositional security of the schemes, thanks to the security proof under the t-SNI notion, [21] achieve such property. To the best of my knowledge, no analysis on secure composition is present in literature for the other two schemes.

Implementation-oriented

Non-ideal behavior of the platform may be counteracted by re-designing the function to be protected. State-of-the-art works focus on *function decomposition* and *domain composition* (see 1.2.1).

Function-oriented One of the first schemes to put forward a generic decomposition-based approach are the so-called *Threshold Implementations* (TIs). Albeit being limited to the first order, and proved to be secure under the *noisy leakage* model, it provided a first step for strong randomization against glitch-based attacks. Key idea of TIs is the *sharing* of any arbitrary transformation F into F_i components, such that *correctness* – $F(\vec{x}) = \odot_i F_i(\vec{x}^i)$ – and *non-completeness* – each component must be independent from, at least, one share of each input – are satisfied. The non-completeness property guarantees independence of outputs and intermediate computation from each input, without requiring extra randomness. The lower-bound on shares, necessary to provide the non-

completeness, is $n \geq s + 1$, with s the number of input variables of the original transformation F .

Bilgin et al. [10] extend the TI theory, providing a constructive proof for a t^{th} -secure gadget, resistant to t^{th} -probing attack with glitches. The proof is based on the concept of t^{th} -order completeness – an extension of the original *non-completeness* to set of (at most) t component functions – and employing results on the security equivalence of t -probing security model and the *noisy* security model [25]. Compositional security of such higher-order TI-based gadgets is based on the satisfaction of the *input encoding uniformity* property for each gadget.

Although the sound security proof for single higher-order TI gadgets is present, their composition is not correctly assessed. Reparaz pinpoints the lack of independence between inputs and outputs as root cause, providing two examples of how this deficiency may reduce the security order [49]. As a mitigation, the author proposes to break any possible dependency by refreshing the results of each TI gadget.

In [50], Reparaz et al. highlight some interesting connections between the TI-based AND and two other well-known multiplication masking, i.e., ISW AND and the Trichina AND-gate, identifying key-points for a sound masking algorithm against a t -probing adversary, able to exploit the glitching behavior of the target platform. Such analysis allows the authors to infer a generic function-decomposition based four-layer structure for masking algorithms, independent on the linear and non-linear transformations involved, at the cost of potential redundant operations (e.g., superfluous refreshing), increasing randomness, area and latency. The generalized scheme employs the same number of input shares of TIs, and leverages the d^{th} -non-completeness property and uniformity of the inputs to achieve higher-order security of individual gadgets, while refreshing layers enable secure composition of gadgets, albeit increasing the randomness complexity of the construction. Interestingly, the authors show how to reduce the number of input shares to $d + 1$, assuming *mutual input dependency*.

Domain-oriented Being a provable sound countermeasure, masking is widely employed for the protection of cryptographic implementations. Nonetheless, it comes with a non-negligible cost and/or ideal assumptions. Two of the most prominent masking algorithms, i.e., ISW and Threshold Implementations, suffer of these problems: the former is generic with respect to the security order, but incurs in a quadratic blow up of the circuit size and is vulnerable to *glitches*; the latter, instead, is inherently immune to glitches, but are subject to expensive designs in terms of area, latency and randomness. Further, the TIs are not scalable to higher-orders, for the component functions to be re-design to

fit the increased number of shares.

Lacking in literature of general and scalable algorithms for sound hardware randomization, Gross et. al. propose a new approach, somewhat similar to the TIs, based on *domains* [30]. The key idea is to *share*, i.e., decompose, a function into *components*, such that each operates solely on shares with same index; such subsets of shares are the so-called *domains*. For a n^{th} -order masking, this means to have $n + 1$ component functions working on $n + 1$ distinct domains. Such decomposition achieves n^{th} -order security, keeping domains mutually independent, i.e., the observation of the component function F_i will measure leakage related only to shares (or their combination) belonging to domain I . The approach is exemplified by implementing two DOM-based variants of the AES cryptosystem, with 128 bit of key, at various orders of protection. The two variants, referred to as *AES-simple* and *AES-interleaved*, are used to show the flexibility of the DOM approach, being able to target of different requirements: the former is designed to achieve better area usage and randomness complexity, at the cost of higher latency; viceversa, the latter reaches the highest throughput, trading area and required randomness.

Apart for the better scalability with respect to the protection order, [30] shows that DOM-based construction are characterized by better figures in terms of *chip-area*, *randomness* and *throughput* than TI implementations, comparing the two DOM-based AES variants with state-of-the-art threshold implementations: the simple variant is more efficient in area consumption by (at least) 12% and saves up (at least) 43% of randomness; latencies and throughput show similar figures. The interleaved version exhibits smoother improvements on area and randomness than the simple one, but better latencies and throughput by (at least) 18%. Furthermore, second order implementations of the DOM AES variants are compared with state-of-the-art second order TI construction of the AES s-box: the two DOM designs outperforms the TI alternative, with an area usage slightly higher than the one employed by the TI s-box, and with a randomness reduction of 57% for the simple version, and 33% for the interleaved one.

In [29, 28], Gross and Mangard combine the DOM strategy to previous works on randomness minimization [8, 6], developing a domain-based masking algorithm, called *Unified Masking Approach* (UMA), able to minimize the randomness complexity both in hardware and software for a masked field multiplication. In particular, the multiplication is represented as a repeated sequence of sums, interleaved by a refreshing stage, where the last sequence is handled by specific randomness optimizations. The unification of DOM efficiency and randomness optimization techniques allows to reach the lowest bound of computational complexity, i.e., $\mathcal{O}(\frac{d(d+1)}{4})$, achieving even the theoretical limits of d , for $d \leq 2$, and $d + 1$, for $2 < d < 5$. A UMA and DOM based implementations

of the ASCON authenticated ciphers, with 1 and 64 parallel s-boxes, and protected at various protection orders, are compared in terms of *area*, *throughput* and *randomness*: with respect to the first two parameters, the one-s-box implementations show similar figures, which get worse with the protection order when the 64-s-boxes variants are considered. Concerning the randomness, in either variants, the UMA construction shows better complexities with the protection order, obtaining a reduction of up to a factor of two with respect to DOM masking. Although the worse area and throughput characteristics, the authors argue that, as long as area-efficient PRNG are employed, the lower randomness requirement of UMA allows to generate implementations of protected crypto routines, suitable for low-area context.

1.3 Memory Transitions against Software Masking

Being a technology amenable for implementation of varied logic cells, most elementary elements of micro-controllers, such as processor registers, buses and micro-architectural buffers, are built on CMOS logic style. Due to the nature of such technique, the switching activity of CMOS based components may leak an amount of consumed power depending on the *distance* of two intermediate values; the *independence leakage assumption* (ILA), on which several masking schemes base their security proof, is violated, reducing the claimed level of security. In this section, the topic of memory-transition attacks is analyzed: by firstly reviewing the main source of power leakage for CMOS logic cells and the hamming-distance function as a good approximation of the consumed power, the effect of a failing ILA is briefly outlined on a ISW $\mathbb{F}_2$ AND software implementation. Despite relieving designers from tedious manual coding and speeding up deployment, it is argued how the hardware abstraction provided automated masking tools may thwart the utility of such tools, and how the micro-architecture memory elements of a platform might represents a viable surface attack for successful power-based attacks. A brief review of some works on hiding memory-transition attacks concludes this section.

1.3.1 Power Consumption in CMOS

In CMOS circuits, logic cells are built by combination of pull-up (if active, it enables a path from power supply to output) and pull-down (if active, it enable a path from output to ground) networks of MOS transistors.

Power dissipation of such circuits is given by three independent terms: (1) *static power*, (2) *short-circuit* (induced) power and (3) *dynamic power* consumption [32].

Static power is caused by constant current flow from energy supply to ground. Thanks to the particular composition of pull-up/pull-down networks, such leakage phenomenon accounts for a very small amount of information leakage.

Figure 1.2: CMOS inverter.

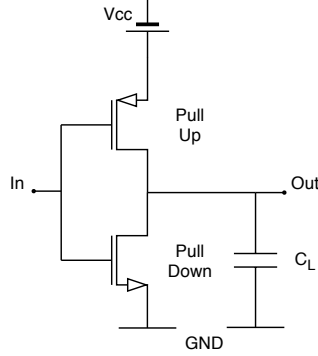


Table 1.4: CMOS Inverter - Currents drawn and switching activity.

INPUT TRANSITION	DYNAMIC POWER
1 → 0	Short-Circuit + Load
0 → 1	Short-Circuit

However, state transition of pull-up/pull-down networks cannot be instantaneous, since we are dealing with realistic devices. Consequently, when both networks are on the active state, a small amount of current is able to flow from supply to ground, consuming an certain quantity of static power, named as *short-circuit* power.

The switching activity of the circuit – activation and deactivation of pull-up/pull-down networks, owe to changes in input signals, or internal ones – is responsible for another power dissipation, the *dynamic power*.

To exemplify this last point, let take the CMOS inverter as an example (Figure 1.2).

As reported in Table 1.4, which outlines the power consumed by the cell when the input transition, a change from high to low in the input implies activation of the pull-up network (and simultaneous deactivation of the pull-down one), and subsequent flowing of current from supply to the output load C_L , dissipating power.

1.3.2 Hamming-Distance Power Model

A slightly more complex example of CMOS based component is represented by (*memory*) *registers*. A register is an elementary element of a computer architecture, employed as small and fast storage for transient values, usually made of several *sequential* cells (e.g., D-LATCH), each storing one bit of information.

Table 1.5: CMOS Memory Register - Transitions and Dynamic Power

OUTPUT TRANSITION	DYNAMIC POWER
$0 \rightarrow 0$	P_{00}
$0 \rightarrow 1$	P_{01}
$1 \rightarrow 0$	P_{10}
$1 \rightarrow 1$	P_{11}

The number of cells composing the register defines the *register bitwidth*.

For a one-bit register cell, the possible transition are given in Table 1.5.

In general, it is assumed that $P_{00} \approx P_{11} \ll P_{01}, P_{10}$ even though it is highly dependent on the type of register cell and implementation technology [41].

Although grouped together, register cells are independent CMOS sequential unit; as in any CMOS circuit, the total amount of power consumed by the circuit (register) is given by the sum of the individual power consumption of composing elements (register's cells).

As a consequence, it is possible to employ some functions to relate the power consumed by the transition of the register's cells to a defined group of inputs.

For instance, let consider a 2-bit register, storing at time instant t_1 the input $x = 10$. At time instant t_2 , a new input is stored, $y = 11$.

The dynamic power dissipated between those two instants is given by:

$$P_{\Delta_{1,2}} = P_{11} + P_{01} \approx P_{01} = \text{distance}(x = 10, y = 11) \quad (1.4)$$

The set of power-inducing input pairs is dependent on the distance function chosen. Knowing x or y , it is possible to cut out some of the pairs, allowing hypothesis on the missing value. A simple, but highly employed power model for transition is the so-called *Hamming-Distance* power model (HD).

The HD model makes some simplifications on the real power consumed (all cells consume in the same way, no static power consumption, $P_{01} \approx P_{10}$) [41]; under this model the power consumed by a transition is proportional to the hamming distance of the two stored values. Resuming the previous example:

$$P_{\Delta_{1,2}} = HD(x = 10, y = 11) = \sum_{i=0}^{n=1} x_i \oplus y_i = 1 \quad (1.5)$$

Under the HD model, the possible set of pairs causing such transition are $\{(00, 01), (00, 10), (01, 11), (10, 11)\}$.

1.3.3 Assumptions on Leakage and Software Masking

In masking, proofs of security are envisioned under some specific assumptions on the attacker's capabilities and/or the amount of informations leaked by

the underlying device, captured, respectively, by the leakage model and the adversarial model. Recall that the goal of the security proof is to formally demonstrate that, in a n^{th} -order masking, the n^{th} -order property is satisfied.

Concerning the leakage model, two of the most recurring assumptions are the *independent leakage assumption* (ILA) [20, 2] – the leakage depends only on a single intermediate value – and the *only computation leaks* [13, 26] – the leakage is due only to the computations, i.e., memory elements do not leak.

Such hypothesis, however, do not model properly the physical nature of the hardware devices, on which masked implementations are ran. In the scope of software masking, architectural registers have been shown to be a viable point of attack, thanks to the power leaked by transitions, which violate both cited assumptions.

In particular, in [20], Coron et al. formally prove how the systematic re-use of registers may leak the distance of consecutively stored values, empirically demonstrated with a first-order attack against a second-order masking of the AES s-box. Balasch et al. extends the work of Coron et al., and define security relations between *value-based* leakage functions – the information content retrieved is assumed to be dependent only on the value actually computed – and *transition-based* ones – the leakage encompasses information belonging to more than one value; most importantly, it is proved that schemes, which satisfy the n^{th} -order property under a value-based power model, suffer a halving of their security order under a transition-based function [20].

Algorithm 1: ISW $\mathbb{F}_2$ AND [33]

Input: $\bar{a} = \langle a_0, a_1, \dots, a_n \rangle$, $a_i \in \mathbb{F}_2$

$\bar{b} = \langle b_0, b_1, \dots, b_n \rangle$, $b_i \in \mathbb{F}_2$

Output: $\bar{c} = \langle c_0, c_1, \dots, c_n \rangle$: $c_i \in \mathbb{F}_2$, $c = (\bigoplus_{i=0}^n a_i) \wedge (\bigoplus_{i=0}^n b_i)$

```

1. begin
2.   for  $i \leftarrow 0$  to  $n$  do
3.     for  $j \leftarrow i + 1$  to  $n$  do
4.        $z_{ij} \leftarrow \{0, 1\}$ 
5.        $z_{ji} \leftarrow (z_{ij} \oplus a_i b_j) \oplus a_j b_i$ 
6.   for  $i \leftarrow 0$  to  $n$  do
7.      $c_i \leftarrow a_i b_i$ 
8.     for  $j \leftarrow 0$  to  $n$  do
9.       if  $i \neq j$  then
10.         $c_i \leftarrow c_i \oplus z_{ij}$ 
11.   return  $\langle c_0, c_1, \dots, c_n \rangle$ 

```

```

1 .PRE_LOOP:
2     ... @ prepare next set of shares in regs
3
4 .LOOP:
5     bl random32      @ generates a 32bit random value
6     and r4, r0, r1 @ (a_i & b_j)
7     and r5, r2, r3 @ (a_j & b_i)
8     eor r4, r5      @ (a_i & b_j) ^ (a_j & b_i)
9     eor r0, r4      @ (z_ij ^ (a_i & b_j)) ^ (a_j & b_i)
10    ands r6, r6      @ exit flag setting
11    bne .PRE_LOOP

```

Listing 1.1: Thumb2 ISW AND – Cross-Term Combination

Transition-Leakage on 1st-order ISW AND To elaborate on how register transitions may be practically employed against software implementations of masking schemes, let us consider a portion of a ARM THUMB-2 assembly implementation of first-order – $n = 2 \cdot 1 + 1 = 3$ – ISW $\mathbb{F}_2$ multiplication, executed on a CORTEX-A7 chip, reported in Listing 1.1, realizing the secure computation of cross-term additions $(a_i b_j \oplus a_j b_i)$. Recall that the ISW scheme is proved to be secure under the *t-probing* model (the security is evaluated under the ILA).

In the original algorithm (Algorithm 1), it is required that the computations follow a precise order, dictated by the round parentheses; however, the naïve encoding displayed misses such requirements, revealing the full set of shares of input b . For instance, let us consider the first two iterations of the loop described in Listing 1.1: in the first iteration, register R4 stores the field sum $x = a_0 b_1 \oplus a_1 b_0$, and, in the successive loop iteration, the same register is loaded with the cross-term $y = a_1 b_2$. As explained in Section 1.3.2, such transition leaks an amount of power proportional to the distance of the two values. Employing the HD as model, the power is proportional to:

$$P \approx HD(x, y) = x \oplus y = (a_0 b_1 \oplus a_1 b_0) \oplus a_1 b_2 \quad (1.6)$$

Thus, the power consumption observed embeds all the three shares of b , statistically correlating the observation to the input: the 1th-order encoding is violated.

Masking and Automation Software implementation of masked routines via assembly language is a really tedious and error prone activity, since the very low-level nature of the architecture have to be explicitly considered, e.g., registers and memory stack. In recent years, compilers and code-generation tools have been proposed as a good fit to aid developers and speed-up deployment of randomized implementations. The main benefit of these tools is the provided layer of hardware abstraction.

Nevertheless, as empirically proved in [2], the abstraction granted by general-purpose compilers, e.g., GCC or LLVM, hides the leakage characteristics of the architecture, resulting in a potential loss when implementing a provably secure scheme; for instance, it is not possible to forbid overlapping of related shares, i.e., belonging to the the same encoding.

1.3.4 Micro-architectural Memory-Transition Leakage

In two recent works, the micro-architecture of modern in-order processors, either low-end single-issue architectures and super-scalar dual-issue ones, have been proved to extend the attack surface against software masking, critically reducing the security of executed implementations.

In [54], Seuschek, De Santis, and Guillen successfully mount a first-order EM attacks against a masked PRESENT s-box software implementation, claimed to be first-order secure, exploiting the transition-based leakages occurring in the CORTEX M0's micro-architecture. In particular, the author identifies specific sequences of instructions that leak the distance of their inputs/outputs.

In [4], Barengi and Pelosi assess the side-channel characteristic of the micro-architectural buffers of the CORTEX-A7's pipeline, identifying the pipe-stage buffers and some functional unit buffers as sources of transition leakages. Their analysis is validated via a successful power analysis attack against a AES128 unmasked implementation, ran in a noisy execution environment (resource-demanding processing simultaneously executed on the same processor).

A Code-Generator based Solution

An interesting solution to automatically counteract leakages stemming from the micro-architecture is provided by Seuschek, De Santis, and Guillen [54], who develop a code-generation tool to simplify development of optimized, leakage-resilient software masked specifications. Specifically, the tool is designed as the combination of an instruction scheduler and register allocator, and accepts as inputs the assembly specifications of the masked routine, and a *conflict graph* $G_C = \langle V_c, E_c \rangle$ describing the conflicting variables: V_c is the set variables making the input specification, while E_c describes pairs of variables $\langle v, w \rangle \in V_c \times V_c$, whose consecutive storing into the same register leaks the unmasked value.

The conflict graph is built on the results of a leakage analysis of the target's micro-architecture; in particular, the leakage occurring with specific instruction sequences is captured. In the scope of their work, the CORTEX M0 is targetted, while the instruction sequences inspected are the ones employed for the development of a decomposition-based masking of the PRESENT s-box (XOR and AND), and memory instructions (LOAD and STORE) for spilling and re-loading.

Leakage-Aware Instruction Scheduling The instruction scheduling algorithm proposed is formulated as a *probabilistic* solver for a topological sorting

problem, defined on a data-flow graph representation of the input specification, constrained by the conflict relationship described by the conflict graph. Algorithm 2 reports the pseudo-code specification of the solver. The re-organization of the stream is lead by the probabilistic selector *epsilon*, which select the next ready computation in the data-flow graph, i.e., the input operand(s) are already computed, such that inputs and output do not conflict with, respectively, inputs and output of previously selected instruction. Due to the impossibility of identifying a non leaking choice for the next instruction to be scheduled, some specifications might require several runs on the scheduling not to fail (impossible identification of a non-leaking computation in the data-flow graph), or require introduction of dummy instructions.

Algorithm 2: Leakage Aware Topological Sorting [54]

Input: $G_d = (V_d, E_d)$, Data Flow Graph

$G_c = (V_c, E_c)$, Conflict Graph

Output: S , topologically sorted instruction stream

```

1. begin
2.    $S \leftarrow []$                                      // Ordered list S
3.    $A \leftarrow V_d^I$                                    // Input vertices
4.   while  $V_d^C \neq \emptyset$  do
5.      $v \leftarrow \epsilon^{A, G_c}(V_d^C)$                // Select vertex
6.     if  $v \neq \perp$  then
7.        $S.append(v)$ 
8.        $V_d^C \leftarrow V_d^C - \{v\}$ 
9.        $A \leftarrow A \cup \{v\}$ 
10.    else
11.      return FAIL
12. return  $S$ 
    
```

Leakage-Aware Register Allocation The register allocation is secured on the assumption that all the variables are stored on the memory stack. Scanning the re-organized stream, for each computation C , a sequence of load and store instructions is issued, respectively, before and after C . This sub-optimal allocation does not leak any information from the micro-architecture: indeed, given two consecutive instructions I_1, I_2 , their inputs and outputs do not conflict, otherwise the scheduler would have not placed them consecutively. Hence, the LOADS (STORE) of the I_1 's inputs (output) do not conflict with the ones for I_2 's.

This code-generation oriented approach have the advantage of not impacting on the randomness cost, since the masking order is the same and no fresh

randomness is required, although the computational complexity may vary due to non-optimal selection of instructions to be scheduled. However, the solution proposed makes use of a random selection of instructions ready to be scheduled in order to avoid distance-leakage; a part for the non-optimal scheduling, the potential inexistence of a schedulable instruction fail the compilation. It is not clear if, with a slightly different implementation, it would be possible to actually produce a compilable scheme; even with the support of the code- generators, the developer would have to try different implementations almost at random. Nevertheless, the work provides a proof-of-concept of solution which, possibly, does not heavily impact on the final implementation and does not penalize constrained devices.

1.3.5 Related Works

Coron et al. analyze the systematic use of constant pre-charing of registers [41], in order to remove the side-channel derived from memory transitions, proving that it is not a solution to problem, by mounting a second order attack against a second order masking scheme [20]. Further, they pinpoint how random uncorrelated values may be profitably used to *clean* the registers, hindering any insight provided by transitions. In [2], the authors propose to actually ignore the problem by incrementing the masking order of the implementation. Even if this may be sound as a viable way, and considering that code generation tools, such as compilers, may be employed to automatically protect the cryptoscheme implementation, this lazy solution comes with non-negligible randomness and computational cost, in particular for hardware platforms with constrained resources.

1.4 Code Generation in LLVM

1.4.1 LLVM: an Overview

Formerly *Low Level Virtual Machine*, LLVM [37] is an open source project providing a set of libraries, modules and support tools for the creation of production-quality compilers, with focus on with a focus on ahead of time translation and code generation. Developed with modularity and efficiency in mind, LLVM exhibits a 5-stage design philosophy, trying to achieve lifelong program analysis and transformation support: program analysis and optimizations are executed throught the entire lifetime of the program itself, encompassing static-compilation (static optimizations), link-time (inter-procedural optimizations), installation time (machine-dependent optimizations), run time (dynamic optimizations) and idle time (profile-guided optimizations between runs).

One of the main strenght of the project is the LLVM-IR: a RISC-like instruction set, characterized by a language-independent, strong-type system; an infinite set of typed virtual register; a *Single Static Assignment* (SSA) form rep-

resentation of the code. These features enable the implementation of high-level concepts, such as polymorphism, and, at the same time, encode any arbitrary program. Further the design of this language allows high-level transformations and efficient analysis of the represented programs.

Each analysis and optimization is encoded in a self-contained module, called *Pass*; dependencies between passes are explicitly stated. The whole optimization process is handled by the LLVM *Pass Manager*, an analysis/transformation agonistic component in charge of determining the most efficient sequence of passes, in terms of compilation time, satisfying each pass's dependencies. The self-contained nature of the passes allow the end-user to pick only analysis and transformations required for his/her goal, without bothering about useless operations, which only increase compile-time and code-density. Due to its well-designed architecture and appealing features, LLVM has become the base for several projects, both in the academic and industrial environment.

1.4.2 Target-Independent Code Generator

One of the strength of LLVM is to provide a set of high-performance code-generation tools, such as register allocation, *independent* from the actual platform being targeted. This feature is enabled thanks to a well-designed separation between algorithm and target, where specific characteristics of the platform, which may drive specific code-generator decisions – e.g., the presence of a MIPS-style *branch-slot* influence the actual scheduling decisions.

Interfacing between the code-generator algorithms and the targets features is allowed by a set of API, called *Abstract Target Description*, exposing methods for querying the actual target model.

In LLVM, targets are fully described via TABLEGEN table-description files, a domain-specific language employed for detailing, with enough expressive power, several features of a target, spanning through ISA, ISA registers, scheduling model and more.

Most of the code-generation modules run on a lowered representation of the LLVM-IR. Such representation, still independent on the actual target, represents exposes elements characterizing instructions, basic block and functions.

Since the actual code-generation passes modified to provide leakage-awareness at this level are *instruction selection*, *register allocation* and *instruction scheduling*, in the remainder is provided an overview of such algorithms; in particular:

- **Basic Register Allocator:** an evolution of the linear-scan algorithm, replacing the interference graphs with a map between the physical registers and the liveness intervals of the variables allocated into them, giving allocation priority to the most frequently used intermediates.
- **Machine Scheduler:** instruction scheduler operating on a serialization of the *direct acyclic graph* generated by the instruction selection phase.

LLVM provides other algorithms for allocation, e.g., a greedy register allocator, and instruction re-organization, ILP scheduler; nevertheless, for implementation and experimental analysis, I decided to choose the aforementioned alternatives, since the other available schedulers are still in an embryonic state, and do not take into account resource tracking, which is fundamental for guaranteeing leakage-awareness at scheduling phase, or are based on *graphs*, which, while being quite precise, are slowly replaced by the nearer-to-the-machine machine scheduler; while the basic allocator, despite its simplicity, is the base of other, more powerful, allocators.

1.4.3 Instruction Selection

The instruction selection phase is responsible for lowering the abstract LLVM-IR representation into a version of the same program closer to the actual machine it will run on, exposing features such as calling conventions, stack slots and registers.

This phase is made of 5 distinct steps, some of them repeated more than one time:

- **DAG Building:** the serialized LLVM-IR is parsed and converted into a *Direct Acyclic Graph*, named as *SelectionDAG*. This transformation is performed according to a naïve peephole approach, expanding each LLVM-IR into a specific pattern of nodes.
- **Type Legalization:** each node of the SelectionDAG is parsed, checking legality of input operands and results. Illegal operands are passed to a legalization step, where they may be transformed into target legal types via *expansion* or *splitting*: expansion is performed when an operand is characterized by a type smaller than the smallest type supported by the platform, splitting, instead, splits an operand into two or more smaller types, legal on the target.
- **Operation Legalization:** unsupported operation on the platform are expanded in an emulated version, i.e., software division for targets with no hardware support.
- **DAG Combination:** specific patterns in the SelectionDAG are identified and converted into target optimized versions, i.e., in ARM, a truncation of a load word (LDR) result is combined into a load byte (LDRB) instruction. Such step is ran after initial DAG building and after each legalization step.
- **Selection:** the optimized and legalized version of the SelectionDAG is pattern-matched with target-specific patterns, described via TABLEGEN.

After instruction selection, the DAG is serialized into a machine-specific representation, organized into functions, basic blocks and instructions.

1.4.4 Basic Register Allocator

The BASIC register allocator is a simple, *intra-procedural* register allocation algorithm, i.e., allocation does not consider register assignments performed in other functions belonging to the same code base, which operates in two main phases: (1) assignment of virtual registers to physical registers, (2) spilling of virtual registers from assigned physical registers. Once an assignment is made, the allocator does not have the possibility to abort it in favour of a more profitable assignment, and can only resort on spilling. Although its trivial logic, it employs a flexible concept of *live interval*.

Definition 6. *Live Segment.* Given an ordered set of instructions $\mathbb{I}$ and an enumeration function $\mathcal{E}: \mathbb{I} \rightarrow \mathbb{N}$, $\forall a, b \in \mathbb{N}: a < b$, a live segment is defined as a convex and limited, left-closed, right-open interval $[a, b) \subseteq \mathcal{E}(\mathbb{I})$.

Definition 7. *Live Interval.* Given an ordered set of instructions $\mathbb{I}$ and an enumeration function $\mathcal{E}: \mathbb{I} \rightarrow \mathbb{N}$, a live interval is defined as an ordered set $\mathbb{L} := \{[a, b) \mid [a, b) \in \mathcal{E}(\mathbb{I})\}$ of live segments, such that $\forall [a, b), [c, d) \in \mathbb{L}: b \leq c$.

Definition 8. *Interference.* Given two live intervals $\mathbb{L}_1, \mathbb{L}_2$, they are said to not interfere, or overlaps, if and only if $\forall [a, b) \in \mathbb{L}_1, [c, d) \in \mathbb{L}_2: b \leq c \vee d \leq a$.

Recalling that the goal of the register allocator is to found a proper assignment of virtual registers to physical registers, such that a given cost function is satisfied, such definition of live interval enables more efficient allocation: since *holes* between live segments of the same live interval are allowed, different live intervals may be interleaved within the same physical register, avoiding potential interferences; hence, minimizing the spill code required to solve overlaps.

Each live interval is characterized by a *cost*, called *spill weight*, representing the overhead on the performances, i.e., execution time, entailed by spilling a specific interval; it is proportional to the number of occurrences with which the interval appears in the function.

Definition 9. *Spill Weight.* Given a live interval $\mathbb{L}$, $\mathbb{I}$ the set of instructions where $\mathbb{L}$ occurs as an input operand or destination operand, $\mathbb{B}$ the set of basic blocks containing at least one instruction in $\mathbb{I}$, and given a function $\mathcal{F}: \mathbb{B} \rightarrow \mathbb{R}$ computing the frequency with which a basic block occurs. We define the spill weight as the following function:

$$W(\mathbb{L}) = \sum_{b \in \mathbb{B}, \mathcal{I} \subseteq \mathbb{I}: i \in \mathcal{I} \Rightarrow i \in b} \frac{(\mathcal{F}(b) \cdot |\mathcal{I}|)}{|\mathbb{I}|} \quad (1.7)$$

This cost is employed as a decision driver by the BASIC allocator, which attempts to find an assignment for all the live intervals which *minimize* the overall spill cost in the function; when dealing with an interference, the allocator will favour the interval with higher spill cost: if the already assigned interval has lower spill cost, it is spilled; otherwise, the allocator looks for another physical register.

In order to track the performed assignments, each physical register is decorated with an *ordered* set of *non-interfering* live intervals, representing the intervals it gets allocated to; these sets are referred to as *live interval unions*.

To give more chance to decrease the amount of spilled intervals, hence reducing the impact on performances, the live intervals are stored inside a priority queue, where the priority is represented by the spill weight.

Algorithm 29 reports a detailed pseudo-code of the BASIC register allocator:

1. **Spill weight computation (Line 3-5):** for each live interval within the given input function $\mathcal{F}$, the corresponding spill weight is computed by invocation of `getSpillWeight`. Once the spill cost is retrieved, it is used to properly order the live interval in the priority queue of live intervals Q .
2. **Physical registers allocation (Line 9-16):** live intervals with higher spill weight are retrieved first from the priority queue Q , and tested for potential interference with intervals assigned to the physical registers available on the target architecture (Line 11-14). The assignment is *greedy*: the first physical register free is taken. Each physical register, which showed an interference with the tested live interval, is recorded in the *SpillCands* queue (Line 16).
3. **Spilling (Line 17-28):** In case the allocator has not found a free physical register for the requesting live interval, the queue of *SpillCands* is scan, attempting spilling of the interfering live interval by means of `spill`. In case the spilling has been successfully performed, the new live interval is assigned to the physical register (Line 20-24). If `spill` had not success on any of the physical registers in *SpillCands*, the requesting live interval is spilled (Line 25-28).

1.4.5 Machine Scheduler

The MACHINE SCHEDULER (MS) is an *intra-procedural* instruction scheduling algorithm, working on a low-level serialization of the *SelectionDAG*, which is emitted at the end of the INSTRUCTION SELECTION phase. Differently from the classical *graph-based* scheduling algorithms provided with LLVM, the low-level representation allows fine-grained decisions, since low-level concepts, such as virtual and physical registers, are explicitly exposed to the scheduler.

A direct acyclic graph, called *Scheduling Graph*, supports the scheduling decisions, providing to the scheduler the dependency relationships among instructions and instruction latencies.

Definition 10. *Scheduling Graph*. Given $\mathbb{I}$ a set of instructions, the *Scheduling Graph* is a triple $G_S = \langle V, E, L \rangle$, where $V = \mathbb{I}$, $E := \{(i, j) \mid i, j \in V \wedge \sharp(j, i) \in E\}$ and $L : E \rightarrow \mathbb{N}$ is a unary function, which associate to each edge $(i, j) \in E$ a

CHAPTER 1. BACKGROUND AND CURRENT STATE OF THE ART31

positive decimal value representing the latency required to compute instruction i .

In order to emit a *precise* scheduling, the MS is supported by:

- **Cycle Counter.** The cycle counter (CC) tracks the *estimated* clock cycle where a scheduling decision is made. In case of targets able to issue multiple instructions in the same clock cycles, multiple decisions may be associated to the same clock cycle.
- **Resource Tracker.** The resource tracker (RT) is in charge of tracking, for each hardware resource provided by the target architecture, the number of clock cycles each resource is kept busy.

The overall logic of the MS follows a *list scheduling*-based approach, where two queues, called *ready queue* and *pending queue*, store, respectively, instructions ready to be scheduled and instructions that cannot be scheduled, also called *pending* instructions. Informally, an instruction is said to be *ready*, or *available*, if all its operands have been computed and no structural hazard occurs; in case a structural hazard is encountered, the instruction is said to be *pending*.

The set of instruction queues, RT and CC is referred to as *scheduling state*.

Definition 11. *Scheduling State.* A scheduling state is defined as the quadruple $S = \langle C, R, Q_R, Q_P \rangle$, where C is the cycle counter, R is the resource tracker, and Q_R, Q_P , respectively, the ready queue and pending queue.

The main logic of the scheduler, either in the *pre* or *post* version, works on small contiguous segments of code called *regions*.

Definition 12. *Scheduling Boundary.* An instruction i for which the scheduler is not allowed to act on is called *scheduling boundary*.

Definition 13. *Region.* Let consider a basic block $\mathcal{B}$, and let consider two instructions $i, j \in \mathcal{B}$: $i < j$ and j is a scheduling boundary and $\nexists k \in \mathcal{B}$: $i < k < j$ and k is a scheduling boundary. A region is defined as one (but not both) of the following:

- The subset of instructions $\mathcal{I} \subseteq \mathcal{B}$ enclosed between i and j , if i is the first instruction of $\mathcal{B}$.
- The subset of instructions $\mathcal{I} \subset \mathcal{B}$ enclosed between h and j , where $h \in \mathcal{B}$: $i < h < j \wedge \nexists k \in \mathcal{B}$: $i < k < h < j$ and i is a scheduling boundary.

Scheduling boundaries are generally represented by *call*-like instructions (since the MS is an *intra-procedural* algorithm); nonetheless, the end-user may specify custom boundaries for a given target.

One of the main strengths of the scheduler is its *modularity*: end-users have the possibility to define custom scheduling *strategies* for their target architecture, while preserving the overall logic of the scheduler. Furthermore, it is possible to enable specific strategies for specific functions. By default the scheduler employs a *register pressure/latency-balancing* strategy for the *pre register allocation* phase, able to support either *top-down* or *bottom-up* scheduling, or a mixture of them – named as *converging* scheduling. Regarding the *post register allocation* scheduling, by default a latency-driven top-down strategy is provided. Both strategy, despite considering different goals, are really similar in the overall logic.

Algorithm 8 describes a simplified, yet complete, version of the machine scheduler; the strategy employed is the one used by default in both pre- and post-allocation scheduling, but the selection heuristics are pushed inside the `compare` method. This allows a unified description of the algorithm.

The MS works in four main phases:

1. **Regions formation (Line 5).** The current basic block is explored, extracting regions to be scheduled.
2. **Scheduling candidate selection (Line 12-14).** Among the instructions ready to be issued, the one which best fits the strategy's policy is selected, inserted in the new basic block and removed from the old one.
3. **Scheduling state update (Line 15-25).** The instructions depending on the selected candidate and which are ready are moved to the queue of ready instructions, if no structural hazard occurs; otherwise they are moved to the queue of pending instructions (Line 15-19). The resource tracker updates each resource employed by the selected candidate, by adding to each consumed resource the number of clock cycles required to process the candidate (Line 23); in case the maximum number of issuable instructions is reached, the CC is incremented by one, and the RT reduces by one the clock cycles required by each resource to be freed (Line 22-25).

The extraction of regions is performed by the `getRegion` auxiliary function (Algorithm 9): given a basic block $\mathcal{B}$ in input, $\mathcal{B}$ is explored sequentially, instruction by instruction. If an instruction is found to be a scheduling boundary, the region is stored in the list of regions, and the search of a new one is initialized (Line 7-8); otherwise, the instruction is inserted in the current region (Line 5).

Selection of the best candidate to be scheduled is delegated to the `pickBest` function (Algorithm 18): potential instructions which were deferred from scheduling due to structural hazard are moved into the ready queue (Line 2-5). If no instruction is found to be ready, the CC is incremented and the RT decrements the clock cycles associated to busy resources until a subset of resources required for the consumption of an instruction in the pending queue is found are freed (Line 6-13). Finally, the best candidate to be scheduled is returned: in

case there are multiple instructions ready to be scheduled, the ready queue is explored, comparing each candidate and selecting the one which best fits the strategy’s policies; otherwise, if only one candidate is available, it is returned (Line 14-17). The `checkHazard` function verifies the presence of potential structural hazards (Algorithm 6). The routine check two kinds of structural hazards:

1. Issue Limit Reached (Line 2-3). It is checked if the target architecture is able to issue another instruction or not.
2. Busy Resources (Line 4-5). For each resource consumed by the instruction in input, it is checked if the resource is freed in a clock cycle after the one read from CC.

1.5 CryptoShield

The experimental code-generation based solution is built upon the protection applied by the CRYPTOSHIELD library. Hence, a brief overview of the main points and strenght of this tools is hereby input to the reader. The section concludes with a short description of the masking schemes employed, being the boolean masking the scope of this research project, more attention is granted to them. For more informations, the interested reader may refer to the original thesis [52].

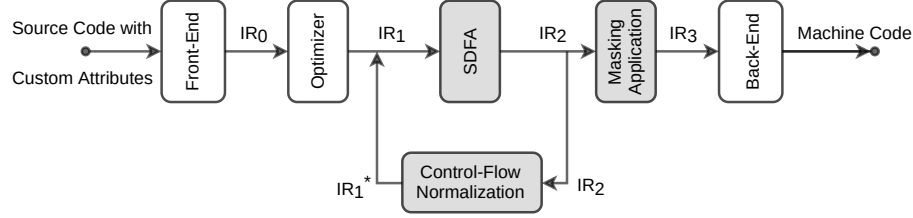
1.5.1 Overview

CRYPTOSHIELD is a LLVM-based library, developed with the purpose of easing the development of side-channel resistant software symmetric cryptographic primitives, by replacing software operations thanks to specific masking schemes. Novelty of this tool is a fine-granularity application of such randomization algorithms, entailing improvements in term of computational complexity and randomness requirements.

Such *partial masking*, proved to be secure in [3], is enabled thanks to the *Secure Data-Flow Graph* paradigm [1], depicted in Figure 1.3. Specifically, the data-flow analysis is actively employed to determine how the bits of sensitive values, e.g., input key and key-material, spreads and contribute to each intermediate values of the cryptoscheme. According to the number of sensitive bits on which a value depends on, it is possible to assign it a *vulnerability index* proportional to such number: the higher the number of sensitive bits involved, the higher is the number of key hypothesis an attacker has to verify, increasing the complexity of a side-channel attack. Thank to this index, the tool applies randomization only to instructions below a user-defined threshold.

The tool accepts any C/C++ specification of either boolean or arithmetic masking scheme. Sensitive assets are user-marked via ad-hoc PRAGMA directive. The protection is parameterizable via the above cited vulnerability threshold, and the order of the mask applied.

Figure 1.3: Secure Data-Flow Framework



From an implementation point of view, the library consists in a *pipeline* of analysis passes and a final transformation pass; all of them executed within the compiler’s middle-end. Hence, the protection is totally transparent to the front-end¹ and the back-end, allowing support for, virtually, any language and target machine.

Furthermore CRYPTOSHIELD provides a visualization feature, which allows to observe bit-level *characteristics* of each instruction’s output: to each bit of instruction’s result is assigned a color, which intuitively helps to determine how a specific characteristic impact on that bit. At the time of writing, it is possible to visualize the key-dependency – how many key-bits contribute to a specific result bit – and ciphertext dependency – how many bit of the ciphertext a result bit impacts on, useful for determine the best attack points in a *fault*-based side channel attack.

1.5.2 Pipeline

The data-flow based pipeline implemented in CRYPTOSHIELD is characterized by 4 *analysis macro*-stages, each composed by a variable number of minor analysis steps, and a final *transformation* stage, responsible for the replacement of vulnerable instructions with semantically equivalent gadgets.

1.5.3 Features Recognition

The LLVM-IR representation of the cryptoroutine is scan, looking first for loading points of plaintext and key bits; then, starting from this nodes, the data-flow graph is traversed breadth-wise, recording for each instruction dependent, either directly or indirectly, on the sensitive value, their *dependency*-depth, defined as the *shortest-path* from the dependent instruction to the loading points. A similar analysis is performed on the *reversed* data-flow graph, on which the dependency depth of ciphertext output instruction on all the intermediate instructions is computed.

¹Nevertheless, the front-end should be modified to introduce an equivalent set of sensitive value markers.

As complementary analysis, look-up table *dead* bits – bits which do not contribute to the final ciphertext output – are identified. Furthermore, to improve the visualization tool, cipher rounds may be optionally spot.

1.5.4 Attack Points Selection

This stage identifies the optimal, i.e., minimal, set of instructions that an attacker may probe to retrieve the whole secret key, either considering *forward* and *backward* attacks – i.e., attack points are located, respectively, in the earlier or final rounds of the cipher.

Likely probing points are identified as a subset of instruction responsible for mixing the cipher state with key material; the actual set is retrieved by determining the group converging the whole bits of the secret key, and such that each partial coverage is performed at the lowest depth in the data-flow graph.

1.5.5 Propagation

Starting from the key input points, key-bit dependencies are propagated through the whole data-flow graph, identifying for each instruction bit the key bits contributing to it. In particular, *linear* and *non-linear* dependencies are accounted for.

In this stage, the vulnerability index for each attack points is computed. It is formally defined as the number *minimum* number of key bits on which any of the results's bit depends on.

1.5.6 Filtering and Aggregation

The set of attack points is filtered out, selecting only instructions that have a vulnerability index below the user-defined threshold, and satisfy either *forward* or *backward* vulnerability property: An instruction is said to be forward or backward vulnerable if its vulnerability index is below the user-defined threshold and, respectively,

- One of its input operands is marked as vulnerable, *or* is part of the key material.
- It uses an operand depending on key material, *or* an instruction using it depends on the plaintext *and* is marked as vulnerable.

1.5.7 Target Masking

Beginning from the top of the instruction stream, each vulnerable point is masked by applying a specific randomization algorithm, according to the kind of transformation to be protected. Due to the *partial masking* nature of the approach, either *encoding* or *decoding* code is placed, respectively, before and

after each instruction that must be protected; the former gives to the gadget a correct representation of the inputs, while the latter allows use of the gadget output by non-vulnerable instructions.

Linear operations are simply masked by independently applying the same transformation to each input shares. Concerning non-linear operations, CRYPTOSHIELD supports masking for both boolean and arithmetic operations:

- Boolean masking schemes. two boolean masking schemes are employed within CRYPTOSHIELD, the Rivain-Prouff’s variant of the ISW AND, which allows higher-order secure computation of the $\mathbb{F}_{2^m}$ field multiplication [51], and the higher-order look-up table masking of Coron [14], which enables higher-order masking of any look-up table.
- Arithmetic masking schemes. concerning $\mathbb{F}_{2^m}$ field addition and subtraction, three arithmetic masking schemes are employed: for 1st order masking the Karroumi, Richard, and Joye’s masked addition [35] and the Coron et al.’s masked addition [19] are employed; for higher-order masking, instead, the Coron, Großschädl, and Vadnala’s higher order masked addition [16] is employed.

1.6 Boolean masking schemes in CRYPTOSHIELD

The two boolean higher-order masking schemes employed in CRYPTOSHIELD, i.e., the Rivain-Prouff’s variant of the ISW AND and the Coron’s generic higher-order look-up table masking, play a fundamental role in the compiler based solution envisioned in this work. In particular, Chapter 2 provides extensions of their security proofs in the *t-probing with transitions* security model. Hence, to introduce the reader the required knowledge to understand the extension of the security proofs, this section concludes the background chapter by detailing the algorithms of the two masking schemes, and reporting their formal security proof in the classical *t-probing* model.

1.6.1 ISW AND - Rivain-Prouffs variant

The Rivain-Prouffs variant (Algorithm 9), from now on referred to as RP scheme, is a transposition of the original ISW AND higher-order masking scheme [33], from the $\mathbb{F}_2$ field to the $\mathbb{F}_{2^m}$ field, with $m > 1$ [51].

The goal of the scheme is the computation of the $\mathbb{F}_{2^m}$ field multiplication $\bar{c} = \bar{a} \otimes \bar{b}$, where $\bar{a}$ and $\bar{b}$ are two probabilistic encodings given in input, without disclosing a relevant amount of information (that is, all the shares of one of the two inputs) to an external observer. This is achieved in two steps:

1. **Symmetric cross-terms addition (Line 2-5).** For all $0 \leq i, j \leq n \wedge i \neq j$, the linear $\mathbb{F}_{2^m}$ field addition of cross-terms $\bar{a}_i \bar{b}_j$ and $\bar{b}_i \bar{a}_j$ is securely computed as $z_{j,i} = (\bar{a}_i \bar{b}_j \oplus z_{i,j}) \oplus \bar{b}_i \bar{a}_j$, where $z_{i,j} \xleftarrow{\$} \mathcal{U}_{2^m}$ is a fresh random value drawn for each pair of symmetric cross-terms to be added.

2. **Output shares generation (Line 6-10).** For all $0 \leq i \leq n$, the output share $\bar{c}_i$ is calculated by first generating the cross-term $\bar{a}_i \bar{b}_i$ (Line 7), then combining it with the intermediate values $z_{i,j}, z_{j,i}$, for all $0 \leq j \leq n \wedge j \neq i$, computed in step (1).

Formal security proof The RP scheme comes with a formal security proof in the t -probing security model, which is the same model where the ISW AND scheme's security is proved. However, a major difference with the proof of the original scheme is the assumption on the input encodings, which are assumed to be *mutually independent*. Such assumption allows the scheme to reach a tighter bound on the security order: with n^{th} -order masking, the scheme guarantees a security of order n , instead of the original $(\frac{n}{2})^{th}$ -order security. Before reporting the proof, the security theorem proved is recalled.

Theorem 1. RP Scheme Security. *Let $(a_i)_{0 \leq i \leq n}$ and $(b_i)_{0 \leq i \leq n}$ be two n -independent $(n + 1)$ -families of shares in input of Algorithm 9. Then, the distribution of every tuple of n or less intermediate variables in Algorithm 9 is independent of the distribution of values taken by $a = \llbracket \bar{a} \rrbracket = \bigoplus_{0 \leq i \leq n} a_i$ and $b = \llbracket \bar{b} \rrbracket = \bigoplus_{0 \leq i \leq n} b_i$.*

Proof [51]. Our proof consists in constructing two sets I and J of indices in $[0; n]$ with cardinalities lower than or equal to n and such that the distribution of any probabilistic encoding of n element $\bar{v}$ of intermediate variables of Algorithm 9 can be perfectly simulated from a subset of shares of $\bar{a}$ and $\bar{b}$. This will prove the theorem statement since, by definition, $\bar{a}_{|I}$ and $\bar{b}_{|J}$ are jointly independent from a and b as long as the cardinalities of I and J are strictly smaller than n . The constructions of I and J is described hereafter.

1. Initially, I and J are empty and all the $\bar{v}'_h$ s are unassigned.
2. For every intermediate variable $\bar{v}_h$ of the form $\bar{a}_i, \bar{b}_i, \bar{a}_i \bar{b}_i, \bar{r}_{i,j}$ (for any $i \neq j$) or a sum of values of the above form (including c_i as a special case) add i to I and J . This covers all the intermediate variables of Algorithm 9 except those appearing in the computation of $\bar{r}_{j,i}$ which are of the form $\bar{a}_i \bar{b}_j$ or $\bar{r}_{i,j} \oplus \bar{a}_i \bar{b}_j$. For those intermediate variables add i to I and j to J .
3. Now that the sets I and J have been determined – and note that since there are at most n intermediate variables $\bar{v}_h$, the cardinalities of I and J can be at most n – we show how to complete a perfect simulation of the probabilistic encoding $\bar{v}$ using only the values of $\bar{a}_{|I}$ and $\bar{b}_{|J}$. First, we assign values to every $\bar{r}_{i,j}$ entering in the computation of any $\bar{v}_h$ as follows:
 - If $i \notin I$ (regardless of j), then $\bar{r}_{i,j}$ does not enter into the computation for any $\bar{v}_h$. Thus, its value can be left unassigned.

- If $i \in I$, but $j \notin I$, then $\bar{r}_{i,j}$ is assigned a random independent value. Indeed, if $i < j$ this is what would have happened in Algorithm 9. If $i > j$, however, we are making use of the fact that $\bar{r}_{j,i}$ will never be used in the computation of any $\bar{v}_h$ (otherwise we would have $j \in I$ by construction). Hence we can treat $\bar{r}_{i,j}$ as a uniformly random and independent value.
 - If $\{i, j\} \subseteq I$ and $\{i, j\} \subseteq J$, then we have access to $\bar{a}_i, \bar{a}_j, \bar{b}_i$ and $\bar{b}_j$ and we thus compute $\bar{r}_{i,j}$ and $\bar{r}_{j,i}$ exactly as they would have been computed in Algorithm 9; i.e., one of them (say $\bar{r}_{i,j}$) is assigned a random value and the other $\bar{r}_{j,i}$ is assigned $\bar{r}_{i,j} \oplus \bar{a}_i \bar{b}_j \oplus \bar{a}_j \bar{b}_i$.
 - If $\{i, j\} \subseteq I$ and $\{i, j\} \not\subseteq J$, then at least $\bar{r}_{i,j}$ or $\bar{r}_{j,i}$ (or both) does not enter into the computation for any $\bar{v}_h$ (otherwise we would have $\{i, j\} \subseteq J$ by construction). Following the same reasoning as previously (case $i \in I, j \notin I$), we can then assign a random independent value to the one (if any) that enters in the computation of the $\bar{v}_h$'s.
4. For every intermediate variable $\bar{v}_h$ of the form $\bar{a}_i, \bar{b}_i, \bar{a}_i \bar{b}_i, \bar{r}_{i,j}$ (for any $i \neq j$), or a sum of values of the above form (including $\bar{c}_i$ as a special case), we know that $i \in I$ and $i \in J$, and all the needed values of $\bar{r}_{i,j}$ have already been assigned in a perfect simulation. Thus, $\bar{v}_h$ can be computed in a perfect simulation.
 5. The only types of intermediate variables remaining are $\bar{v}_h = \bar{a}_i \bar{b}_j$ or $\bar{v}_h = \bar{r}_{i,j} \oplus \bar{a}_i \bar{b}_j$. By construction, we have $i \in I$ and $j \in J$ which allows us to compute $\bar{a}_i \bar{b}_j$, and since all the $\bar{r}_{i,j}$ (entering into the computation of the $\bar{v}_h$'s) has been assigned, the value of $\bar{v}_h$ can be simulated perfectly.

1.6.2 Coron's Higher-Order Look-up Table Masking

The higher-order masking scheme for look-up tables developed by Coron [14], referred to as CORON scheme from now on, is an extension of the original re-computation method detailed by Chari et al. in [13].

Given a look-up table

$$S : \{0, 1\}^k \rightarrow \{0, 1\}^{k'}$$

with $k \leq k'$, and a query to the index x , for which the look-up $y = S(x)$ would be performed, the scheme takes in input an encoding $\bar{x}$ of x , and returns an encoding $\bar{y}$ such that $y = \llbracket \bar{y} \rrbracket = \bigoplus_{0 \leq i \leq n} y_i = S(x)$.

The scheme performs the look-up by re-computing $n - 1$ times the look-up table; at time $1 \leq i \leq n - 1$, each entry $u \in \{0, 1\}^k$ is *re-wired* to the entry corresponding to the index $u \oplus \bar{x}_i$. Such re-wiring is referred to as *shifting*. Once all the entries has been shifted by $\bar{x}_i$, the whole shifted table is refreshed, by combining each entry with a dedicated set of random values. Finally, at

the end of the $n - 1$ shifting, the output encoding $\bar{y}$ is retrieved by quering the re-computed table with the index $\bar{x}_n$.

The masking scheme, which pseudo-code is reported in Algorithm 10, makes use of two temporary tables, T and T' , with k bits in input, and both storing, for each index, n elements of k' bits:

$$T, T' : \{0, 1\}^k \rightarrow (\{0, 1\}^{k'})^n$$

where, for $1 \leq j \leq n$, $T(u)[j]$ and $T'(u)[j]$ represent the j -th element of the u -th entry stored, respectively, in table T and T' .

The scheme operates in three steps:

1. **Initialization (Line 2-3).** The T table is initialized by performing, for each $u \in \{0, 1\}^k$, the assignments $T(u)[0] \leftarrow S(u)$, and $T(u)[j] \leftarrow 0$ for all $1 \leq j \leq n$.
2. **Shifting (Line 4-9).** For all $1 \leq i \leq n - 1$, the table T is shifted by $\bar{x}_i$, by first performing the assignment $T'(u)[j] \leftarrow T(u \oplus \bar{x}_i)[j]$; then the T table is updated with $T(u) \leftarrow \text{RefreshMasks}(T'(u))$. Note that at the end of step $1 \leq i \leq n - 1$ (Line 9), for each $u \in \{0, 1\}^k$ holds the following:

$$\bigoplus_{j=0}^n (T(u)[j]) = S(u \oplus \bar{x}_0 \oplus \dots \oplus \bar{x}_i)$$

3. **Look-up (Line 10).** At the end of the shifting step, for each $u \in \{0, 1\}^k$ holds the following:

$$\bigoplus_{j=0}^n (T(u)[j]) = S(u \oplus \bar{x}_0 \oplus \dots \oplus \bar{x}_{n-1})$$

Hence, $y = \bigoplus_{j=0}^n (T(\bar{x}_n)[j]) = S(\bar{x}_n \oplus \bar{x}_0 \oplus \dots \oplus \bar{x}_{n-1}) = S(x)$. A final application of **RefreshMasks** is applied to the retrieved $\bar{y}$.

Formal security proof The security of the scheme was firstly proved in the t -probing security model, where the scheme is guaranteed to achieve $(\frac{n}{2})^{th}$ -order security with n shares [14]. Later, Coron, Rondepierre, and Zeitoun prove the CORON scheme to be secure in the t -probing security model extended with the t -SNI property, guaranteeing $(\frac{n}{2})^{th}$ -order security with n shares [18]. Due to its interesting improvements, in the following is reported the proof in the latter setting. Before exposing the proof, the proved security theorem is hereby recalled.

Theorem 2. (t -SNI of Coron scheme). *Let $(x_i)_{0 \leq i \leq n}$ be the input and let $(y_i)_{0 \leq i \leq n}$ be the output of Algorithm 10. For any set of t intermediate variables and any subset O of output indices such that $t + |O| < n$, there exists a subset I of input indices with $|I| \leq t$, such that the t intermediate variables and the output variables $\bar{y}_{|O}$ can be perfectly simulated from $\bar{x}_{|I}$.*

CHAPTER 1. BACKGROUND AND CURRENT STATE OF THE ART40

Proof [18]. We prove the security of a larger circuit than the original circuit corresponding to Algorithm 10, in order to have a recursive security proof. Namely we assume that instead of performing a final table-look up at Line 10 of Algorithm 10, one performs again a final shift of the full table by $\bar{x}_n$, followed by a REFRESHMASK of all the rows, and eventually the full table T is returned, instead of only a single row. The output $\bar{y}$ in Algorithm 10 then corresponds to the row found at index 0 when the table T has been shifted by $\bar{x}_n$. Since we are considering a larger circuit, and moreover the outputs of the smaller circuit are a subset of the outputs of the larger circuit, the t -SNI security of the larger circuit implies the t -SNI security of the original circuit; namely we are only giving more power to the adversary. More precisely, we consider an algorithm HT_i taking as input $\langle \bar{x}_1, \dots, \bar{x}_i \rangle$ and outputting a table T with n output shares such that for all $u \in \{0, 1\}^k$:

$$\bigoplus_{j=1}^n T(u)[j] = S(u \oplus \bar{x}_1 \oplus \dots \oplus \bar{x}_i)$$

The algorithm HT_i can be defined recursively as follows. We define HT_0 as outputting the table T with n output shares, with $T(u) = S(u, 0, \dots, 0)$ for all $u \in \{0, 1\}^k$. The algorithm HT_{i+1} takes as input $\bar{x}_1, \dots, \bar{x}_{i+1}$, and the algorithm HT_i is first recursively applied on $\bar{x}_1, \dots, \bar{x}_i$. A shift by $\bar{x}_{i+1}$ is then applied on the table T returned by HT_i , followed by a REFRESHMASKS of all the rows of the table T ; we denote by SR the combination of the *shift* by $\bar{x}_{i+1}$ and the subsequent REFRESHMASKS. We denote by I^i the set of observations made in the gadget HT_i , with $t_i = |I^i|$ and I' the set of observations made in the gadget SR.

We prove recursively that the HT_i algorithm satisfies the t -SNI property. This means that for any set of $t_i = |I^i|$ intermediate variables and any subset O_i of output indices such that $t_i + |O_i| < n$, there exists a subset I^i of input indices with $|I^i| \leq t_i$, such that the t_i intermediate variables and the output variables $T|O_i$ can be perfectly simulated from $x|I^i$. We denote by $T|O_i$ the set of variables $T(u)[j]$ for all $j \in O_i$ and all rows $u \in \{0, 1\}^k$. It is easy to check that HT_0 is t -SNI, since it does not take any share as input, and it processes only public values. Assuming now that HT_i is t -SNI, we must prove that HT_{i+1} is t -SNI. Therefore letting $t_{i+1} = |I^{i+1}|$ be the number of probes in the circuit, we have the condition:

$$t_{i+1} + |O_{i+1}| \leq n \tag{1.8}$$

and we must show that the t_{i+1} probed variables and the output variables $T|O_{i+1}$ can be perfectly simulated from $\bar{x}|I^{i+1}$, for some I^{i+1} with $|I^{i+1}| \leq t_{i+1}$.

We first consider the SR gadget corresponding to the shift of the table T by $\bar{x}_{i+1}$, followed by a REFRESHMASKS of all the rows of the table T . We apply Lemma 1 to the REFRESHMASKS performed on the rows of the table T . We obtain that all probed intermediate variables and all output variables

corresponding to O_{i+1} can be perfectly simulated from the input variables corresponding to O_i , where $O_i = O_{i+1} \cup J$ with $|J| \leq |I'|$. We now consider the HT_i gadget, for which the output variables corresponding to O_i must be simulated. We have from $t_{i+1} = |I^{i+1}| = |I^i| + |I'|$ and (1.8):

$$|I^i| + |O_i| \leq |I^i| + |O_{i+1}| + |J| \leq |I^i| + |O_{i+1}| + |I'| \leq t_{i+1} + |O_{i+1}| < n$$

Therefore the t -SNI condition is recursively satisfied for Gadget HT_i , and all probed intermediate variables and all output variables corresponding to O_i can be perfectly simulated from $\bar{x}_{|I^i|}$, with $|I_i| \leq |I^i|$. It remains to show that the simulation of both gadgets HT_i and SR can be performed from some subset I_{i+1} with $|I_{i+1}| \leq t_{i+1}$. We distinguish two cases:

- If $|I'| = 0$, then the SR gadget has not been probed and we can apply Lemma 2. Since $|O_{i+1}| < n$, thanks to the REFRESHMASK of every row we can simulate all output variables corresponding to O_{i+1} without the knowledge of any input variables. In particular, we don't need to know $\bar{x}_{i+1}$ to perform that simulation, and we can let $I_{i+1} = I_i$.
- If $|I'| \geq 1$, the knowledge of $\bar{x}_{i+1}$ is required to perform the simulation of the SR gadget, so we let $I_{i+1} = I_i \cup \{i+1\}$.

In both cases we obtain $|I_{i+1}| \leq |I_i| + |I'|$. From $|I_i| \leq |I^i|$, we obtain:

$$|I_{i+1}| \leq |I_i| + |I'| \leq |I_{i+1}| = t_{i+1}$$

which shows that HT_{i+1} is t -SNI. This terminates the proof.

Algorithm 3: Basic Register Allocator

Input: Function F
Data: S_{LI} , set of live intervals
 $M_{LI,W}$, map from live intervals to spill weight
 Q_{LI}^P , priority queue of live intervals
 Q_{Phys} , ordered list of physical registers
Output: $M_{LI,Phys}$, map from live intervals to assigned physical registers

```

1. begin
2.    $M_{LI,W} \leftarrow []$ 
3.   for  $l \leftarrow S_{LI}$  do
4.     // Compute spill weight for each interval
5.      $M_{LI,W}[l] \leftarrow \text{getSpillWeight}(l)$ 
6.      $Q_{LI}^P \leftarrow l$ 
7.    $M_{LI,Phys} \leftarrow []$ 
8.   for  $l \leftarrow Q_{LI}^P$  do
9.      $A \leftarrow \text{false}$  // Is the current physical register allocated?
10.    for  $p \leftarrow L_{Phys}$  and not  $A$  do
11.       $L_{Cand} \leftarrow []$ 
12.      // Assign free phys reg to live range
13.      if not  $\text{checkInterference}(M_{LI,Phys}, l, p)$  then
14.        assign( $M_{LI,Phys}, l, p$ )
15.         $A \leftarrow \text{true}$ 
16.        continue
17.      else
18.        // Record candidate for later spilling
19.         $L_{Cand} \leftarrow p$ 
20.    // Attempt spilling a cheaper virtual register
21.    for  $c \leftarrow L_{Cand}$  and not  $A$  do
22.       $L'_{LI} \leftarrow []$ 
23.      spill( $l, c$ )
24.      if not  $\text{checkInterference}(M_{LI,Phys}, l, c)$  then
25.        assign( $M_{LI,Phys}, l, c$ )
26.         $A \leftarrow \text{true}$ 
27.        enqueue( $Q_{LI}^P, L'_{LI}$ )
28.        continue
29.    // No other virtual register can be spilled; spill
30.    // the current one
31.    if not  $A$  then
32.       $L'_{LI} \leftarrow []$ 
33.      spill( $l$ )
34.      enqueue( $Q_{LI}^P, L'_{LI}$ )
35.  return  $M_{LI,Phys}$ 

```

Algorithm 4: checkInterference - Look for overlapping virtual registers

Input: $M_{LI,P}$, map from live intervals to allocated physical registers

l , candidate live interval

p , candidate physical register

Data: $M_{P,Q_{LI}}$, map from physical registers to ordered list of assigned live intervals

Output: **true**, if an interference is detected; **false**, otherwise

```

1. begin
2.   for  $l' \leftarrow M_{P,Q_{LI}}[p]$  do
3.     if overlaps( $l, l'$ ) then
4.       return true
5.   return false

```

Algorithm 5: checkHazard - Check for Structural Hazards.

Input: i , instruction

Data: T , resource tracker

N_W , maximum number of issuable instruction at each cycle

N_I , number of instructions issued in the current cycle

C , current cycle

Output: If a hazard detected, **true**; otherwise, **false**

```

1. begin
2.   if  $N_I + 1 > N_W$  then
3.     return true
   // For each consumed resource, checks if it is
   // available.
4.   for  $r \in T : \text{consumed}(r, i)$  do
5.     if freedAt( $r$ ) >  $C$  then return true
6.   return false

```

Algorithm 6: getRegions - Divide Basic Block in Sub-Blocks.

Input: B , basic block

Output: Q_R , list of regions

```

1. begin
2.    $Q_R \leftarrow []$                                      // Ordered list of regions
3.   for  $i \leftarrow B$  do
4.     if not isBoundary( $i$ ) then
5.        $R \leftarrow i$ 
6.     else
7.        $Q_R \leftarrow R$                                  // Store region
8.        $R \leftarrow []$                                  // Initialize new region
9.   return  $Q_R$ 

```

Algorithm 7: pickBest - Retrieve the best available instruction.

Input: none

Data: Q_R , queue of ready instructions
 Q_P , queue of pending instructions
 T , resource tracker
 N_I , number of instructions issued in the current cycle
 C , current cycle

Output: b , best scheduling candidate instruction

```

1. begin
2.   for  $i \leftarrow Q_P$  do
3.     if not checkHazard( $i$ ) then
4.       remove( $Q_P$ ,  $i$ )
5.       enqueue( $Q_R$ ,  $i$ )
6.   // Elapse cycles until an instruction may
   // be issued.
7.   while empty( $Q_R$ ) do
8.      $C \leftarrow C + 1$ 
9.      $N_I \leftarrow 0$ 
10.    update( $T$ ,  $C$ )
11.    for  $i \leftarrow Q_P$  do
12.      if not checkHazard( $i$ ) then
13.        remove( $Q_P$ ,  $i$ )
14.        enqueue( $Q_R$ ,  $i$ )
15.   $b \leftarrow Q_R$ 
16.  if count( $Q_R$ ) > 0 then
17.    // Find best ready instruction, according to a
    // hard-coded set of heuristics.
18.    for  $i \in Q_R$  do
19.       $b \leftarrow \text{compare}(b, i)$ 
20.  return  $b$ 

```

Algorithm 8: MachineScheduler - Main Logic

Input: F , function**Data:** Q_R , queue of ready instructions Q_P , queue of pending instructions T , resource tracker N_W , maximum number of issuable instruction at each cycle N_I , number of instructions issued in the current cycle C , current cycle**Output:** F^* , scheduled function

```

1. begin
2.    $F^* \leftarrow []$  // Ordered list of basic blocks.
3.    $B^* \leftarrow []$  // Ordered list of instructions.
4.   for  $B \leftarrow F$  do
5.      $Q_{Reg} \leftarrow \text{getRegions}(B)$ 
6.     for  $R \leftarrow Q_{Reg}$  do
7.        $C \leftarrow 1$ 
8.        $N_I \leftarrow 0$ 
9.       for  $i \leftarrow R$  do
10.        if isReady( $i$ ) then enqueue( $Q_R$ ,  $i$ )
11.      while not empty( $R$ ) do
12.         $c \leftarrow \text{pickBest}()$  // retrieve best in  $Q_R$ .
13.        enqueue( $R^*$ ,  $c$ )
14.        remove( $R$ ,  $c$ )
15.        // Release instructions with ready operands.
16.        for  $s \leftarrow R : \text{isSuccessor}(s, i)$  and readyOps( $s$ ) do
17.          if checkHazard( $s$ ) then
18.            enqueue( $Q_P$ ,  $c$ )
19.          else
20.            enqueue( $Q_R$ ,  $c$ )
21.          // Update scheduling state.
22.          update( $T$ ,  $c$ )
23.           $N_I \leftarrow N_I + 1$ 
24.          if  $N_I > N_W$  then
25.            update( $T$ ,  $C$ )
26.             $C \leftarrow C + 1$ 
27.             $N_I \leftarrow 0$ 
28.        enqueue( $B^*$ ,  $R^*$ )
29.      enqueue( $F^*$ ,  $B^*$ )
30.   return  $F^*$ 

```

Algorithm 9: ISW AND - Rivain-Prouff variant [51]

Input: $\bar{a} = \langle a_0, a_1, \dots, a_n \rangle$, $a_i \in \mathbb{F}_{2^m}$, $m > 1$
 $\bar{b} = \langle b_0, b_1, \dots, b_n \rangle$, $b_i \in \mathbb{F}_{2^m}$, $m > 1$
Output: $\bar{c} = \langle c_0, c_1, \dots, c_n \rangle$: $c_i \in \mathbb{F}_{2^m}$, $m > 1$, $c = (\bigoplus_{i=0}^n a_i) \otimes (\bigoplus_{i=0}^n b_i)$

1. **begin**
2. **for** $i \leftarrow 0$ **to** n **do**
3. **for** $j \leftarrow i + 1$ **to** n **do**
4. $z_{ij} \leftarrow \{0, 1\}^m$
5. $z_{ji} \leftarrow (z_{ij} \oplus a_i b_j) \oplus a_j b_i$
6. **for** $i \leftarrow 0$ **to** n **do**
7. $c_i \leftarrow a_i b_i$
8. **for** $j \leftarrow 0$ **to** n **do**
9. **if** $i \neq j$ **then**
10. $c_i \leftarrow c_i \oplus z_{ij}$
11. **return** $\langle c_0, c_1, \dots, c_n \rangle$

Algorithm 10: Coron's Higher-Order Look-Up Table Masking [14]

Input: $\bar{x} = \langle x_0, x_1, \dots, x_n \rangle$, $x_i \in \{0, 1\}^k$
Output: $\bar{y} = \langle y_0, y_1, \dots, y_n \rangle$, $y_i \in \{0, 1\}^k$, such that $\llbracket \bar{y} \rrbracket = \bigoplus_{0 \leq i \leq n} y_i = S(x)$

1. **begin**
2. **forall** $u \in \{0, 1\}^k$ **do**
3. $T(u) \leftarrow (S(u), 0, \dots, 0) \in (\{0, 1\}^k)^n$
4. **for** $i \leftarrow 0$ **to** $n - 1$ **do**
5. **forall** $u \in \{0, 1\}^k$ **do**
6. **for** $j \leftarrow 1$ **to** n **do**
7. $T'(u)[j] \leftarrow T(u \oplus x_i[j])$
8. **forall** $u \in \{0, 1\}^k$ **do**
9. $T(u) \leftarrow \text{RefreshMask}(T'(u))$
10. $\bar{y} \leftarrow \text{RefreshMask}(T(x_n))$
11. **return** $\bar{y}$

Algorithm 11: Rivain and Prouff's Mask Refresh [51]

Input: $\bar{z} = \langle z_0, z_1, \dots, z_n \rangle, z_i \in \{0, 1\}^k$
Output: $\bar{z} = \langle z_0, z_1, \dots, z_n \rangle, z_i \in \{0, 1\}^k$

```

1. begin
2.   for  $j \leftarrow 1$  to  $n$  do
3.      $tmp \leftarrow \{0, 1\}^k$ 
4.      $z_0 \leftarrow z_0 \oplus tmp$ 
5.      $z_j \leftarrow z_j \oplus tmp$ 
6.   return  $\bar{z}$ 

```

Chapter 2

A Compiler-based Solution

This chapter presents the compiler-based solution envisioned to protect software implementation of higher-order masked cryptoschemes. A first section is dedicated to an overview of the method and main points. Then, follows a description of the general computing architecture used as reference for the development of the methodology. The employment of a generic architecture is motivated by the willing to create a general approach, applicable to the largest possible set of devices. Next, the set of informations employed to identify shares of specific encoding are described; for each of the non-linear masking schemes used in CRYPTOSCHIELD, it is described how such informations should be attached to. Eventually, the chapter concludes with a detailed description of the actual modifications carried out on the BASIC register allocator and the MACHINE SCHEDULER; algorithm modification are reported on the original algorithm, and highlighted to the reader as **brick red** pseudo-code.

2.1 Methodology

As discussed in Section 1.3, distance-leakage based side-channel attacks against masking schemes has been shown to dangerously thwart the claimed security level. The work of [54] provides an interesting solution, independent on the software encryption scheme to be protected, and ensuring first-order security at compile time. In this section, a new approach to statically preserve the higher-order security of software masking schemes employed in the protection of cryptoschemes is introduced. Differently from the work of Seuschek, De Santis, and Guillen, where a specific simple single issue architecture is considered, the methodology considers a more generic architecture is together with the complications inherited by this new generality; specifically, the method is designed to work on a generic super-scalar, multiple issue architecture. A piece of information fundamental for the development of the methodology are the DL TAGS, a pair of positive decimal values which allow to uniquely identify a share. By properly *tagging* each intermediate value with such DL TAGS, it is possible

to instruct the register allocation and the instruction scheduler algorithms to avoid, respectively, allocation of physical register and re-organizations of the instruction streams that may cause a register transition involving two shares belonging to the same probabilistic encoding.

Protection of the register allocation phase is simple: given a set of variable to which assign physical registers, the allocator must avoid the consecutive assignment of the same physical register to two variables representing shares of the same probabilistic encoding; the verification of what share an intermediate represent is enabled thanks to the DL TAGS they have assigned to.

Concerning the hardening of the instruction scheduling, the basic idea is to precisely track the usage of micro-architectural buffers, and use such knowledge to guide the decisions made by the scheduler: each time the scheduler is required to choose an instruction to be scheduled, it must account for the additional constraint on the micro-architectural buffers. Specifically, the scheduler once has chosen an instruction to be scheduled, it has to check if its input operands and result do not clash with other intermediate values stored in the same micro-architectural buffers; further, auxiliary storage elements assigned to the computing units within the pipeline, e.g., a potential memory data register (mdr) of load/store, should be checked too for potential leakage. When the scheduler identifies a leakage with the scheduling of an instruction candidate, two cases are possible:

- Other instructions may be safely scheduled.
- No other instruction may be scheduled.

The former allows to continue the re-organization, although the final program might be sub-optimal with respect to the cost function of the scheduler, e.g., overall latency of the scheduled program. In the latter case, the scheduling is locked. Instead of aborting the whole compilation, a solution is to clean the buffers where leakage occurs, by issuing an ad-hoc *cleaning* instruction; such step is referred to as *whitening*. How the cleanign instruction is created is critical. The solution adopted is to *recycle* the shares already available in the physical registers, and used them as inputs for the cleaning instruction. Using shares already in the physical registers allows immediate issuing of a cleaning instruction, avoiding shortcomings caused by on-demand generation of random values. Note that, by construction, shares are random values, different for each invocation of the algorithm. Recycle of shares, thus of randomness, guarantees a minor randomness overhead on the target platform, and improve the overall latency of the masked implementation, making this approach ideal for constrained devices.

In the remainder of this chapter, the term *leakage-awareness* is used to refer to the ability to perform an operation, or sequence of operation, without causing a possible leakage of informations regarding a sensitive value, e.g., the cryptographic key of the cryptoscheme.

Micro-Architectural Knowledge Differently from register allocation, which is somewhat simpler to protect, bringing leakage-awareness to instruction scheduling requires to provide to the algorithm a precise description of the underlying architecture: specifically, it must describe instruction latencies, execution pipelines, functional units and buffers that leak the distance of stored intermediates. Typically, such knowledge is really limited/imprecise¹ or classified, for being part of protected intellectual property. Nonetheless, in [4] the authors propose a novel approach to characterize the micro-architectural features of a platform, resorting only on a physical instance of the device and micro-benchmarks, successfully revealing functional unit, their pipelining, existing buffers and their leakage characteristic for the CORTEX-A7.

2.2 Computing Architecture

Modern microarchitectures, even the simplest, are characterized by a FETCH-DECODE-EXECUTE paradigm: an instruction is *fetched* from the device's memory; *decoded*, identifying the type of operation requested and input operands required; *executed* by the circuit (called *functional unit*, FU) appointed to process the decoded operation.

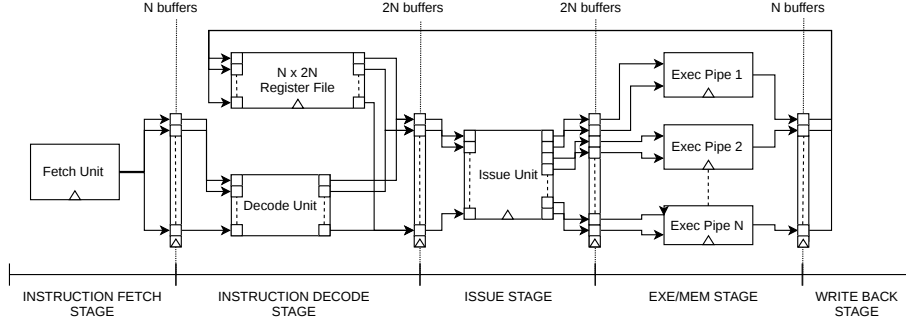
In the scope of this work, instructions are assumed to be fetched, decoded and executed *in-order*.

Super-scalar architectures improve such paradigm by introducing some enhancements:

- **Pipelining:** the microarchitecture is designed as a *pipeline* of phases, where each phase operates, independently, on a different instruction. Each phase, called *pipeline stage*, or simply *stage*, is isolated by a set of registers, the *stage buffers*, responsible for temporarily store the result of the current stage, and feed it to the following one. Although the FETCH-DECODE-EXECUTE pattern is the base for all architectures, particular variants may actually be designed, splitting each phase in two or more stages each separated by stage buffers.
- **Multiple Issuing:** the microarchitecture may show replica of a set of functional units, and stage buffers may store more than one entry, allowing *issuing* of multiple instructions at each clock cycle.

¹The CORTEX-A7 is described as being *partial* dual-issue, but no knowledge is public available on this point.

Figure 2.1: Generic computing architecture. For the development of the methodology, a generic architecture is employed as reference model. The architecture is a n -multiple issue, in-order 5-stage pipeline, which each stage separated by a set of buffer stages. Several execution pipeline are provided, each exposing a subset of possible functional units. To allow computation of any possible program, the first execution pipeline is assumed to be endowed with all the required functional units.



- **FU Pipelining:** some instructions, such as hardware division or floating point operations, may require multiple cycles to be completed, slowing down the whole execution. The same concept of pipelining may be applied to functional units too.

Multiple-issue architectures group functional units into *execution pipelines*, or *lanes*, where each of them may contain (at most) one instance of a given functional unit. In general, the multiple issue capability of an architecture directly depends on the available instances of a specific functional unit: as an example, the CORTEX-A7 provides two (asymmetric) instances of arithmetic-logic units (ALU), allowing dual-issuing of mathematical operations; all other operations can only be issued alone, or paired with a mathematical one, for the presence of a single functional unit instance.

It is assumed that the first execution pipeline is provided with all the units required to consume any possible instruction provided by the ISA. This assumption is fundamental to avoid possible programs that may be non computable by the underlying architecture.

Without loss of generality, for the development of the methodology I refer to a 5-stage, in-order pipelined super-scalar microarchitecture, with the possibility of issuing multiple instructions in each clock cycle (Figure 2.1). In such reference, the EXECUTE stage is split in three phases: the ISSUE phase, where the REGISTER FILE is queried for the operands of the decoded instructions; the EXECUTE/MEMORY phase, in charge of processing arithmetic-logic operations, as well as dealing with memory instructions; the WRITE-BACK phase, committing the results of consumed instructions in an ISA registers.

Each stage buffer is characterized by an arbitrary, but coherent, number of entries. Note that, buffers isolating the ISSUE stage own a double number of entries, since dealing with the input operands of decoded instructions.

The EXECUTE/MEMORY stage encompasses a number of execution pipelines, coherent with the number of available buffer stages' entries; functional units assigned to each lane, and their pipelining, is arbitrary.

2.3 Tagging

To enhance code generation algorithms with leakage-awareness, it is fundamental to determine whether an intermediate value represents a share and, in the positive case, know to which probabilistic encoding it belongs to. Indeed, in order to avoid leaking register transitions, intermediates stored consecutively in the same register must not be part of the same encoding.

To this end, processed values are *tagged* with a pair of informations called DISTANCE LEAKAGE TAGS, or DL TAGS.

Definition 14. Distance Leakage Tags. *Distance leakage tags are defined as a pair $\langle m, s \rangle \in \mathbb{N} \times \mathbb{N} \cup \{\langle \text{NO TAG}, \text{NO TAG} \rangle\}$, where the first element, m , is called MASK ID, while the second element, s , is called SHARE ID.*

The MASK ID is used to identify a probabilistic encoding, while the SHARE ID uniquely identify a share in a given probabilistic encoding. As an example of tagging, let us consider two probabilistic encodings of n elements $\bar{a}, \bar{b}$. The encoding $\bar{a}$ may be assigned with a MASK ID $m_a \in \mathbb{N} \cup \{\text{NO TAG}\}$, while $\bar{b}$ has $m_b \in \mathbb{N} \cup \{\text{NO TAG}\}$ as MASK ID. Then, for all $0 \leq i \leq n$, each share $\bar{a}_i$ is labeled with the DL TAGS $\langle m_a, i \rangle$, while each share $\bar{b}_i$ is labeled with $\langle m_b, i \rangle$.

Note that two probabilistic encodings may share the same MASK ID: this represents the case where two encoding cannot be safely combined together without generating leakage.

To provide overall consistency, a special pair $\{\langle \text{NO TAG}, \text{NO TAG} \rangle\}$ is reserved for values that are not shares, but simple unmasked values. Note that if an intermediate value is assigned a MASK ID $m = \text{NO TAG}$, its SHARE ID must be equal to NO TAG; dually, if an intermediate is assigned a MASK ID $m \neq \text{NO TAG}$, the assigned SHARE ID must be different from NO TAG.

2.3.1 Tagging Masked Transformations

The first elements to be tagged are the shares of input and output encodings of each gadget. However, while for linear transformations this is sufficient, for the shares being independently manipulated, intermediate values generated inside non-linear transformations must be properly tagged too; indeed, as shown in Section 1.3, such intermediates, generated by the combination of more than one share, may reveal several shares with a single observation.

In the scope of non-linear transformations, the assignment of DL TAGS is transformation-dependent, and require a careful analysis to identify which pair of intermediate may leak sensitive informations.

Recall that in CRYPTOSHIELD, the $\mathbb{F}_{2^m}, m \geq 1$ field multiplication and S-BOX look-ups are the non-linear transformations masked, respectively, via Rivain and Prouff's higher-order masking of the field multiplication (Section 1.6.1) and Coron's generic higher-order masking of look-up tables (Section 1.6.2). In the following, the tagging process for each algorithm is described, by considering each possible pairs of intermediate values and evaluating the leakage of a register transition involving each pair.

2.3.2 Tagging the RP scheme

As detailed in Section 1.6.1, the Rivain-Prouff's variant of ISW AND (RP scheme in the following) allows the secure computation of the $\mathbb{F}_{2^m}$ field multiplication of two probabilistic encodings $\bar{a}, \bar{b}$. It guarantees a security of order t for a masking of order $n \geq t$, under the assumption of *mutual independence* of the inputs $\bar{a}$ and $\bar{b}$.

The algorithm securely compute the multiplication by combining intermediate values with a set of random intermediate values, generated with each invocation of the algorithm. However, in order to satisfy the security property, the combinations must be performed following a *strict* order.

Furthermore, as described in Section 1.3.3, the consecutive storing of two intermediates may cause unexpected leakage, violating the ILA assumption on which relies the security proof in the *t-probing* model.

Proper assignment of DL TAGS to sensitive intermediates of the scheme instructs the compiler to preserve the ILA assumption, forbidding dangerous register assignments and scheduling of instructions. In such way, the original proof in the *t-probing* model still holds even in presence of memory transitions.

In order to properly assign the DL TAGS, it is useful recall that the security proof in the *t-probing* model guarantees that each observation performed by an attacker reveal at most one share of the input encodings, and, with t observations, no more than t shares of both input encodings can be learnt. Hence, it is crucial that intermediates causing a register transition revealing more than one input share are tagged in such a way they cannot be stored, consecutively, in the same memory register.

In the following, the register transitions that may take place are analyzed, describing how they should be tagged; the algorithm of the RP scheme is reported to ease the reading.

Remark 1 *Cross-terms $(a_i b_j)$ cannot be consecutively stored into the same register.*

If allowed to consecutively share the same registers, an observation of the register transition would reveal informations on (at most) 4 shares; two a 's and

Algorithm 11: ISW AND - Rivain-Prouff variant [51]

Input: $\bar{a} = \langle a_0, a_1, \dots, a_n \rangle, a_i \in \mathbb{F}_{2^m}, m > 1$
 $\bar{b} = \langle b_0, b_1, \dots, b_n \rangle, b_i \in \mathbb{F}_{2^m}, m > 1$
Output: $\bar{c} = \langle c_0, c_1, \dots, c_n \rangle: c_i \in \mathbb{F}_{2^m}, m > 1, c = (\bigoplus_{i=0}^n a_i) \otimes (\bigoplus_{i=0}^n b_i)$

```

1. begin
2.   for  $i \leftarrow 0$  to  $n$  do
3.     for  $j \leftarrow i + 1$  to  $n$  do
4.        $z_{ij} \leftarrow \{0, 1\}^m$ 
5.        $z_{ji} \leftarrow (z_{ij} \oplus a_i b_j) \oplus a_j b_i$ 
6.   for  $i \leftarrow 0$  to  $n$  do
7.      $c_i \leftarrow a_i b_i$ 
8.     for  $j \leftarrow 0$  to  $n$  do
9.       if  $i \neq j$  then
10.         $c_i \leftarrow c_i \oplus z_{ij}$ 
11.   return  $\langle c_0, c_1, \dots, c_n \rangle$ 

```

two b 's. Recalling the RP's proof by simulation, this is equivalent to increase the sets of observed input shares I, J by two indexes each, instead of one. In a masking of order $n = t$, if (at least) $\frac{t}{2}$ of such register transitions take place, an attacker is able to break a masking of order n with just $\frac{t}{2}$ observation; the security order is effectively halved.

To reinstate the security guarantees of the proof, cross-terms must have the same MASK ID assigned to.

Remark 2 *Output shares (c_i) cannot be consecutively stored into the same register.*

In the security proof of the RP scheme, the observation of a single output share c_i reveals informations on both the input shares involved in its computation. As a consequence, the leakage stemming from the consecutive storing of two output shares $c_i, c_j, i \neq j$ reveals informations on two shares of $\bar{a}$ and $\bar{b}$. As explained in Remark 1, such register transitions may lead an attacker to break a scheme of order $n = t$ with just $\frac{t}{2}$ observations.

To reinstate the security guarantees of the proof, output shares must have the same MASK ID assigned to.

Remark 3 *Symmetric z -terms $(z_{i,j}, z_{j,i})$ cannot be consecutively stored into the same register.*

$z_{j,i} = (a_i b_j \oplus z_{i,j}) \oplus a_j b_i$ and $z_{i,j}$ share the same random value (that is, $z_{i,j}$). It follows that a register transition involving symmetric z -terms discloses

informations on two shares of both inputs $\bar{a}$ and $\bar{b}$: indeed, the $\mathbb{F}_{2^m}$ linear sum $a_i b_j \oplus a_j b_i$ may be observed by an attacker. As explained in Remark 1, if a sufficient number of such transitions may be observed (at least $\frac{t}{2}$ in a masking of order $n = t$), an attacker is able to break a scheme performing a reduced number of observations.

To reinstate the security guarantees of the proof, symmetric z-terms must have the same MASK ID assigned to.

Remark 4 *Asymmetric z-terms $(z_{i,j}, z_{h,k})$ may be consecutively stored into the same register.*

Let us consider a n^{th} -order instance of the RP scheme, and let us consider 4 indexes $0 \leq i \neq j \neq h \neq k \leq n$. There are three cases that must be analyzed for the pair of asymmetric z-terms $z_{i,j}, z_{h,k}$:

1. $z_{i,j}, z_{h,k} \leftarrow \mathcal{U}_{2^m}$.

Since $z_{i,j}$ and $z_{h,k}$ are random values, the information leaked by the transition of the register is random too.

2. $z_{i,j} = (z_{j,i} \oplus a_i b_j) \oplus a_j b_i$, $z_{h,k} = (z_{k,h} \oplus a_h b_k) \oplus a_k b_h$.

$z_{i,j}$ and $z_{h,k}$ are made random by random values drawn from $\mathcal{U}_{2^m}$, that is $z_{j,i}$ and $z_{k,h}$. Hence, the information leaked by the transition of the register is random too.

3. $z_{h,k} \leftarrow \mathcal{U}_{2^m}$, $z_{i,j} = (z_{j,i} \oplus a_i b_j) \oplus a_j b_i$.

$z_{h,k}$ is a fresh random value drawn from the random distribution $\mathcal{U}_{2^m}$, while the $z_{i,j}$ is randomized by a fresh random value, i.e., $z_{j,i}$, drawn from the same distribution. As a consequence, the leakage stemming from the transition of the register is random too.

4. $z_{i,j} \leftarrow \mathcal{U}_{2^m}$, $z_{h,k} = (z_{k,h} \oplus a_h b_k) \oplus a_k b_h$.

See Point 3.

Since a register transition involving $z_{i,j}, z_{h,k}$ leaks a random information, the two intermediates may be assigned two distinct MASK IDs.

Remark 5 *Intermediates values employed for the computation of $\bar{c}$'s shares should not be consecutively stored in the same register.*

Intermediates values combined for the computation of each $\bar{c}_i$ (Algorithm 11, Line 10) are in one of the following form: $z_{i,j}, z_{j,i} = (z_{i,j} \oplus a_i b_j) \oplus a_j b_i$, $c_i \oplus z_{i,j}$, or $c_i \oplus z_{j,i}$, where $z_{i,j} \leftarrow \mathcal{U}_{2^m}$.

Hence, each intermediate embeds a random value, and the transition of a register involving a pair of them leaks a random information. However, as a conservative approach, such intermediates are assigned the same textscmask id:

during the construction of the output shares $\bar{c}_i$, which is performed in n iterations, c_i^k (c_i at iteration k) is assigned the same tags of c_i^j , where $0 \leq j < k \leq n$.

Limitation on the approach The use of a single pair of tags implies the impossibility to properly handle some cases, leaving the tagged implementation open to dangerous observations. For instance, in the scope of a 2^{nd} -order masking of the RP scheme tagged according with the above remarks, and let us consider the following intermediate values:

$$a_0b_2, a_2b_0, z_{0,1}, z_{1,0} = (z_{0,1} \oplus a_0b_1) \oplus a_1b_0.$$

Due to Remark 1, a_0b_2 and a_2b_0 are tagged with the same MASK ID. $z_{0,1}$ and $z_{1,0}$, due to Remark 3, get assigned the same MASK ID. Also, let us consider the following sequence of register assignments

$$\begin{aligned} r_x &\leftarrow a_0b_2 \\ r_y &\leftarrow a_2b_0 \\ r_x &\leftarrow z_{0,1} \\ r_y &\leftarrow z_{1,0} \end{aligned}$$

where r_x and r_y are two distinct registers. Such sequence complies with the above remarks; nonetheless, it is possible break the 2^{nd} -order implementation by observing the transitions taking place in register r_x and r_y : indeed, an attacker would observe the pair $\langle HD(a_0b_2, z_{0,1}), HD(a_2b_0, z_{1,0}) \rangle$, which is statistically correlated to the unshared value a . To properly handle such case, cross-terms must share the same MASK ID with the symmetric z-terms. However, this is not currently possible since a pair of tags is already assigned to each intermediate.

2.3.3 Tagging the CORON scheme

The generic method of Coron (from now on, CORON scheme) allows the secure look-up of any look-up table, guaranteeing a security of order t with a masking of order $n \geq t$.

The scheme is a re-computation based method that, given a generic look-up table

$$S : \{0, 1\}^k \rightarrow \{0, 1\}^{k'}$$

with $k \leq k'$, and given an encoding $\bar{x}$ of the index x to be queried, returns the encoding $\bar{y}$ of the output $y = S(x)$ by computing at run-time a new table T . Re-computation is performed in $n - 1$ steps, by *shifting* the table T by $\bar{x}_i$, with $1 \leq i \leq n - 1$, and refreshing it with a fresh set of random values.

The security proof of the scheme is provided in the *t-probing* model under the *t-SNI* property. However, the proof does not take into account the violation of the ILA assumption, caused by register transitions.

In the following, the register transitions that may take place are analyzed, describing how they should be tagged; the algorithm of the CORON scheme is reported to ease the reading.

Algorithm 12: Coron's Higher-Order Look-Up Table Masking [14]

Input: $\bar{x} = \langle x_0, x_1, \dots, x_n \rangle$, $x_i \in \{0, 1\}^k$
Output: $\bar{y} = \langle y_0, y_1, \dots, y_n \rangle$, $y_i \in \{0, 1\}^k$, such that $\llbracket \bar{y} \rrbracket = \bigoplus_{0 \leq i \leq n} y_i = S(x)$

1. **begin**
2. **forall** $u \in \{0, 1\}^k$ **do**
3. $T(u) \leftarrow (S(u), 0, \dots, 0) \in (\{0, 1\}^k)^n$
4. **for** $i \leftarrow 0$ **to** $n - 1$ **do**
5. **forall** $u \in \{0, 1\}^k$ **do**
6. **for** $j \leftarrow 1$ **to** n **do**
7. $T'(u)[j] \leftarrow T(u \oplus x_i[j])$
8. **forall** $u \in \{0, 1\}^k$ **do**
9. $T(u) \leftarrow \text{RefreshMask}(T'(u))$
10. $\bar{y} \leftarrow \text{RefreshMask}(T(x_n))$
11. **return** $\bar{y}$

Remark 1 *Shares $\bar{x}_i$ of the index $\bar{x}$ cannot be consecutively stored in the same register.*

The register transition involving two shares $\bar{x}_i$ partially discloses the unshared index x . Considering a masking of order $n = t$, where t is the security order of the CORON scheme instance, $\frac{t}{2}$ observations of such transitions are sufficient for revealing the index x .

To avoid such register transitions, the shares $\bar{x}_i$ must be tagged with the same MASK ID.

Remark 2 *Shares of the output $(\bar{y}_i)$ cannot be consecutively stored in the same register.*

As for Remark 1, in a masking of order $n = t$, where t is the security order of the CORON scheme instance, $\frac{t}{2}$ register transitions involving distinct pairs of output's shares are sufficient for revealing the retrieved value $S(x)$; from the latter, since S is not a surjection, an attacker has the possibility to learn the index x .

To avoid such transitions, the shares $\bar{y}_i$ must be tagged with the same MASK ID.

Remark 3 *The shifted index $u \oplus x_i$ and the share $T(v \oplus x_j)[h]$ may consecutively share the same register.*

When performing a shift by the index's share x_i , where $1 \leq i \leq n-1$, the intermediate $u \oplus x_i$ is computed for all $u \in \{0, 1\}$; this value happens to be random due to the random nature of x_i . Furthermore, given any $1 \leq j \leq n-1$ which may coincides with i , and given any $0 \leq h \leq n$, the share $T(v \oplus x_j)[h]$ is random too, for all $v \in \{0, 1\}$. Indeed, if $i \neq j$, the $T(v \oplus x_j)[h]$ has been generated when shifting by x_j ; thus, $T(v \oplus x_j)[h]$ has been refreshed (at least) one time. A register transition involving the two intermediates $u \oplus x_i, T(v \oplus x_j)[h]$ would leak a random information. In case $i = j$, due to the random nature of $u \oplus x_i$, a transition involving the latter and $T(v \oplus x_i)[h]$ would leak a random information too.

Consequently, for all $u, v \in \{0, 1\}^k$ and for all $1 \leq i, j \leq n-1$ and for all $0 \leq h \leq n$, the intermediate values $u \oplus x_i, T(v \oplus x_j)[h]$ may be assigned different MASK IDs.

Remark 4 *The index's shares (x_i) and output' share (y_j) may consecutively share the same register.*

This is a consequence of Remark 3: recalling that $y_j = \text{RefreshMask}(T(x_n[j]))$, for $0 \leq j \leq n$, it follows that a register transition involving x_i and y_j , where $1 \leq i \leq n-1$, leaks a random information.

As for Remark 3, for all $0 \leq i, j \leq n$, index's shares x_i and output' share y_j may get different MASK IDs.

Remark 5 *The intermediate values $u \oplus x_i, v \oplus x_j$ cannot consecutively share the same register.*

Given $1 \leq i \neq j \leq n-1$, for all $u, v \in \{0, 1\}$, the register transition involving $u \oplus x_i$ and $v \oplus x_j$ partially unveils the unshared index x . In a masking of order $n = t$, where t is the security order of the CORON scheme instance, $\frac{t}{2}$ register transitions involving distinct pairs of output's shares are sufficient for revealing the unshared index x .

To avoid such register transitions, for all $u, v \in \{0, 1\}$ and for all $1 \leq i \neq j \leq n-1$, $u \oplus x_i$ and $v \oplus x_j$ must be tagged with the same MASK ID.

Transition leakage between different shiftings. There is still a critical case that must be analyzed, which take places between two consecutive shifts of the T table must be properly. Without loss of generality, let us consider a

2^{nd} -order instance of the CORON scheme, and let us consider the end of the first shift (Algorithm 12, Line 9).

A register may store intermediate values in the form

$$T(u \oplus x_0) \oplus \langle r_i \rangle_0^n \quad (2.1)$$

where $\langle r_i \rangle_0^n = r_0 \oplus r_1 \oplus \dots \oplus r_n$, and each $r_i \in \{0, 1\}^{k'}$ is a fresh random value.

At the beginning of the second shift (Algorithm 12, Line 7), the following intermediate is computed:

$$T'(u \oplus x_1) = T(u \oplus x_1 \oplus x_0) \oplus \langle r_i \rangle_0^n$$

Such intermediates, if consecutively shares the same register of 2.1, causes a register transition leaking the following information:

$$T(u \oplus x_0) \oplus T(u \oplus x_1 \oplus x_0) \quad (2.2)$$

Despite being *syntactically* dependent on two out of three shares of the index x , the potential dependence on x is masked by $T(u \oplus x_0)$: the random nature of x_0 makes the retrieved output random too, effectively randomizing the output of $x_1 \oplus x_0$.

As a consequence, $T(u \oplus x_0)$ and $T(u \oplus x_1 \oplus x_0)$ may share the same MASK ID.

Although being showed for the second order case, and at the very first shifts, this holds true in general.

Remaining cases. Other register transitions involve shares of different entries (thus, masked by different set of fresh shares) – hence causing leakage of a random information – or shares of same entry $u \in \{0, 1\}^k$, where $u \in \{0, 1\}$ but belonging to two different shiftings; again, the two are masked by two different set of fresh shares, and the potential register transition leaks a random information. Hence, the remaining cases may use distinct MASK ID.

Extension to the security proof It is possible to extend the *t-SNI* security proof for the Coron’s generic look-up [18], noting that that the allowed register transitions leak an amount of power which is independent on the input shares; hence, such consumptions may be perfectly simulated by fresh random values. Therefore, the sets of observed input indexes I^i, I' are not touched, preserving the validity of the proof.

2.4 BASIC Register Allocator Hardening

The main logic of the BASIC register allocator (Section 1.4.4) is to explore the set of live intervals in decreasing order of spill weight, assigning to each a physical register which is not already allocated to a live interval with which an

interference occurs. The set of assigned live intervals to a physical register is represented as an ordered sequence of live intervals. Since a live interval is essentially an ordered sequence of non-contiguous live segments, an interference between two live interval happens when (at least) two of their segments overlaps. In case an interference is detected the allocator attempts to find another physical register; if the search fails, the allocator scans again the set of physical registers, looking for a live interval with less spill weight to be spilled. If none is found, the current live interval is marked for spilling.

Enhancing the BASIC register allocator with a distance-leakage policy adds a further constraint on the assignment of a physical register: informally, a live interval representing a share of a probabilistic encoding must not have its live segments adjacent to any other live segment of a live interval representing a share of the same probabilistic encoding. With this constraint, the allocator won't consecutively store in the same register shares of the same encoding.

To allow detection of leakage-induced interferences, *live intervals* are augmented with the DL TAGS marking the intermediate values they represent.

Definition 15. *Interference (leakage-aware).* *Given two live intervals $\mathbb{L}_1, \mathbb{L}_2$, they are said to not interfere, or overlaps, if and only if the following holds true:*

1. $\forall [a, b] \in \mathbb{L}_1, [c, d] \in \mathbb{L}_2 : b \leq c \vee d \leq a$
2. $M([a, b]) = M([c, d]) \wedge S([a, b]) = S([c, d])$ or $M([a, b]) \neq M([c, d])$.

Recall that the `checkInterference` algorithm (Algorithm 5) embeds all the logic responsible for identification of potential live interval interferences; hence, it is sufficient to modify this piece of code to avoid undesirable allocations. Algorithm 10 describes the applied modifications: after it has been verified that the live interval V , assigned to the physical register P , does not interfere with the candidate live interval L (Line 3-4), the algorithm checks whether V and L represents shares belonging to the same probabilistic encoding (1) and if any of L 's live segments is followed (2) or follows (3) a live segment of V (Line 5-9). If at least one the three conditions is not satisfied, the algorithm returns FALSE, stating that the two live intervals do not interfere under the constrained definition of interference (Line 10). Otherwise, if (1) and (2) and/or (1) and (3) are satisfied, the algorithm returns TRUE, notifying the scheduler of an interference between the candidate live interval L and a live interval already assigned to the physical register P (Line 7 and Line 9).

2.5 MACHINE SCHEDULER Hardening

The MACHINE SCHEDULER (MS) is employed both as *pre register allocation* (pre-RA) and *post register allocation* (post-RA) instruction scheduler (Section 1.4.5).

Algorithm 13: checkInterference (leakage-aware) - Verify potential interference under the additional constraint.

Input: $M_{LI,P}$, maps from live interval to allocated physical registers

l , candidate live interval

p , candidate physical register

Data: $M_{P,Q_{LI}}$, map from physical register to ordered list of assigned live interval

Output: **true**, if an interference is detected; **false**, otherwise

```

1. begin
2.   for  $l' \leftarrow M_{P,Q_{LI}}$  do
3.     if overlaps( $l, l'$ ) then
4.       return true
5.     // Check if any of the  $L$ 's live segments precedes
6.     // or follow a  $V$ 's live segment, and  $L$  and  $V$ 
7.     // are distinct shares of the same probabilistic
8.     // encoding.
9.     if precedes( $l', l$ ) and sameEnc( $l', l$ ) then
10.      return true
11.    if follows( $l', l$ ) and sameEnc( $l', l$ ) then
12.      return true
13.  return false

```

Since either forms of stream re-organization are performed by the same module, the logic employed for re-organizing the stream is really similar, up to specific details, e.g., handling of WAR and WAW hazards.

Consequently, the leakage-aware modifications applied are shared between the two forms of scheduling, but some significant differences exist on how the protection is carried out in the critical situation where the scheduler is stuck, unable to find a candidate instruction which does not causes a leakage of informations in the pipeline's buffer.

Core of the modification is the *whitening step*, employed in order to clean micro-architectural buffers when the scheduler is not allowed to schedule an instruction without causing leakage in the pipeline. Informally, in this step an ad-hoc crafted instruction W is created and scheduled, such that the buffers where leakage occurs is removed. The creation of such instruction requires the selection of proper input operands, in order to effectively clean the buffers. These operands must be mutually independent, and must not cause any other leakage in the pipeline. A naïve solution would be the on-demand generation of randomness and its employment. However, it comes with several shortcomings:

1. **Randomness overhead.** The generation of new randomness implies an overhead on the randomness requirement of the protected cryptoscheme.

2. **Register pressure.** The register pressure increases, incrementing the amount of spill and re-load instructions inserted by the register allocator
3. **Semantic preservation.** Generation of randomness in the post-RA phase requires to properly handle potential overwriting of a physical register containing a value used later in the program. In such case, if it is not possible identify a register which value is not reused, the latter must be saved on the stack, and re-loaded once the instruction W has been scheduled. However, this requires to carefully decide which register save to memory, in order to not generate leakage; furthermore, the re-loading of the value may be the cause of the leakage, requiring, then the issuing of another whitening instruction which, in turns, require other randomness. In multiple-issue architecture, the degree of freedom represented by the additional execution lanes makes hard to prevent such problematic.
4. **Execution time overhead.** The execution time of the protected implementation is affected by a higher overhead, which is induced by the three previous points.

The solution adopted in this work is to *recycle* the intermediate values already available in the physical registers. Re-using shares already in the physical registers allows immediate issuing of a cleaning instruction, avoiding the aforementioned problems. Shares, by construction, are random values, different for each invocation of the algorithm. However, gadget composition defaults must be avoided [7]: specifically, intermediate values may belong to two distinct gadgets, which share an input encoding. This may cause violation in the ILA assumption, invalidating the security guarantees of the involved gadgets. To solve this problem, it is possible to refresh, via a t-SNI gadget, the output encoding of a gadget each time it is used as an input

Before detailing how the *whitening phase* is handled, two modules are first introduced: the BUFFER MANAGER and the TAGS MAPPER. These modules extend the capabilities of the MS, by providing, respectively, the possibility to track the actual content of micro-architectural buffers employed in the pipeline, and track the DL TAGS assigned to each virtual/physical register.

BUFFER MANAGER The BUFFER MANAGER (BM) is a module in charge of tracking the state of the pipeline’s buffers within the scheduler’s lifetime. Specifically, for each micro-architectural buffer of a given architecture (either pipeline stages and registers reserved for specific functional units) it records the DL TAGS assigned to the intermediate value stored in a specific buffer; each time the MS selects an instruction to be scheduled, the BM recovers the DL TAGS assigned to the output and input operands, updating the tags assigned to buffer whose state is changed, i.e., it is employed by the scheduled instruction.

A *commit buffer* is provided as a temporary memory to record multiple assignments, which are consequence of multiple instruction scheduling within

the same clock cycles, i.e., multiple issuing of instructions. Buffered entries gets committed once the scheduler moves on to the next clock cycle.

TAGS MAPPER An important aspect of the methodology is the *whitening* step, which, during the instruction stream re-organization, cleans the micro-architectural buffers in case the scheduler cannot schedule an instruction without causing leakage of sensitive informations. This step is actively supported by the TAGS MAPPER (TM), which tracks two sets of informations:

- **Register** $\rightarrow$ **Tags**. This map records what DL TAGS are assigned to a given virtual/physical register.
- **Tags** $\rightarrow$ **Register**. This maps records to what virtual/physical register a given pair of DL TAGS is assigned to.

A *commit buffer* is provided as a temporary memory to record multiple entries, which are consequence of multiple instruction scheduling within the same clock cycles, i.e., multiple issuing of instructions. Buffered entries gets committed once the scheduler moves on to the next clock cycle.

Nonetheless, in the pre-RA scheduling phase, the actual informations tracked in the TM are slightly different: during the pre-RA scheduling phase, since the code is provided to the MS in SSA form and virtual registers are employed, no re-definition may take place. When a cleaning instruction has to be issued, the scheduler may consider an old virtual register as input of the former. Although this is correct from a semantically point of view, either for the hardening and the re-organized function, such virtual register is required to live for an extended period of time, potentially overcoming its lifetime in an unprotected version of the cryptoscheme; hence, implying a higher register pressure, as well as higher latencies due to instructions inserted by the register allocator for spilling and re-loading, and further checks during the instruction scheduling to avoid potential leakages generated by spilling/re-loading code.

The solution adopted is to track *at most three* shares, each belonging to a *different* encoding. For each share, the *age* of each is recorded; the age of an intermediate values is informally defined as the number of clock cycles elapsed from its generation to the instant its age is queried. Such number of shares is the *minimum* required to have a pool of randomness where uncorrelated intermediates are drawn: indeed, tracking just two shares might create the case where their combination generates a value replacing one of the two; an immediate leakage then may require the new value and one of the two old, but their recombination reveal the replaced old share, which may conflict with the result of the leaking instruction. Hence a slightly greater pool allows to have two alternative pairs to choose. Values higher than three are possible to provide a richer randomness pool; however, to provide low register pressure and minimize spilling, the most recent shares should be preferred.

Concerning the post-RA scheduling phase, since re-definition are enabled, and the set of registers is fixed by the target architecture, ages have not to be tracked, and most recent shares are automatically recorded.

Leakage-aware enhancement The leakage-aware enhancement of the MS's logic is achieved by the employment of some auxiliary algorithms; in the following, each new helper routine, and modifications carried out on already existing ones, are described; then, the changes to the MS's logic to employ such routines, introducing leakage-awareness in the instruction scheduling phase, are discussed.

Algorithm 14: checkLeakage - Verifying potential leakage in μ arch buffers

Input: I , instruction
Data: RT , resource tracker
 BM , buffer manager
Output: **true**, if issuing of I generates leakage; **false**, otherwise.

```

1. begin
2.    $Lane \leftarrow \text{getIssueLane}(RT, I)$ 
3.    $ResList \leftarrow \text{getEmployedFunctionalUnits}(RT, Lane, I)$ 
4.    $BufList \leftarrow \text{getMicrobuffers}(ResList, I)$ 
5.   for  $B \in BufList$  do                                // Check for issue lane's buffers
6.       // Check if I's output and B's entry are
7.       // different shares of the same encoding.
8.       if  $\text{sameEnc}(\text{getEntry}(BM, B), I)$  then
9.           return true
10.  for  $R \in ResList$  do                                // Check for resources' buffers
11.       $ResBufList \leftarrow \text{getMicrobuffers}(R)$ 
12.      for  $RB \in ResBufList$  do
13.          // Check if I's output and B's entry are
14.          // different shares of the same encoding.
15.          if  $\text{sameEnc}(\text{getEntry}(BM, B), I)$  then
16.              return true
17.  return false

```

- **Leakage verification.** The checkLeakage auxiliary routine (Algorithm 13) is employed by the MS to verify if a specific instruction i (typically, a scheduling candidate) may cause leakage in one of the micro-architectural registers of the pipeline. First, the resource tracker T is queried for the whole list of stage buffers the instruction i consumes when issued in the specific execution pipeline (Line 2 - 4). For each of the retrieved buffers, the BM is asked for the entry stored, and it is checked whether the in-

puts or the output of the instruction i are assigned the same DL TAGS of the stored intermediate value, returning TRUE in such case (Line 6). If the check on the stage buffers is negative, the algorithm continues by retrieving the buffers assigned to the functional units processing the instruction i , and verifying whether the output of the instruction i may cause leakage with entries stored in such buffers, returning TRUE in the positive case (Line 8 - 12). If the check is negative in both the above cases, FALSE is returned, stating that the instruction i does not generate leakage (Line 13).

- **Best candidate selection.** The `pickBest` algorithm is responsible for searching in the queue of ready instructions Q_R the best scheduling candidate, according to a set of pre-defined heuristics. The leakage-aware variant of this routine (Algorithm 24) modifies how the best scheduling candidate is selected: if both the current best candidate b and the current instruction i extracted from the ready queue Q_R leaks, or both of them do not leak, use the set of heuristic to determine which improves the scheduling (Line 17 - 20). In the case where only the current best candidate B leaks, the algorithm evaluates the current instruction I as a better option to be scheduled (Line 22): although creating a sub-optimal scheduling, this prevents the execution of the whitening step before scheduling the best candidate, which is required in order to remove the leakage stemming in the pipeline; hence avoiding the overheads implied by issuing instructions in charge of removing the leakage. Otherwise, if the current instruction i leaks, it keeps b as the best candidate and continue the exploration of Q_R (Line 23).
- **Whitening strategy.** The whitening strategy, or whitening step, is a target-specific strategy executed when the best candidate b to be scheduled causes a leakage in one of the micro-architectural buffers it employs. The strategy, represented by the `applyWhiteningStrategy` routine, is *target-specific* in the sense that the employed instructions depends on the set of instructions provided by the target, and the actual sequence of instructions depends on the actual pipeline's structure (e.g., allows multiple-issuing of instructions). Nonetheless, the overall idea of the strategy is common to all the possible implementations, and acts by:
 1. **Leakage identification.** The inputs and output operands of the candidate b are analyzed, verifying if they cause leakage. There are 4 cases to be considered: (1) Only one of the two input operands causes leakage; (2) Both input operands causes leakage; (3) The output operand causes leakage; (4) Leakage occurs in one of the buffers of the functional units processing b .
 2. **Cleaning instruction generation.** In order to clean the buffers where leakage occurs, a proper cleaning instruction W must be scheduled. The generation must properly select the opcode of the

instruction W and its input operands: in case (1), (2) and (3) any opcode (thus, any instruction) theoretically fits, since the input operands are stored only in the ID/IS and IS/EXE stages, and the output is stored in the EXE/WB stage; however, the functional units processing the cleaning instruction W may have a dedicated buffer, which entry might cause a leakage with the intermediate value generated by the inputs assigned to W . For case (4), it is fundamental to choose an opcode (thus, an instruction) that targets the functional unit's buffer where leakage arises. Hence, it is chosen the same opcode of b . Concerning the selection of the input operands for W , in case (1), the TM is queried for a register with a pair of DL TAGS different from the one assigned to the input causing leakage; the other input register is simply copied from the candidate b . In the three other cases, two input operands for W are retrieved by querying the TM for two registers, which DL TAGS are different with respect to the tags assigned to the inputs of the instruction generating leakage. A further constraint on the selection of the input registers for the cleaning instruction W is that their DL TAGS must be *mutually* different; otherwise, a recombination of shares belonging to the same probabilistic encoding would take place.

Concerning the post-RA scheduling phase, the whitening step must take into account the *semantic* of the program: specifically, during the post-RA scheduling, the construction of the cleaning instruction W encompasses the selection of a physical *destination* register. Its selection cannot be performed randomly, since W may overwrite a value which is re-used later in the program; hence, causing violation of the program's semantic. The destination register should be selected among the ones containing dead intermediates (i.e., not re-used in the program); if none is present, it is possible to use the destination register of the instruction b , but its content must be spilled before scheduling the cleaning instruction W . After the issuing of W , the original value may be reloaded in the physical register from which it has been evicted. These semantic-preserving operations are not required for the pre-RA scheduling phase, since the code is SSA form and each new instruction defines a new virtual registers.

- **Buffer Manager update.** The `updateBuffers` utility is in charge of recording, for each buffer employed in a given clock cycle, the DL TAGS assigned to the intermediate value stored in such buffer; these informations are stored in the commit buffer of the BM until the scheduler shifts to the next clock cycle. The routine queries the resource tracker T for the execution lane l where the scheduling candidate i is issued, and properly updates the stage buffers: ID/IS and IS/EXE are updated with the tags assigned to C 's inputs, while the tags assigned to i 's output are assigned to the EXE/WB stage buffer, according to the computing reference 2.1) (Line 2 - 3). The micro-architectural buffers assigned to the functional

Algorithm 15: updateBuffers - Update micro-architectural buffers**Input:** i , instruction which result and input operands fill the buffers**Data:** T , resource tracker
 M_{Buff} , buffer manager**Output:** none

```

1. begin
2.    $l \leftarrow \text{getIssueLane}(T, i)$ 
   // Update pipe-stage of target execution lane.
3.    $\text{update}(M_{Buff}, l, i)$ 
4.    $Q_{Res} \leftarrow \text{getEmployedFnUnits}(T, l, i)$ 
5.   for  $r \leftarrow Q_{Res}$  do
6.      $Q_{Buff} \leftarrow \text{getMicrobuffers}(R)$ 
7.     for  $b \leftarrow Q_{Buff}$  do
8.        $\text{update}(M_B, b, i)$ 

```

units processing i are updated as well, by first identifying the functional units belonging to the lane l and consuming the candidate i (Line 4); then, for each unit, the BM's entry associated to the unit's buffer is updated with the DL TAGS associated to i 's output (Line 5 - 8).

Algorithm 16: updateTags - Update tracked tags (pre-RA version)**Input:** i , candidate instruction**Data:** M_{Tags} , tags mapper**Output:** none

```

1. begin
2.    $t \leftarrow \text{getTags}(i)$ 
3.    $r \leftarrow \text{getDstReg}(i)$ 
4.   if  $\text{isFull}(M_{Tags})$  then  $\text{removeOldest}(M_{Tags})$ 
   // Track most recent mask.
5.   if  $\text{isTracked}(M_{Tags}, \text{getMaskID}(t))$  then
6.      $\text{unmap}(r, t)$ 
7.      $\text{unmap}(t, r)$ 
   // Used to efficiently retrieve tags of in-register values.
8.    $\text{map}(r, t)$ 
   // Used to efficiently find an uncorrelated value.
9.    $\text{map}(t, l)$ 
10.  return

```

- **Tags Mapper update.** The `updateTags` routine updates the pieces of informations tracked by the TM module. Algorithm 10 reports the pseudo-code for the pre-RA variant of the `updateTags` utility: recall that in the pre-RA scheduling, the most recent three intermediate val-

ues, which DL TAGS are distinct, are recorded. The tags assigned to the scheduling candidate i are retrieved, together with the i 's destination register (Line 2 - 3). In case the TM already tracks three intermediate values, it removes the oldest one (i.e., the one which age is greater than the two other's) (Line 4). Since the intermediate value generated by i is newer than any others, a potential register with a MASK ID identical to the MASK ID of i 's output is removed from the tracked entries of TM (Line 5 - 7). Finally, two new pairs of informations $\langle i$'s output, Tags $\rangle$ and $\langle$ Tags, i 's output $\rangle$ are recorded in TM's commit buffer; as the MS moves to the next clock cycle, the entries in the commit buffer are permanently registered (Algorithm 34, Line 27 - 31).

Concerning the update of the TM in the post-RA scheduling, the logic reduces to the simple recordings of the new information pairs.

- **Main logic.** When the best scheduling candidate i is retrieved (Line 12), before scheduling it, the scheduler checks if its issuing may create leakage in the buffers of the pipeline; in the positive case, the pipeline buffers are cleaned by means of a *whitening strategy* (Line 14). Once the candidate i has been scheduled (Line 15 - 16), the scheduler's state *and* the state of the BM and TM are updated too, by inserting in the respective commit buffers the new entries. (Line 22 - 24). When the maximum number of issuable instruction is reached, the scheduler shifts to the next clock cycle, and the buffered entries are finally committed (Line 27 - 31).

Algorithm 17: pickBest - Get best ready instruction (Leakage-aware)

Input: none**Data:** Q_R , queue of ready instructions
 Q_P , queue of pending instructions
 T , resource tracker
 N_I , number of instructions issued in the current cycle
 C , current cycle**Output:** B , best scheduling candidate instruction

```

1. begin
2.   for  $i \leftarrow Q_P$  do
3.     if not checkHazard( $i$ ) then
4.       remove( $Q_P$ ,  $i$ )
5.       enqueue( $Q_R$ ,  $i$ )
6.   // Elapse cycles until an instruction may
7.   // be issued.
8.   while empty( $Q_R$ ) do
9.      $C \leftarrow C + 1$ 
10.     $N_I \leftarrow 0$ 
11.    update( $T$ ,  $C$ )
12.    for  $i \leftarrow Q_P$  do
13.      if not checkHazard( $i$ ) then
14.        remove( $Q_P$ ,  $i$ )
15.        enqueue( $Q_R$ ,  $i$ )
16.   $b \leftarrow Q_R$ 
17.  if count( $Q_R$ ) > 0 then
18.    // Find best ready instruction, according to a
19.    // hard-coded set of heuristics.
20.    for  $i \in Q_R$  do
21.      if checkLeakage( $b$ ) and checkLeakage( $i$ ) then
22.         $b \leftarrow \text{compare}(b, i)$ 
23.      else if not checkLeakage( $b$ ) and not checkLeakage( $i$ ) then
24.         $b \leftarrow \text{compare}(b, i)$ 
25.      else if checkLeakage( $b$ ) then
26.        // Sub-optimal choice to minimize code size
27.        // and spilling.
28.         $b \leftarrow i$ 
29.      else
30.        // Carry on exploration of candidates.
31.  return  $b$ 

```

Algorithm 18: MachineScheduler - Main Logic (Leakage aware)

Input: F , function
Data: Q_R , queue of ready instructions
 Q_P , queue of pending instructions
 T , resource tracker
 M_{Buff} , buffer manager
 M_{Tags} , tags mapper
 N_W , maximum number of issuable instruction at each cycle
 N_I , number of instructions issued in current cycle
 C , current cycle
Output: F^* , scheduled function

```

1. begin
2.    $F^* \leftarrow []$  // Ordered list of basic blocks.
3.    $B^* \leftarrow []$  // Ordered list of instructions.
4.   for  $B \leftarrow F$  do
5.      $Q_{Reg} \leftarrow \text{getRegions}(B)$ 
6.     for  $R \leftarrow Q_{Reg}$  do
7.        $C \leftarrow 1$ 
8.        $N_I \leftarrow 0$ 
9.       for  $i \leftarrow R$  do
10.        if isReady( $i$ ) then enqueue( $Q_R$ ,  $i$ )
11.      while not empty( $R$ ) do
12.         $c \leftarrow \text{pickBest}()$  // retrieve best in  $Q_R$ .
13.        if checkLeakage( $c$ ) then
14.          applyWhiteningStrategy( $B^*$ ,  $c$ )
15.          // Schedule the selected candidate.
16.          enqueue( $R^*$ ,  $c$ )
17.          remove( $R$ ,  $c$ )
18.          // Release instructions with ready operands.
19.          for  $s \leftarrow R : \text{isSuccessor}(s, i)$  and readyOps( $s$ ) do
20.            if checkHazard( $s$ ) then
21.              enqueue( $Q_P$ ,  $c$ )
22.            else
23.              enqueue( $Q_R$ ,  $c$ )
24.          // Update  $\mu$ -buffers' state.
25.          updateBuffers( $M_{Buff}$ ,  $i$ )
26.          // Update tracked tags.
27.          updateTags( $T$ ,  $i$ )
28.          // Update scheduler's state.
29.          update( $T$ ,  $c$ )
30.           $N_I \leftarrow N_I + 1$ 
31.          if  $N_I > N_W$  then
32.            update( $T$ ,  $C$ )
33.            commitMaps( $T$ )
34.             $C \leftarrow C + 1$ 
35.             $N_I \leftarrow 0$ 
36.          enqueue( $B$ ,  $R^*$ )
37.        enqueue( $F^*$ ,  $B^*$ )
38.   return  $F^*$ 

```

Chapter 3

Performance Analysis and Results

In this conclusive chapter, the performance analysis evaluating the compiler based solution is reported in detail. A first section is dedicated to the description of the architectural target of this work, the CORTEX-A7. The CORTEX-A7 is a super-scalar in-order processor with *limited* dual-issue capability. For the analysis, the OLINUXINO A20 MICRO board, produced by the OLIMEX company, is employed. The board is endowed with the ALLWINNER A20 system-on-chip (SoC), integrating a dual-core version of the CORTEX-A7. Major implementation aspects regarding the development of the methodology are, then, detailed. In particular, how the DL TAGS have been actually integrated with the LLVM-IR and preserved through the several compilation steps, up to the machine code emission phase. In order to provide to the scheduler a correct vision of the state of the micro-architectural buffers during the re-organization of the micro-architectural buffers had to be modeled. The methodology developed requires the design of a *whitening strategy* for a specific target architecture: such strategy is employed for cleaning the micro-architectural buffers in case a leakage occurs in the pipeline. The strategy implemented for the CORTEX-A7 is detailed, illustrating it with three use cases, corresponding to the possible leakage occurrences. The second part of this chapter is dedicated to the actual performance analysis and evaluation of the results. Analysis has been carried out on different implementations of a reference implementation of AES with T-TABLES and 128 bits of key. Each implementation represents the application of a specific masking order (up to the fifth order) and application of the code-generation protections. Note that this work mainly focused on the protection against micro-architectural attacks, which are of particular interest. Protection against attacks exploiting the leakage stemming from ISA registers is performed only by avoiding consecutive assignments of the same register to shares of the same encoding; in case the allocator is not able to find a physical register, the compilation aborts. In order to guarantee reproducibility of the benchmarks, the overall settings is described, detailing the board settings employed. Results are reported in tabular and graphical form, describing the mean and standard

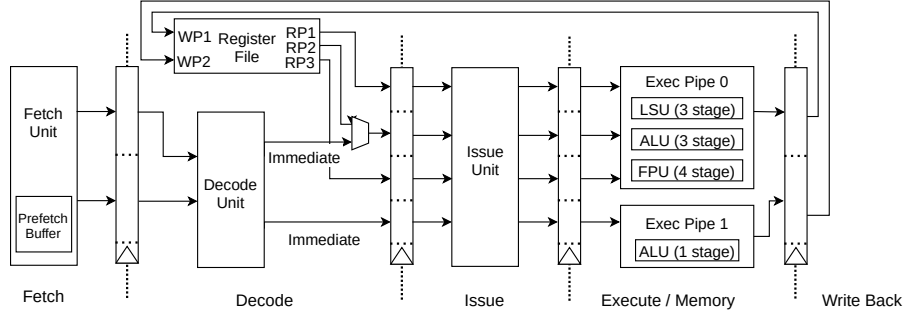


Figure 3.1: Cortex-A7 Pipeline Structure

deviation of the execution times of the various implementations, both in clock cycles and in seconds, and the size, expressed in bytes, of each implementation. A final comparison of the implied overhead on time and size is provided, reporting, for each code-generation protected implementation, the overhead expressed in percentage points with respect to a masked implementation on which no compiler protection is applied.

3.0.1 Target Architecture: Cortex-A7

The ARM CORTEX-A7 [38] is a 32-bit low-power, high performance processor, supporting from 1 to 4 cores on a single multiprocessor chip, characterized by an in-order, *partial* (sometimes referred to as *asymmetric*) dual-issue pipeline. The whole pipeline organization is reported in Figure 3.1, showing a division into 8 stages, each separated by a pipe buffer, characterized by two entries, one for each execution lane.

The dual-issue capability of the microarchitecture is limited to simultaneous execution of specific instruction patterns: an instruction consumed on the first lane may be issued together with a *register-immediate* instruction variant, or a CALL instruction. This asymmetric nature finds its reason in the actual organization of pipeline into two differently-capable execution lanes, one able to process the whole set of ARM ISA instructions, encompassing the FPU/NEON and memory instructions; the other characterized by a simple ALU unit, able to process only register-immediate ALU instructions, but not the multiplication and shift/shift-rotate ones, for the ALU not being provided with multiplication and barrel-shifter units.

As reported in [4] all the functional units are pipelined, and some of them are endowed with a dedicated buffer for temporarily store their result: the barrel-shifter and the LOAD/STORE unit; the latter is also characterized by an ALIGN buffer, employed for performing byte and halfword extraction on loaded/stored intermediates (the processor supports a 32-bit memory space).

From the leakage analysis performed in [4], and reported in Table 3.1, func-

FU	HD	HW
Is/Ex Buffer	Yes	No
Ex/Wb Buffer	Yes	No
Shift Buffer	No	Yes
Memory Data Register	Yes	No
Align Buffer	Yes	No

Table 3.1: Cortex-A7 Microarchitectural Buffers - Leakage Description.

tional units' buffers and stage buffers leaks an information content proportional to the distance of consecutively stored values.

Such scheduling related informations, along with leakage characterization of microarchitectural buffers, as discussed in Section 3.1.2, are provided to the code-generation algorithms via ad-hoc models.

3.0.2 OlinuXino A20

The Olimex OlinuXino All-Winner 20-Micro is a single board computer mounting a A20 System-on-Chip, characterized by a dual-core ARM CORTEX-A7 cpu. The board is characterized by 1 GB of DDR3 RAM MEMORY, 256KiB L2-Cache (shared between two cores) 32KiB (Instruction) / 32KiB (Data) L1-Cache per core. An embedded security system module provides the board with cryptographic acceleration capabilities, allowing execution of cryptographic algorithm on hardware. The board is endowed with hardware-based pseudo-random number generator (PRNG), requiring an initial value of 192 bits and emitting at most 160 bits of randomness, provided as 5 32 bit words. The PRNG module is provided with two execution mode: a *single-shot* mode, which allows on-demand generation of 160 bits of randomness, and a *continuous* mode, which continuously generates a 32bit of randomness.

3.1 Implementation Aspects

This section details the relevant implementation aspects of the experimental compiler based solution: DL TAGS assignments and their preservation through the compilation, micro-architectural description and ad-hoc scheduling model for the CORTEX-A7 architecture. The coded instance of the methodology is delivered as a patch for the LLVM CORE LIBS (ver. 9.0.0, commit's SHA-1: a10a70238ac). The set of modifications applied to LLVM are described, starting from the expansion of the LLVM-IR with ad-hoc *intrinsics*, their lowering into SELECTION-DAG nodes and conversion into special THUMB2 ISA instructions; some lines are spent to explain how CRYPTOSHIELD – specifically, the *masking application* stage – is modified to support the code-generation hardening. Micro-architectural model and the scheduling model of the target architecture

are described via the TABLEGEN tool, an utility, widely adopted within the LLVM CORE LIBS project, for the generation of C/C++-specifications of models provided via the TABLEGEN description language. Finally, it is provided the pseudo-code of the whitening strategy specifically created for the CORTEX-A7. The strategy reported is specific for the post-RA scheduling phase, which is of particular interest with respect to the one employed in the pre-RA scheduling phase, since the semantic preservation of the scheduled program must be guaranteed. The strategy is exemplified through three distinct examples of leakages: for each of them, it is explained how the strategy effectively removes the potential leakage via creation and scheduling of a whitening instruction.

3.1.1 Intrinsics and Tags Assignment

As explained in Section 2.1, a fundamental piece of information to enable leakage awareness in the code generation phases are the distance-leakage tags, by which means is possible to precisely identify which share an intermediate value represents, and to which encoding it belongs to.

Such kind of informations, which essentially are *metadata* characterizing a specific instruction's output, must be preserved until the end of the post register allocation phase. Although LLVM provides special data structures representing metadata, attachable to different elements of the IR, they are not guaranteed to be preserved: indeed, during the *instruction selection* stage – in particular, in the *DAG combining* phase – such metadata may be lost due to transformations operated on the representation, mining the correctness of the hardening procedures.

The solution adopted in this body of work are the so-called *intrinsics*: an intrinsic is a special kind of function, automatically inserted by the compiler when it recognizes that a specific code pattern may be efficiently implemented via target-specific instructions.

New intrinsics are totally *opaque*: that is, the backend has no knowledge on their meaning, and, consequently, is unable to carry out any transformation on them. This provides a viable mean for attaching and preserving the tags throughout the transformations passes operated by the backend. Specifically, all the LLVM-IR instructions involved in the computation of shares and, thus, require attachment of the tags to their *result*, are replaced by ad-hoc crafted intrinsic functions, each characterized by the very same return type and operands type of the mocked IR instruction. Since different type of instructions may have different inputs, tags are inserted as the first two input operands of the intrinsic to ease their identification.

Replacement with these tags-endowed intrinsics is performed within the last pass of CRYPTOSHIELD, in charge of masking vulnerable instructions: here, the instructions employed to encode a gadget are systematically substituted with the matching intrinsics, following the strategy detailed in Section 2.1.

As stated before, the opaqueness of new intrinsics is exploited to guarantee preservation of the tags; however, from the compiler's point-of-view, introduc-

tion of intrinsics is roughly equivalent to introducing new kind of instructions in the IR: the compiler must be instructed to deal with them, otherwise the compilation can't be carried on.

Usually, within LLVM, intrinsics follow the very same first steps of all the other IR instructions, from lowering up to instruction selection. However, since the tags must be preserved, new node, encompassing the tags, for the lowering phase has to be introduced. Type legalization has to be adapted to deal with the new nodes. Most of the optimizations provided by the combination steps are not provided, generating unoptimized code; nonetheless, to support the usage of specific micro-architectural features, i.e., the LOAD/STORE alignment buffer, the combiner pass is instructed to correctly combine memory operations with truncation/extension operations.

Since the output of the instruction selection phase is a serialized sequence of machine-specific instructions, the new nodes has to be matched via new low-level *pseudo* instructions. Pseudo instructions provides a mean to *virtually* expand the target's ISA, without the need of specify the actual machine encoding of the instruction self. As for the previous ad-hoc elements (intrinsic sand DAG nodes), these new pseudos are characterized by the first two input parameters to be the distance-leakage tags.

Right after the end of the code-generation phase, the ad-hoc pseudo instructions are *expanded*, i.e., converted, into target-specific instructions with the same semantic, but without the attached tags, enabling execution of final emission code stages.

3.1.2 TABLEGEN: Micro-Architectural Buffers Modelization

To transparently (with respect to the instruction scheduler) enable the definition of what are the buffers that may impact on the security of the masking based protection, the TABLEGEN module is expanded with the new **ProcBuffer** record. To link a specific buffer to its functional unit, the **ProcResourceUnits** record, defining a functional unit, is augmented with a (potentially empty) list of **ProcBuffers**. This modification allows the instruction scheduler to keep its independence on a particular back-end, while easing the life of end-user that has to bring such protection to another platform.

Concerning the pipeline's stage buffers, no modelization is performed since it is assumed to deal with an architecture similar to the generic reference (Section 2.2): (1) each execution pipeline is characterized by the same set of pipeline stage buffers, (2) the number of **ISSUE** buffer slots is the double of the execution pipelines, for assuming each μ -instruction support at most two register input operands.

Note that assumption (2) does not impact on the modelization of particular architectures, such as the CORTEX-A7, where the second issue slot may be consumed only by *register-immediate* instructions: indeed, it is sufficient to instruct the scheduler to no track the second buffer of such slot.

3.1.3 Whitening Strategy for CORTEX-A7 - Post-RA Scheduling

The whitening strategy adopted for the post-RA scheduling is structured as a small sequence of specific *instruction creation - schedule* steps: destination register spill, whitening instruction scheduling, destination register re-load. The strategy specially handles the case of leakage generated by scheduling of STR/STRB instructions. Algorithm 19 reports a pseudo-code description of the strategy: it is first checked the type of the instruction i , which causes leakage if scheduled (Line 2). In case it is a store instruction, a register containing a value which does not create leakage is retrieved; then a store instruction is created, scheduled in the basic block B where i belongs to, and the scheduling is resumed (Line 2 - 5). Such instruction stores the retrieved value and stores it into an ad-hoc stack slot s , effectively cleaning the stage buffers and the MDR where the r is stored.

If the instruction i is not a store-like instruction, then its destination register r_D is retrieved (Line 6) and saved on the ad-hoc stack slot s via creation and scheduling of an ad-hoc store instruction (Line 7). If the instruction i may be dual-issued (e.g., it is a register-immediate XOR instruction), it will be issued on the second lane of the CORTEX-A7's pipeline, preventing correct cleaning of the pipeline if the leakage happen on the first lane. In order to correctly place the whitening instruction in case leakage happens on the first lane and, at the same time, deal with the case of potential leakage on the second lane, a whitening instruction, with same opcode of i , is created and issued on the second lane (Line 8 - 11). This whitening instruction has as input operands an input register r_X such that does not cause leakage with intermediate values stored in the unique slot of the Is/EX stage buffer of the second lane. Note that the register r_D is employed by the store instruction spilling it, and the just created whitening instruction; nonetheless, the former performs a *read* of the register, while the latter a *write*, which are operations happening in two different clock cycles.

Another whitening instruction, identical to the previous one but with the input operand r_X taken such that it does not generate leakage on the first lane (Line 13).

In case the instruction i takes two registers as inputs, an additional register r_Y is retrieved and placed as input of the whitening instruction (Line 14 - 16). Finally, the original value of the destination r_D is re-loaded from the stack slot s , and the scheduling is resumed (Line 18).

To exemplify on how the leakage is correctly removed, in the following two practical use cases are described, considering the CORTEX-A7 platform. The examples deal only with the post-RA phase, for the pre-ra scheduling does not have to handle register re-uses to preserve the semantic.

Simple graphics are used to show the issue history on the pipeline, together with leakage cases, and the particular issue strategy employed to remove leakage

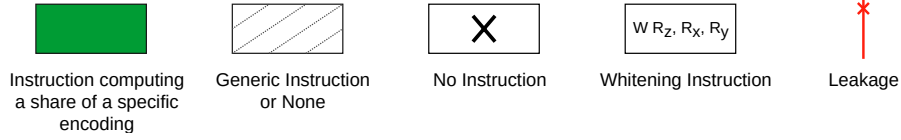


Figure 3.2: Pipeline Issue History Legend

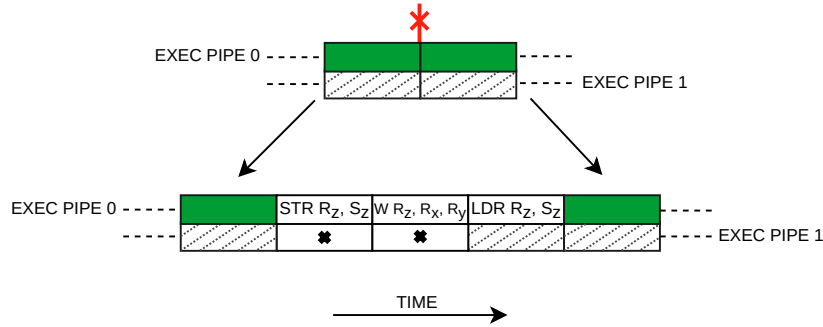


Figure 3.3: Leakage occurring on the first lane.

while preserving the semantic. Figure 3.2 reports the legend for interpreting such graphics: each rectangular represents the instruction issued at a given time, or none. The history is explained on two dimensions, where the vertical one represents the execution pipelines (increasing order, top-to-bottom), while the horizontal one represents the time (most recent issuing on the right hand).

First-Lane Leakage On the CORTEX-A7, leakage on the first lane may happen with either register-register and register-immediate instructions. The case of register-register instruction induced leakage is depicted in Figure 3.3. In such case, the leakage is easily solved, since register-register instructions may be issued only on the first execution pipeline; thus, the cleaning register-register instruction is issued for sure between the two leaking ones.

Concerning the leakage caused by two register-immediate instructions, the effect of dual-issuing should be accounted for. Indeed, as shown by Figure 3.4, the register-immediate whitening instruction is issued together with the spilling instruction¹, preventing the cleansing of the first lane's buffers. To solve this problem, the same whitening instruction, endowed with the same operands, is issued immediately after. Note that the second leaking instruction (lastest issued green block) is not issued on the second lane, although being a register-immediate operation: according to the whitening strategy employed for the CORTEX-A7, the physical register of such instruction (R_z in the picture) is employed for the cleansing instruction; writing on the same register in the same

Algorithm 19: Post-RA whitening - ARM-specific Approach

Input: i , instruction causing leakage
 B , basic block
Data: T , resource tracker
 M_{Buff} , buffer manager
 M_{Tags} , tags mapper
 s , stack slot reserved to store spilled values
Output: none

1. **begin**
2. **if** `isStore(getOpcode( $i$ ))` **then**
3. $r \leftarrow \text{getUncorrelated}(M_{Tags}, \text{getSrcReg}(i))$
4. $B \leftarrow \text{createInstr}(\text{STR}, r, s)$
5. **return**;
6. $r_D \leftarrow \text{getDstReg}(i)$
 // Save value for later re-load.
7. $B \leftarrow \text{createInstr}(\text{STR}, r_D, s)$
 // Leakage on first lane, instruction is dual-issuable.
8. **if** `issuableOnLane( $T, i, 2$ )` **then**
9. $e \leftarrow \text{getEntry}(M_{Buff}, \text{issueLane}(i), \text{IsEx}, 1)$
10. $r_X \leftarrow \text{getUncorrelated}(M_{Tags}, e)$
11. $B \leftarrow \text{createInstr}(\text{getOpcode}(i), r_D, r_X, \#0)$
12. $e \leftarrow \text{getEntry}(M_{Buff}, \text{issueLane}(i), \text{IsEx}, 1)$
13. $r_X \leftarrow \text{getUncorrelated}(M_{Tags}, e)$
14. **if** `3-addr( $i$ )` **then**
15. $e \leftarrow \text{getEntry}(M_{Buff}, \text{issueLane}(i), \text{IsEx}, 2)$
16. $r_Y \leftarrow \text{getUncorrelated}(M_{Tags}, e)$
17. $B \leftarrow \text{createInstr}(\text{getOpcode}(i), r_D, r_X, r_Y \text{ or } \#0)$
 // Re-load original value.
18. $B \leftarrow \text{createInstr}(\text{LDR}, r_D, s)$
19. **return**

clock cycle is not allowed, forcing the serialization of the instruction stream.

Second-Lane Leakage Leakage on the second lane may be caused only by register-immediate, for they are the solely to be issuable on such execution pipeline. The issuing of the whitening instruction is straight forward, since it will automatically be issued on the second lane thanks to the spilling instruction issued on the first one.

¹Reading and writing of the same physical register happen in two different clock cycles.

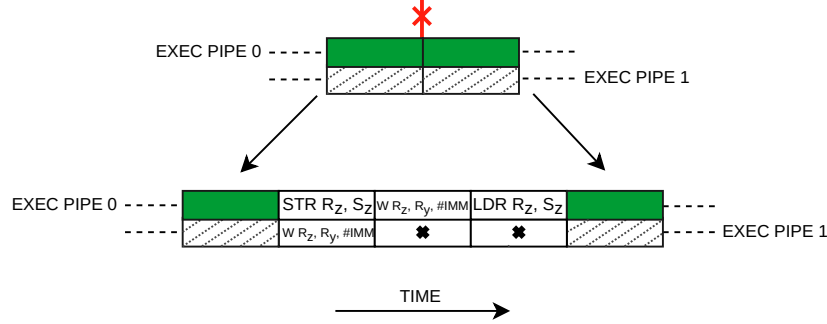


Figure 3.4: Leakage occurring on the first lane (dual-issue case).

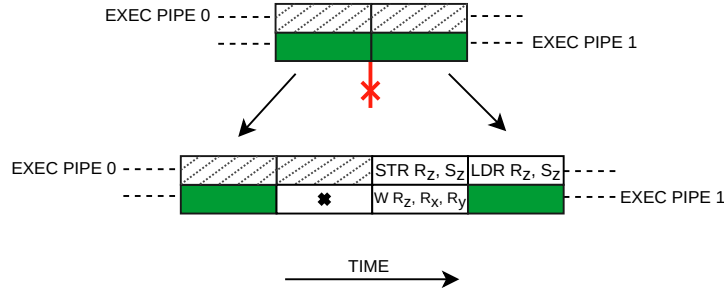


Figure 3.5: Leakage occurring on the second lane.

3.2 Performance Evaluation

This section is dedicated to the description of the performance evaluation of the compiler solution. In the first part, the experimental parameters are defined, in order to make results reproducible. In the second part, the results of the benchmarks are reported and discussed.

3.2.1 Experimental Parameters

For the execution of the experiments the CORTEX-A7 processor, integrated in the A20 SoC, is set to run with a 120 MHz frequency. The dual-issue capability of the processor has been enabled, in order to evaluate the performances of the leakage-aware MACHINE SCHEDULER. In order to have precise evaluation of the execution times, the second core of the processor has been disabled; further, some performance evaluations are performed in order to measure the effects on the timings of the instruction cache (I\$), data cache (D\$) and the prefetcher integrated in the processor. These evaluations are performed by selectively turning on/off the caches or the prefetcher, and measuring the mean execution

time and the standard deviation over 1'000 encryptions of an unmasked version of AES with T-TABLES and an encryption key of 128 bits. Table 3.2 reports the evaluations for the case of all elements turned off, only the prefetcher active, only the D\$ and I\$ active and all of the active. The data empirically show how the activation of the caches allows a drastic decrease of the mean execution time and of the standard deviation, respectively of 71.62% and 91.61%, comparing to the case of all the elements inactive, and of 71.61% and of 93.65%, comparing to the case of prefetch active and both caches inactive. Activation of all the three elements guarantees a slightly lower value in the execution time, with respect to the case of only caches actives. Nonetheless, the standard deviation is greater, implying a higher variation of the timings. Consequently, for the performance evaluation the prefetcher has been turned off, while the caches are turned on. Regarding the generation of randomness, fundamental for masked implementations, since the documentation does not provide any sort of indication on the minimum number of clock cycles required to generate a 32 bits of randomness, the *continuous mode* of the hardware prng provided by the OLINUXINO A20-MICRO is employed, enabling constant generation of fresh random values and reducing the execution time to the solely retrieving of such value from the prng module.

3.2.2 Results

The performance evaluation has been executed on several implementation of the reference C specification of the T-TABLE variant of AES with a key of 128 bits. Each implementation represents a combination of *masking order*, from order zero (i.e., unmasked version of AES), to order five, and *code-generation* protection (no protection, register allocation hardening, instruction scheduling hardening, register allocation hardening and instruction scheduling hardening). For each implementation, the mean execution time and the standard deviation over 1'000 encryptions have been recorded. For some combination of masking order and code-generation hardening, e.g., third order masking and register allocation hardening, no data is available. This because in this work the main focus is the protection from micro-architectural attacks; hence, the whitening step in the BASIC register allocator has not been implemented. As a consequence, in some situations the allocator is not able to assign a physical register without generating leakage; this causes the failure of the compilation.

Execution time Table 3.3 reports the performance figures in clock cycles, showing on the vertical axis the code-generation protections considered, and on the horizontal axis the masking order. For same masking order/code-generation protection, the mean execution time and standard deviation for a full masked implementation a keysize masked implementation is are reported. In order to give an intuition of the actual execution time for each implementation, Table 3.4 shows the same figures expressed in seconds, considering a processor clocked to 120 MHz. As expected, the overhead implied by masking, in par-

ticular masking of non-linear transformations (e.g., S-BOX look-ups), induces a blow up of the mean execution time as the masking order increases. Figure 3.6a, Figure 3.6b and Figure 3.6c report graphically depict the variation of the execution time for each code-generation protection and for no protection. The case of register allocation hardening is not reported, since only two values have been retrieved. What is really interesting to analyze are the overhead on the execution time of each code-generation protection on a given order with respect to a baseline implementation, masked at the same order but not protected via code-generation. Table 3.6 reports the relative overheads in percentage points. While for the first order there are increments in the mean execution time – up to 9% for the implementation with instruction scheduling and register allocator + instruction scheduling – for masking of order greater than one the overheads becomes really marginal, up to 1.17% for a third order masking with protection of the register allocation and instruction scheduling. Notable is the presence of negative percentage: some implementations show a reduction of the mean execution time with respect to an implementation masked at the same order, but with no protection on the code-generation side. Although some implementations show a really marginal improvement (a 0.06% overhead reduction for a second order masking with register allocation protection), other implementations shows better results, with improvements over the 0.5%, and reaching almost a 1.0% overhead reduction. This represents a first positive result for the methodology, showing how the recycling of randomness allows the code-generation protections to keep a low impact on the execution times. The presence of improvements on the overhead with respect to baseline implementation may be justified by non-optimal scheduling decision performed by the MACHINE SCHEDULER when the protection against leakage is not taken into account.

Size Concerning the impact of the protections on the size of the binary images, Table 3.5 reports the figures for the executable segment of the binary, expressed in bytes. The overall trend is an exponential increase with the masking order, which is highlighted in Figure 3.7a, Figure 3.7b and Figure 3.7c. Code-generation protections tends to exacerbate this trend. Table 3.7 reports the overhead of code-generation protections, in percentage points, with respect to a baseline implementation masked at the same order. For implementations protected only on the register allocation side, the overhead is relatively small, up to a 11.34% for a full first order masking. For such implementations, the reason of the overhead may be found in the higher register pressure implied by the additional leakage-related constraint on the allocator. Implementations protected on the instruction scheduling side exhibit a large percentage overhead, with a small increase with each masking order (less than 8%), up to a 49.15% for a fourth order masking with all code-generation protections enabled. This high increase in the size overhead is in contrast with the relative time overhead reported in Table 3.6: implementations masked with an order

Table 3.2: Performance impact on mean execution time and standard deviation, expressed in clock cycles ($\times 10^3$)

	None	Prefetcher	I\$ + D\$	All
Mean	10'624.655	10'619.694	3'014.981	3'013.623
Std. Dev	9.029	11.933	0.758	0.781

Table 3.3: Mean execution time and standard deviation, in clock cycles ($\times 10^3$), for several implementation of the T-TABLE variant of the AES. For each code-generation protection, the first row refers to an implementation masking instructions with a index vulnerability lower that the keylength; the second row refers to an implementation where all instructions are masked.

	Unmasked		First Order		Second Order		Third Order		Fourth Order		Fifth Order	
	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ
None	2.827	175.99	893.71	396.82	3'004.06	529.06	6'006.65	680.35	10'261.94	854.13	15'841.17	1'615.66
			2'268.89	547.42	7'502.87	806.73	15'045.41	1'242.90	25'641.15	1'963.06	39'692.04	6'091.64
RA	-	-	958.19	317.25	3'002.16	578.47	-	-	-	-	-	-
			2'390.99	462.29	7'498.44	864.35	-	-	-	-	-	-
IS	-	-	976.82	373.23	2'976.71	674.75	5'968.99	911.28	10'362.69	1'446.58	15'667'546	1'811.49
			2'399.14	574.03	7'433.11	1'246.57	14'948.32	2'166.42	26'011.03	4'786.07	39'338.64	7'889.97
All	-	-	977.781	466.97	3'015.64	818.12	6'076.69	1'398.72	-	-	-	-
			2'443.25	753.27	7'528.99	1'565.38	15'176.55	2'759.76	-	-	-	-

greater than 1 show lower execution time overhead and, at the same time, a high overhead on the executable segment of the binary. Generally, a large `.text` segment implies a higher amount of code to be executed; thus, a large execution time. Interestingly, for the fifth order there is an inversion of the trend, showing a lower overhead with respect to implementations of order lower or equal to four. However, since there are not enough data, it is hard to derive an explanation for this phenomenon. The implementations encompassing both code-generation protections show the highest overhead, with an increment of about 10% points for each masking order, up to a 74.18% for a third order implementation.

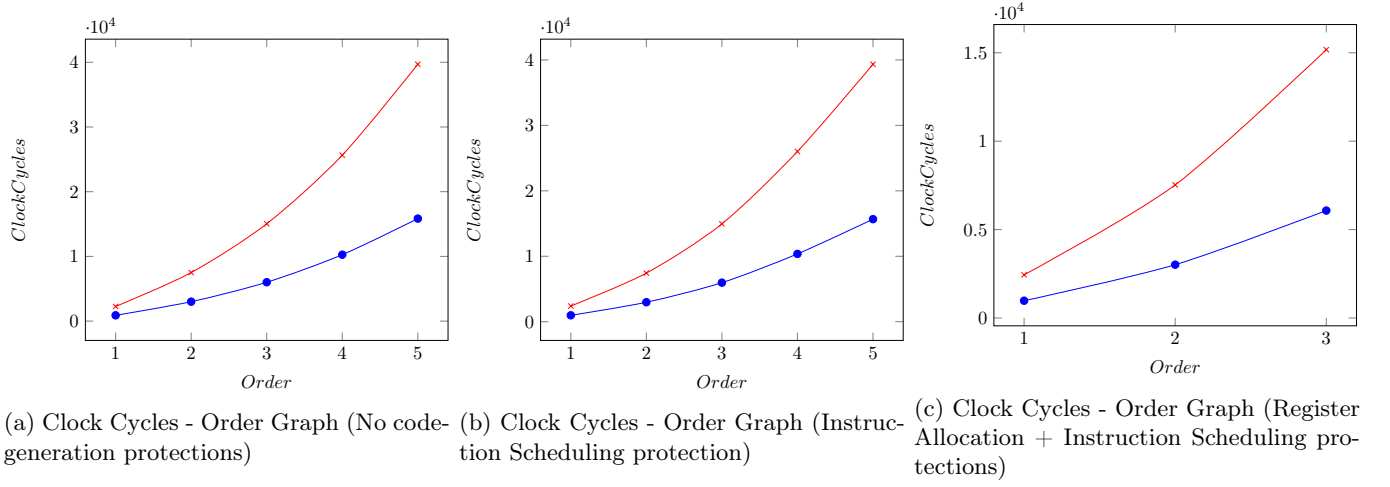


Figure 3.6: Clock cycles - Order graphs for the several code-generation protections

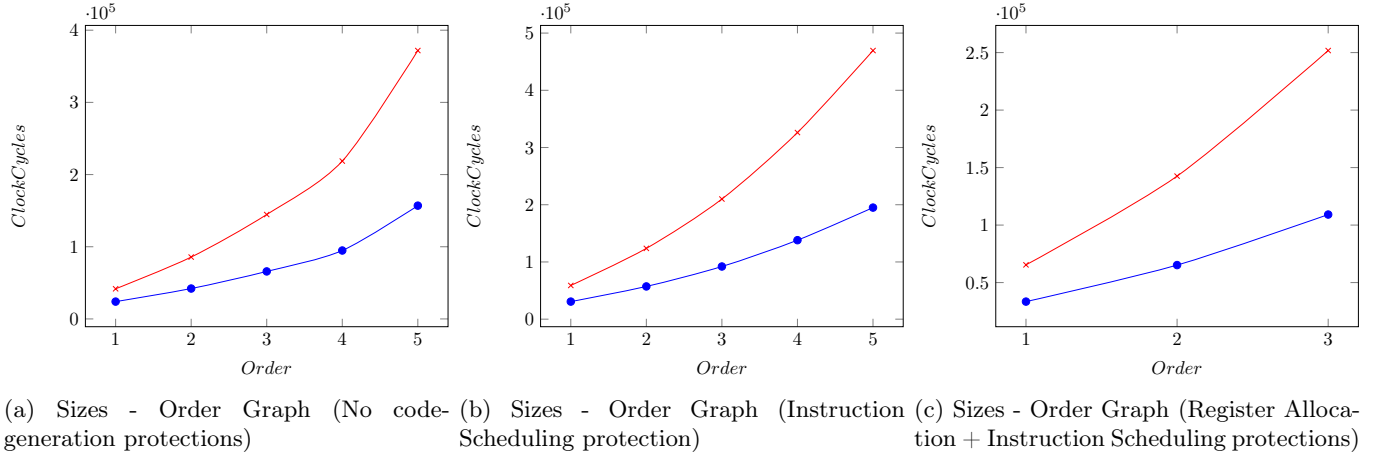


Figure 3.7: Size - Order graphs for the several code-generation protections

Table 3.4: Mean execution time and standard deviation, in seconds, for several implementation of the T-TABLE variant of the AES. For each code-generation protection, the first row refers to an implementation masking instructions with a index vulnerability lower that the keylength; the second row refers to an implementation where all instructions are masked.

	Unmasked		First Order		Second Order		Third Order		Fourth Order		Fifth Order	
	$\mu(\mu s)$	$\sigma(\mu s)$	$\mu(ms)$	$\sigma(\mu s)$	$\mu(ms)$	$\sigma(\mu s)$	$\mu(ms)$	$\sigma(\mu s)$	$\mu(ms)$	$\sigma(\mu s)$	$\mu(ms)$	$\sigma(\mu s)$
None	23.56	1.467	7.45	3.31	25.03	4.4	50.06	5.67	85.52	7.12	132.01	13.46
			18.91	5.46	62.52	6.72	125.38	10.36	213.68	16.36	330.78	50.76
RA	-	-	7.98	2.64	25.02	4.82	-	-	-	-	-	-
	-	-	19.92	3.85	62.49	7.20	-	-	-	-	-	-
IS	-	-	8.14	3.11	24.81	5.62	49.74	7.59	86.36	12.06	130.56	15.1
	-	-	19.99	4.78	61.94	10.39	124.57	18.05	216.76	39.84	327.82	65.75
All	-	-	8.15	3.89	25.13	6.82	50.64	11.66	-	-	-	-
	-	-	20.36	6.28	62.74	13.06	126.47	22.99	-	-	-	-

Table 3.5: Size of the `.text` segment for several implementation of the T-TABLE variant of the AES. For each code-generation protection, the first row refers to an implementation masking instructions with a index vulnerability lower that the keylength; the second row refers to an implementation where all instructions are masked.

	Unmasked	First Order	Second Order	Third Order	Fourth Order	Fifth Order
None	10'355	24'003	42'103	65'815	94'783	156'943
		41'751	85'703	144'635	218'683	371'819
RA	-	25'687	44'899	-	-	-
	-	46'487	93'099	-	-	-
IS	-	30'699	57'199	92'039	138'043	194'999
	-	58'919	123'719	209'975	326'155	469'479
All	-	33'451	65'207	109'155	-	-
	-	65'411	142'567	251'779	-	-

	First Order	Second Order	Third Order	Fourth Order	Fifth Order
RA	7.21	-0.06	-	-	-
	5.38	-0.06	-	-	-
IS	9.30	-0.91	-0.63	0.98	-1.09
	5.74	-0.93	-0.65	1.44	-0.89
All	9.41	0.39	1.17	-	-
	7.69	0.35	0.87	-	-

Table 3.6: Relative % overhead (mean execution time) of the several code-generation protections. The figures represent the overhead of the specific implementation with respect to the masked implementation but with no code-generation protection. For each implementation, the mean execution time for a full masked implementation and an implementation masking only instructions with a vulnerability index lower than the keysize are reported.

Table 3.7: Relative % overhead (size of `.text` segment) of the several code-generation protections. The figures represent the overhead of the specific implementation with respect to the masked implementation but with no code-generation protection. For each implementation, the size of `.text` segment for a full masked implementation and an implementation masking only instructions with a vulnerability index lower than the keysize are reported.

	First Order	Second Order	Third Order	Fourth Order	Fifth Order
RA	7.01	6.64	-	-	-
	11.34	8.62	-	-	-
IS	27.89	35.85	39.85	45.64	24.25
	41.12	44.36	45.18	49.15	26.26
All	39.94	54.87	65.85	-	-
	56.94	66.35	74.08	-	-

Conclusions and Future Works

Masking is an effective countermeasure against passive side-channel attacks targetting cryptoschemes: by splitting a sensitive value in $n + 1$ random values, called *shares*, it is able to thwart attackers trying to exfiltrate secrets from the device running the implementation of the cipher. Nonetheless, the security of masked software implementations is threatened by attacks targetting the leakage stemming from micro-architectural features of the platform. In this work, I presented a novel methodology for the employment of code-generation algorithms, which are embedded in modern compilers, as a mean to automatically and efficiently protect masked software implementation of cryptoschemes against side-channel attacks specifically targeting the leakage stemming from the switching activity of micro-architectural registers. With a primary focus on the protection on the micro-architectural side, the methodology avoids potential leakages identifying pairs of shares, which consecutive store in the same register leak an amount of information dependent on the masked sensitive value, and forbids particular allocations of registers and re-organizations of the code. In case the instruction scheduler is not able to find a scheduling candidate due to possible leakage stemming from the micro-architectural buffers, a *whitening* step, in charge of cleaning the micro-architectural buffers where leakage may occur, prevents information disclosure and compilation failures. The whitening strategy is implemented via *recycling* randomness represented by shares available in the physical registers of the target architecture. Such strategy has several benefits, since the randomness cost of the protection is identical to the one of the masking schemes employed; furthermore both the register pressure, and the need to schedule non-functional instructions are minimized; as a consequence, the overall overhead on the execution times is minimized. The methodology is implemented as an extension of the CRYPTOSHIELD tool, a LLVM-based library for the automatic application of partial masking on symmetric cryptoschemes. Performance evaluations performed on the CORTEX-A7 processor, a superscalar in-order architecture with *limited* dual-issue capabilities, empirically confirm the efficiency of the methodology, evaluating the mean execution time of several protected implementations of a reference implementation of the T-TABLE variant of the AES cipher. Interestingly, the implementations protected on the code-generation side show a marginal overhead on the mean execution times with respect to a baseline version, unprotected from the

code-generation point of view; in several cases, the implementations exhibit a lower execution time, which may be justified by non-optimal scheduling decisions taken by the scheduler when protection against leakage is not considered. The overheads on the size of the emitted code are non negligible (+39% – +74%), however, the entire code segment is still below 512 kiB even for a fifth order protected cipher. I conclude noting that the field of automated protection against micro-architectural attack is really new, and few works take in consideration the protection of masked software implementations. To the best of my knowledge, this work is the first of its kind to employ a production-grade compiler to protect masked implementations against micro-architectural attacks against a superscalar architecture with multiple issue capability, although being limited. Several future developments are possible: as an example recent works developed efficient methods to automatically determine if a set of shares is statistically dependent on a secret value. Such methods may be integrated in the current tools for automatically determine which pair of intermediate values should not consecutively share the same register, which, currently, may be verified only by hand. Furthermore, currently the implementation relies on the creation of ad-hoc intrinsics to identify the shares of masked transformations: the LLVM backend is not able to automatically handle transformations on such elements, forbidding any optimization. An interesting development would be the identification of another method to identify such shares, without sacrificing the potential for further optimizations. However, being the security of the implementation the paramount concern, a conservative approach was adopted in this regard. Currently, SIMD instructions are not considered in the developed tool, which may represent an interesting extension, providing high-throughput implementation of masked schemes.

Bibliography

- [1] Giovanni Agosta et al. “Compiler-based Side Channel Vulnerability Analysis and Optimized Countermeasures Application”. In: *Proceedings of the 50th Annual Design Automation Conference*. DAC '13. Austin, Texas: ACM, 2013, 81:1–81:6. ISBN: 978-1-4503-2071-9. DOI: 10.1145/2463209.2488833. URL: <http://doi.acm.org/10.1145/2463209.2488833>.
- [2] Josep Balasch et al. “On the Cost of Lazy Engineering for Masked Software Implementations”. In: *Smart Card Research and Advanced Applications*. Ed. by Marc Joye and Amir Moradi. Cham: Springer International Publishing, 2014, pp. 64–81. ISBN: 978-3-319-16763-3.
- [3] A. Barenghi and G. Pelosi. “On the security of partially masked software implementations”. In: *2014 11th International Conference on Security and Cryptography (SECRYPT)*. 2014, pp. 1–8.
- [4] A. Barenghi and G. Pelosi. “Side-channel security of superscalar CPUs : Evaluating the Impact of Micro-architectural Features”. In: *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. June 2018, pp. 1–6. DOI: 10.1109/DAC.2018.8465784.
- [5] Gilles Barthe et al. *maskVerif: automated analysis of software and hardware higher-order masked implementations*. Cryptology ePrint Archive, Report 2018/562. 2018. URL: <https://eprint.iacr.org/2018/562>.
- [6] Gilles Barthe et al. “Parallel Implementations of Masking Schemes and the Bounded Moment Leakage Model”. In: *Advances in Cryptology – EUROCRYPT 2017*. Ed. by Jean-Sébastien Coron and Jesper Buus Nielsen. Cham: Springer International Publishing, 2017, pp. 535–566. ISBN: 978-3-319-56620-7.
- [7] Gilles Barthe et al. *Strong Non-Interference and Type-Directed Higher-Order Masking*. Cryptology ePrint Archive, Report 2015/506. 2015. URL: <https://eprint.iacr.org/2015/506>.
- [8] Sonia Belaïd et al. “Randomness Complexity of Private Circuits for Multiplication”. In: *Advances in Cryptology – EUROCRYPT 2016*. Ed. by Marc Fischlin and Jean-Sébastien Coron. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 616–648. ISBN: 978-3-662-49896-5.

- [9] Shivam Bhasin et al. “A Low-Entropy First-Degree Secure Provable Masking Scheme for Resource-Constrained Devices”. In: *Proceedings of the Workshop on Embedded Systems Security*. WESS’13. Montreal, Quebec, Canada: Association for Computing Machinery, 2013. ISBN: 9781450321457. DOI: 10.1145/2527317.2527324. URL: <https://doi.org/10.1145/2527317.2527324>.
- [10] Beül Bilgin et al. *Higher-Order Threshold Implementations*. Cryptology ePrint Archive, Report 2014/751. 2014. URL: <https://eprint.iacr.org/2014/751>.
- [11] Roderick Bloem et al. “Formal Verification of Masked Hardware Implementations in the Presence of Glitches”. In: *Advances in Cryptology – EUROCRYPT 2018*. Ed. by Jesper Buus Nielsen and Vincent Rijmen. Cham: Springer International Publishing, 2018, pp. 321–353. ISBN: 978-3-319-78375-8.
- [12] Claude Carlet et al. “Higher-Order Masking Schemes for S-Boxes”. In: *Fast Software Encryption*. Ed. by Anne Canteaut. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 366–384. ISBN: 978-3-642-34047-5.
- [13] Suresh Chari et al. “Towards Sound Approaches to Counteract Power-Analysis Attacks”. In: *Advances in Cryptology — CRYPTO’ 99*. Ed. by Michael Wiener. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 398–412. ISBN: 978-3-540-48405-9.
- [14] Jean-Sébastien Coron. “Higher Order Masking of Look-Up Tables”. In: *Advances in Cryptology – EUROCRYPT 2014*. Ed. by Phong Q. Nguyen and Elisabeth Oswald. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 441–458. ISBN: 978-3-642-55220-5.
- [15] Jean-Sébastien Coron, Johann Großschädl, and Praveen Kumar Vadnala. “Secure Conversion between Boolean and Arithmetic Masking of Any Order”. In: *Cryptographic Hardware and Embedded Systems – CHES 2014*. Ed. by Lejla Batina and Matthew Robshaw. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 188–205. ISBN: 978-3-662-44709-3.
- [16] Jean-Sébastien Coron, Johann Großschädl, and Praveen Kumar Vadnala. “Secure conversion between boolean and arithmetic masking of any order”. In: *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer. 2014, pp. 188–205.
- [17] Jean-Sébastien Coron, Emmanuel Prouff, and Matthieu Rivain. “Side Channel Cryptanalysis of a Higher Order Masking Scheme”. In: *Cryptographic Hardware and Embedded Systems - CHES 2007*. Ed. by Pascal Paillier and Ingrid Verbauwhede. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 28–44. ISBN: 978-3-540-74735-2.

- [18] Jean-Sebastien Coron, Franck Rondepierre, and Rina Zeitoun. *High Order Masking of Look-up Tables with Common Shares*. Cryptology ePrint Archive, Report 2017/271. 2017. URL: <https://eprint.iacr.org/2017/271>.
- [19] Jean-Sébastien Coron et al. “Conversion from arithmetic to boolean masking with logarithmic complexity”. In: *International Workshop on Fast Software Encryption*. Springer. 2015, pp. 130–149.
- [20] Jean-Sébastien Coron et al. “Conversion of Security Proofs from One Leakage Model to Another: A New Issue”. In: *Constructive Side-Channel Analysis and Secure Design*. Ed. by Werner Schindler and Sorin A. Huss. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 69–81. ISBN: 978-3-642-29912-4.
- [21] Jean-Sébastien Coron et al. “Faster Evaluation of SBoxes via Common Shares”. In: *Cryptographic Hardware and Embedded Systems – CHES 2016*. Ed. by Benedikt Gierlichs and Axel Y. Poschmann. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 498–514. ISBN: 978-3-662-53140-2.
- [22] Jean-Sébastien Coron et al. “Higher-Order Side Channel Security and Mask Refreshing”. In: *Fast Software Encryption*. Ed. by Shiho Moriai. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 410–424. ISBN: 978-3-662-43933-3.
- [23] J. Deogirikar and A. Vidhate. “Security attacks in IoT: A survey”. In: *2017 International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*. Feb. 2017, pp. 32–37. DOI: 10.1109/I-SMAC.2017.8058363.
- [24] J. Dofe, J. Frey, and Q. Yu. “Hardware security assurance in emerging IoT applications”. In: *2016 IEEE International Symposium on Circuits and Systems (ISCAS)*. May 2016, pp. 2050–2053. DOI: 10.1109/ISCAS.2016.7538981.
- [25] Alexandre Duc, Stefan Dziembowski, and Sebastian Faust. “Unifying Leakage Models: From Probing Attacks to Noisy Leakage.” In: *Advances in Cryptology – EUROCRYPT 2014*. Ed. by Phong Q. Nguyen and Elisabeth Oswald. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 423–440. ISBN: 978-3-642-55220-5.
- [26] S. Dziembowski and K. Pietrzak. “Leakage-Resilient Cryptography”. In: *2008 49th Annual IEEE Symposium on Foundations of Computer Science*. Oct. 2008, pp. 293–302. DOI: 10.1109/FOCS.2008.56.
- [27] Hassan Eldib and Chao Wang. “Synthesis of Masking Countermeasures against Side Channel Attacks”. In: *Computer Aided Verification*. Ed. by Armin Biere and Roderick Bloem. Cham: Springer International Publishing, 2014, pp. 114–130. ISBN: 978-3-319-08867-9.

- [28] Hannes Gross and Stefan Mangard. “A unified masking approach”. In: *Journal of Cryptographic Engineering* 8.2 (June 2018), pp. 109–124. ISSN: 2190-8516. DOI: 10.1007/s13389-018-0184-y. URL: <https://doi.org/10.1007/s13389-018-0184-y>.
- [29] Hannes Gross and Stefan Mangard. “Reconciling $d + 1$ Masking in Hardware and Software”. In: *Cryptographic Hardware and Embedded Systems – CHES 2017*. Ed. by Wieland Fischer and Naofumi Homma. Cham: Springer International Publishing, 2017, pp. 115–136. ISBN: 978-3-319-66787-4.
- [30] Hannes Gross, Stefan Mangard, and Thomas Korak. “Domain-Oriented Masking: Compact Masked Hardware Implementations with Arbitrary Protection Order”. In: *Proceedings of the 2016 ACM Workshop on Theory of Implementation Security. TIS ’16*. Vienna, Austria: ACM, 2016, pp. 3–3. ISBN: 978-1-4503-4575-0. DOI: 10.1145/2996366.2996426. URL: <http://doi.acm.org/10.1145/2996366.2996426>.
- [31] Vincent Grosso, Emmanuel Prouff, and François-Xavier Standaert. “Efficient Masked S-Boxes Processing – A Step Forward –”. In: *Progress in Cryptology – AFRICACRYPT 2014*. Ed. by David Pointcheval and Damien Vergnaud. Cham: Springer International Publishing, 2014, pp. 251–266. ISBN: 978-3-319-06734-6.
- [32] D. Helms, E. Schmidt, and W. Nebel. “Leakage in CMOS Circuits – An Introduction”. In: *Integrated Circuit and System Design. Power and Timing Modeling, Optimization and Simulation*. Ed. by Enrico Macii, Vassilis Paliouras, and Odysseas Koufopavlou. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 17–35. ISBN: 978-3-540-30205-6.
- [33] Yuval Ishai, Amit Sahai, and David Wagner. “Private Circuits: Securing Hardware against Probing Attacks”. In: *Advances in Cryptology – CRYPTO 2003*. Ed. by Dan Boneh. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 463–481. ISBN: 978-3-540-45146-4.
- [34] Arun Kanuparthi, Ramesh Karri, and Sateesh Addepalli. “Hardware and Embedded Security in the Context of Internet of Things”. In: *Proceedings of the 2013 ACM Workshop on Security, Privacy & Dependability for Cyber Vehicles. CyCAR ’13*. Berlin, Germany: Association for Computing Machinery, 2013, pp. 61–64. ISBN: 9781450324878. DOI: 10.1145/2517968.2517976. URL: <https://doi.org/10.1145/2517968.2517976>.
- [35] Mohamed Karroumi, Benjamin Richard, and Marc Joye. “Addition with blinded operands”. In: *International Workshop on Constructive Side-Channel Analysis and Secure Design*. Springer. 2014, pp. 41–55.
- [36] Paul Kocher, Joshua Jaffe, and Benjamin Jun. “Differential Power Analysis”. In: *Advances in Cryptology — CRYPTO’ 99*. Ed. by Michael Wiener. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 388–397. ISBN: 978-3-540-48405-9.

- [37] Chris Lattner and Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation”. In: *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO’04)*. Palo Alto, California, Mar. 2004.
- [38] ARM Ltd. *Cortex-A7 MPCore Technical Reference Manual*. 2011. URL: <https://developer.arm.com/docs/ddi0464/f/preface> (visited on 04/02/2020).
- [39] P. Luo et al. “Compiler-Assisted Threshold Implementation against Power Analysis Attacks”. In: *2017 IEEE International Conference on Computer Design (ICCD)*. Nov. 2017, pp. 541–544. DOI: 10.1109/ICCD.2017.94.
- [40] H. Maghrebi et al. “Register leakage masking using Gray code”. In: *2012 IEEE International Symposium on Hardware-Oriented Security and Trust*. June 2012, pp. 37–42. DOI: 10.1109/HST.2012.6224316.
- [41] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power Analysis Attacks*. Boston, MA: Springer US, 2007. ISBN: 978-0-387-30857-6. DOI: 10.1007/978-0-387-38162-6. URL: <https://www.springer.com/us/book/9780387308579>.
- [42] Stefan Mangard, Thomas Popp, and Berndt M. Gammel. “Side-Channel Leakage of Masked CMOS Gates”. In: *Topics in Cryptology – CT-RSA 2005*. Ed. by Alfred Menezes. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 351–365. ISBN: 978-3-540-30574-3.
- [43] Stefan Mangard and Kai Schramm. “Pinpointing the Side-Channel Leakage of Masked AES Hardware Implementations”. In: *Cryptographic Hardware and Embedded Systems – CHES 2006*. Ed. by Louis Goubin and Mitsuru Matsui. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 76–90. ISBN: 978-3-540-46561-4.
- [44] Andrew Moss et al. “Compiler Assisted Masking”. In: *Cryptographic Hardware and Embedded Systems – CHES 2012*. Ed. by Emmanuel Prouff and Patrick Schaumont. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 58–75. ISBN: 978-3-642-33027-8.
- [45] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. “Threshold Implementations Against Side-Channel Attacks and Glitches”. In: *Information and Communications Security*. Ed. by Peng Ning, Sihan Qing, and Ninghui Li. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 529–545. ISBN: 978-3-540-49497-3.
- [46] NIST. *Advanced Encryption Standard*. 2001. URL: <https://www.nist.gov/publications/advanced-encryption-standard-aes> (visited on 04/02/2020).

- [47] Emmanuel Prouff and Matthieu Rivain. “Masking against Side-Channel Attacks: A Formal Security Proof”. In: *Advances in Cryptology – EURO-CRYPT 2013*. Ed. by Thomas Johansson and Phong Q. Nguyen. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 142–159. ISBN: 978-3-642-38348-9.
- [48] Mirza Abdur Razzaq et al. “Security Issues in the Internet of Things (IoT): A Comprehensive Study”. In: *International Journal of Advanced Computer Science and Applications* 8.6 (2017). DOI: 10.14569/IJACSA.2017.080650. URL: <http://dx.doi.org/10.14569/IJACSA.2017.080650>.
- [49] Oscar Reparaz. *A note on the security of Higher-Order Threshold Implementations*. Cryptology ePrint Archive, Report 2015/001. 2015. URL: <https://eprint.iacr.org/2015/001>.
- [50] Oscar Reparaz et al. *Consolidating masking schemes*. Cryptology ePrint Archive, Report 2015/719. 2015.
- [51] Matthieu Rivain and Emmanuel Prouff. “Provably Secure Higher-Order Masking of AES”. In: *Cryptographic Hardware and Embedded Systems, CHES 2010*. Ed. by Stefan Mangard and François-Xavier Standaert. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 413–427. ISBN: 978-3-642-15031-9.
- [52] Stefano Sanfilippo. “Compiler-based Technique for Automated Analysis and Protection against Side-Channel Attack”. MA thesis. Politecnico di Milano, 2016. URL: <https://www.politesi.polimi.it/bitstream/10589/126206/3/manuscript.pdf>.
- [53] Kai Schramm and Christof Paar. “Higher Order Masking of the AES”. In: *Topics in Cryptology – CT-RSA 2006*. Ed. by David Pointcheval. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 208–225. ISBN: 978-3-540-32648-9.
- [54] Hermann Seuschek, Fabrizio De Santis, and Oscar M. Guillen. “Side-channel Leakage Aware Instruction Scheduling”. In: *Proceedings of the Fourth Workshop on Cryptography and Security in Computing Systems*. CS2 ’17. Stockholm, Sweden: ACM, 2017, pp. 7–12. ISBN: 978-1-4503-4869-0. DOI: 10.1145/3031836.3031838. URL: <http://doi.acm.org/10.1145/3031836.3031838>.
- [55] Adi Shamir. “How to Share a Secret”. In: *Commun. ACM* 22.11 (Nov. 1979), pp. 612–613. ISSN: 0001-0782. DOI: 10.1145/359168.359176. URL: <http://doi.acm.org/10.1145/359168.359176>.
- [56] Madura A Shelton et al. *Rosita: Towards Automatic Elimination of Power-Analysis Leakage in Ciphers*. Cryptology ePrint Archive, Report 2019/1445. 2019. URL: <https://eprint.iacr.org/2019/1445>.

- [57] François-Xavier Standaert. “Introduction to Side-Channel Attacks”. In: *Secure Integrated Circuits and Systems*. Ed. by Ingrid M.R. Verbauwhede. Boston, MA: Springer US, 2010, pp. 27–42. ISBN: 978-0-387-71829-3. DOI: 10.1007/978-0-387-71829-3_2. URL: https://doi.org/10.1007/978-0-387-71829-3_2.
- [58] M. Tang et al. “A Generic Table Recomputation-Based Higher-Order Masking”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 36.11 (Nov. 2017), pp. 1779–1789. ISSN: 1937-4151. DOI: 10.1109/TCAD.2017.2661823.
- [59] M. Tang et al. “PFD – A Flexible Higher-Order Masking Scheme”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 36.8 (Aug. 2017), pp. 1327–1339. ISSN: 1937-4151. DOI: 10.1109/TCAD.2016.2629441.
- [60] H. Tschofenig and E. Baccelli. “Cyberphysical Security for the Masses: A Survey of the Internet Protocol Suite for Internet of Things Security”. In: *IEEE Security Privacy* 17.5 (Sept. 2019), pp. 47–57. ISSN: 1558-4046. DOI: 10.1109/MSEC.2019.2923973.
- [61] Jingbo Wang, Chungha Sung, and Chao Wang. “Mitigating Power Side Channels during Compilation”. In: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2019. Tallinn, Estonia: Association for Computing Machinery, 2019, pp. 590–601. ISBN: 9781450355728. DOI: 10.1145/3338906.3338913. URL: <https://doi.org/10.1145/3338906.3338913>.
- [62] Z. Zhang et al. “IoT Security: Ongoing Challenges and Research Opportunities”. In: *2014 IEEE 7th International Conference on Service-Oriented Computing and Applications*. Nov. 2014, pp. 230–234. DOI: 10.1109/SOCA.2014.58.